

ICC2

3

^

T

~

~

~

~

目录

一、ICC2 基础数据及库	11
1. Libraries and Blocks.....	11
1). Design Libraries	11
2). Reference Library List	13
3). Technology Data	13
4). Blocks	14
5). Block Types	15
6). Sparse (稀疏、稀少) Libraries	15
7). File Attachments.....	17
8). Library Packaging.....	17
2. Data Import and Export.....	19
1). Reading a Verilog Netlist	19
2). Allowing Inconsistent Design Data	19
3). Saving a Design in ASCII Format	21
4). ICC2 Layer Mapping File	21
5). ICC Layer Mapping File Syntax.....	22
6). Changing Object Names	23
3. Working With Design Data	25
1). Application Options.....	25
2). Objects and Collections.....	26
3). Bus and Name Expansion.....	26
4). Design Hierarchy	27
5). Technology Data Access	27
6). Reporting Design Information.....	27
7). Polygon (多边形) Manipulation	28
8). Undoing and Redoing Changes to the Design	30
9). Schema (图解、概要) Versions (略)	31
二、ICC2 SDC 格式	31
1. 基础指令:	32
2. Object Access 指令:	32
三、内嵌 TCL.....	34
1. 工具 TCL 指令	35
2. 基本 TCL 指令	35
3. 流控制命令	36
4. 自定义函数.....	37
四、ICC2 Implementation	37
1. 准备阶段.....	38
1). 基本说明.....	38
2). 新概念 (先进节点)	39
(1). 基础概念:	39
(2). Multiple-Patterning Concepts	40
3). 调用基本命令	41
4). 设置 search_path.....	41
5). Design library 类型说明.....	41
6). Floorplan 基础说明	42
7). 时序基础说明	42
8). 电地基础说明	43

(1).	创建逻辑电地连接	43
(2).	检查电地定义	43
9).	优化基础说明	43
(1).	设置单元的使/禁用	43
(2).	设置 CTS/CTO 单元选项	43
(3).	阻止优化	43
(4).	限制单元优化	44
(5).	限制 size 时 pin name	44
(6).	对输入/输出隔离	44
10).	翻转率基本说明	44
11).	绕线资源基本说明	44
(1).	全局约束	44
(2).	Net 约束	45
(3).	时钟树约束	45
(4).	优先绕线方向约束	45
12).	并行基础说明	45
(1).	管理多进程（多核同一机器）	45
(2).	管理分布式进程（不同机器）	46
2.	Place	47
1).	约束	47
(1).	Placement Blockages	47
(2).	Placement Bounds	47
(3).	Voltage Areas	47
(4).	Guard Bands	49
(5).	Density Constraints	49
(6).	Cell Spacing Constraints	49
(7).	Inserting Multivoltage Cells	49
(8).	Marking the Clock Networks	50
2).	执行及优化	50
(1).	主流程简介	50
(2).	Power 优化简介	51
(3).	Scan Chain 优化简介	52
(4).	Preroute 阶段自动使用 Nondefault Routing Rules 优化	52
(5).	定义优化单元的前缀名	52
(6).	SPG 流程（使用 DCG 的 Physical Guidance）	52
(9).	执行 Multibit Banking	53
(10).	Incremental Placement 及 Optimization	54
(11).	执行 Magnet Placement	54
(12).	Refining Placement	54
(13).	处理多电压域模块	54
3).	分析	55
(1).	使用属性 “status”	55
(2).	报告命令	55
(3).	GUI 分析	55
(4).	时序分析	55
(5).	功耗分析	56
3.	CTS	56
1).	CTS 前提条件（准备）	56

2).	分析时钟基本结构.....	56
3).	时钟定义.....	56
4).	检查.....	58
5).	Logic DRC 设置.....	58
6).	CTS 通用设置.....	59
7).	报告时钟设置.....	60
8).	综合时钟树.....	60
9).	时钟树优化.....	61
10).	平衡不同时钟.....	61
11).	CTS 后优化.....	62
12).	时钟详尽绕线.....	62
13).	创建 Clock Straps.....	62
14).	绕线连接 Clock Drivers 和 Sinks 到 Clock Straps.....	63
15).	绕线后优化.....	64
16).	标注已存在时钟树.....	64
17).	删除时钟树.....	65
18).	分析时钟树.....	65
4.	Zroute.....	66
1).	基本流程.....	66
2).	准备检查.....	66
4).	定义 Vias.....	67
5).	检查 Routability.....	67
6).	Routing 约束.....	67
7).	绕线选项基本分类.....	70
8).	多电压域设计.....	70
9).	对 Clock Nets 绕线.....	70
10).	对 Critical Nets 绕线.....	71
11).	Routing Secondary Power and Ground Pins.....	71
12).	对 Signal Nets 绕线.....	71
13).	屏蔽线.....	72
(1).	独立设置屏蔽.....	72
(2).	绕线过程中屏蔽.....	73
(3).	绕线后屏蔽.....	73
(4).	示例脚本.....	74
(5).	检查.....	74
14).	绕线后优化.....	74
15).	ECO 绕线.....	75
16).	GUI 操作.....	75
17).	删除冗余绕线.....	75
18).	分析绕线结果.....	76
(1).	拥塞报告.....	76
(2).	DRC 报告.....	76
(3).	Signoff DRC 检查.....	76
(4).	LVS (Layout-Versus-Schematic) 检查.....	76
(5).	整体报告.....	76
19).	保存 route 信息.....	77
5.	DFM.....	77
1).	插 Tap Cells.....	77

2).	插 Boundary Cells.....	77
3).	修复天线效应	78
(1).	定义天线效应约束	78
(2).	天线效应属性	81
(3).	分析、修复天线效应	82
4).	打多孔	83
(1).	时钟线打多孔	83
(2).	信号线打多孔	83
5).	优化线长、孔数.....	85
6).	降低关键区域（Critical Areas）	85
7).	插 Filler Cells	86
8).	插 Metal Fill	86
6.	IC Validator	86
1).	ICV 准备设置	86
2).	Signoff DRC 检查	86
3).	自动修复 DRC violation	88
4).	插 Metal Filler	88
5).	自动修复孤立孔	88
7.	使用 Relative Placement 处理 Physical Datapath	89
8.	顶层收敛.....	89
1).	Abstract Views 流程.....	89
2).	Frame View 流程.....	89
3).	使用 Abstracts 顶层收敛.....	89
4).	PT 创建 ETMs.....	90
5).	在 Library Manage 中创建 ETM Reference Libraries.....	91
6).	ETM 流程	91
9.	ECO.....	91
1).	未流片 ECO	92
2).	已流片 ECO	92
3).	Signoff ECO	92
4).	Incremental Signoff ECO.....	93
5).	手动例化 Spare Cells	93
6).	自动例化 Spare Cells	93
7).	Programmable 添加 Spare Cells	93
8).	使用“eco_netlist”对网表进行修改.....	94
9).	网表修改相关指令.....	94
10).	摆放 ECO Cells.....	94
11).	Adding Buffers to Drive Specific Loads of a Nets.....	95
五、	Design Planing	96
1.	提取约束.....	97
1).	基本流程.....	97
2).	示例.....	97
3).	平行处理任务	98
4).	监控分布式提交进程	99
2.	创建 Floorplan	99
3.	黑盒.....	101
1).	识别.....	101
2).	创建黑盒 reference	101

3).	创建黑盒时序 model.....	101
4).	示例.....	101
4.	规划 IO 及 Flip-Chip Bumps.....	102
1).	创建 IO ring/guide	102
2).	创建 Bump 阵列	103
3).	预设位置摆放 bump	103
4).	IO 约束	104
5).	DRL 绕线	104
5.	管理底层模块.....	105
1).	分析逻辑层次.....	105
2).	创建 module 边界	105
3).	提交成模块.....	106
4).	创建 placement 用 abstract.....	106
5).	调整模块原点.....	106
6.	电压设计.....	107
7.	全局规划.....	107
1).	创建 Bundle（束）约束	107
2).	使用绕线优先通道（Corridor）	108
3).	添加 Repeater cells.....	108
4).	将 Repeater cells 应用到模块中（push down）	109
8.	Pin 规划.....	109
1).	全局约束.....	110
2).	分立约束.....	110
3).	读取约束.....	111
(1).	拓扑图.....	111
(2).	控制反馈（Feedthrough）	112
(3).	Pin 物理约束.....	112
(4).	Pin 间距约束.....	112
(5).	模块 Pin 约束.....	113
4).	拓扑约束示例.....	113
5).	对 pin 的 Global Routing.....	114
6).	Channel Congestion Map	114
7).	Placing Pins.....	114
8).	检查 Pin Placement.....	115
9).	写出反馈型绕线指令	115
10).	顶层/模块中交互	115
9.	Timing Budget	116
1).	创建 Block Timing Abstracts	116
2).	虚拟局部（Virtual In-Place）优化	117
3).	自定义 Budget 约束	118
4).	刷新 Budget 信息	118
5).	写出 Budget 设置	119
10.	时钟主干（clock trunk）规划.....	120
1).	为时钟数规划的虚拟局部优化.....	120
2).	指定 Derates 和时钟 Endpoints	120
3).	执行时钟树分析.....	120
4).	更新时钟 Pin 位置.....	121
5).	在顶层插时钟树 Buffer.....	121

6).	写出时钟树位置.....	122
-----	--------------	-----

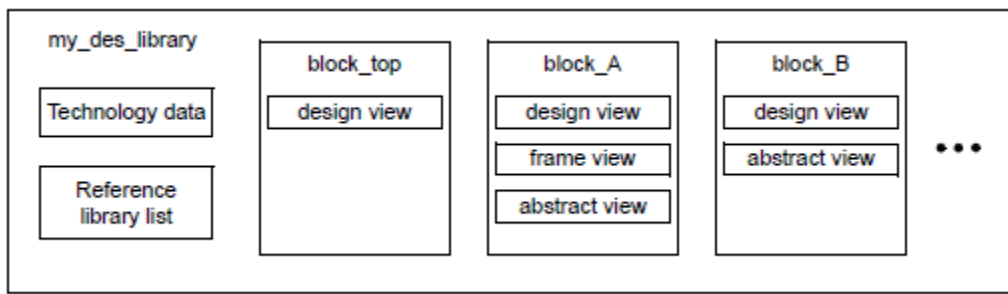
一、ICC2 基础数据及库

1. Libraries and Blocks

模块的完整信息包括物理和功能设计数据。单元库（library）包括相关模块信息及相关工艺信息。顶层设计通常包含多个模块，每个模块通常都有单独的设计库。低层次的单元库可以作为另一单元库的 reference 库。

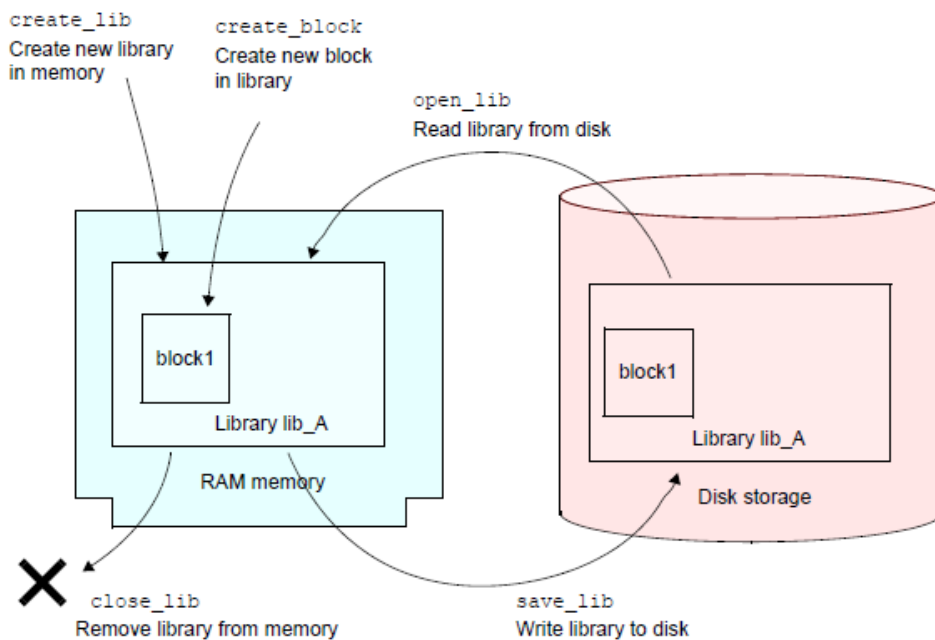
1). Design Libraries

ICC2 的 shell 标注为“icc2_shell”，对应的 library manager 工具为“icc2_lm_shell”。下图为 Design Library 的基本信息：



Design Library 中同一 design 可能有多个 version，默认开启最新的。<current_lib>报告当前 library。

<create_lib>创建新的设计库，<create_block>在设计库中创建新的模块。对应命令还有<open_lib>、<save_lib>。关于 reference library 命令有<set_ref_libs>、<report_ref_libs>。如下示意图说明上述命令关系：



<open_lib>相关命令可以使用相对路径、绝对路径和直接的 design library 名(在\$search_path 中遍历)。ICC2 中创建的设计库不再是 Milkyway, 而是 “.ndm” 格式。在顶层<open_lib -ref_libs_for_edit>可以在顶层取得对 reference 库的编辑权限。

<get_libs>会报出所有打开的设计库, 包括 reference 库; <get_libs -explicit>报出直接打开的库; <get_libs -implicit>报告通过调用 reference 而开启的库。

列出在设计库中所有 blocks 使用命令, <get_blocks -all>, <list_blocks>。示例如下:

```
icc2_shell> get_blocks -all
```

```
{lib_A:block1.design lib_A:block2.design lib_B:blocka.design lib_B:blockb.design}
```

```
icc2_shell> list_blocks
```

```
Lib lib_A /path/libs/lib_A tech current
```

```
-> 0 block1.design Apr-20-16:36 (-代表 block not open)
```

```
+> 0 block2.design Apr-20-16:38 current (+代表 block open)
```

```
Lib lib_B /path/libs/lib_B tech
```

```
-> 0 blocka.design Apr-18-14:02 (>代表此 block 作为 reference 载入)
```

```
-> 0 blockb.design Apr-18-14:20 (*代表有修改但为保存)
```

```
icc2_shell> report_ref_libs
```

```
Library: lib_B
```

```
...
```

```
Name Path Location
```

```
-----
```

```
*+ ref_lib1 ../libs/path/libs/ref_lib1 (*代表 library 打开)
```

```
*+ ref_lib2 ../libs/path/libs/ref_lib2 (+代表有 tech file)
```

```
*+ stdhvt ../libs/path/libs/std/stdhvt.ndm (-代表 library name&location 不可用)
```

```
icc2_shell> get_techs -of_objects lib_A (报告工艺文件)
```

```
{tech28nm1p}
```

```
icc2_shell> report_lib -parasitic_tech my_tech_lib (报告 TLUPlus 文件)
```

```
...
```

```
Library: my_tech_lib
```

```
...
```

```
Full name: /path/tech-info/.../my_tech_lib:my_tech
```

```
File name: /path/tech-info/.../my_tech_lib
```

```
Design count: 0
```

```
No timing data
```

```
Parasitic tech data:
```

```
-----
```

```
Parasitic tech name: my_tech_Cmax
```

```
Parasitic itf technology name: my_tech_Cmax
```

```
Parasitic tech type: TLUPLUS
```

```
Parasitic source file name: /path/Tlup/...
```

```
...
```

```
icc2_shell> save_lib -all
```

```
Saving all libraries...
```

```
5 (报告保存的 design 数目)
```

新版设计库新增属性 “open_count”, 记录此库的开启次数, 比如 “3 次<open_lib lib_A> (open_count of library 'lib_A' 从 0 到 3) 需要 3 次<close_lib lib_A> (open_count of library 'lib_A' 从 3 到 0)”。可以<get_attribute [get_libs \$lib] open_count>。不考虑库的开启数直接关闭, <close_lib \$lib -purge>。关闭时工具不再提醒未保存。

2). Reference Library List

可创建使用 block 例化的设计，当 block 属于其他的设计库时，需要把其他设计库当作当前设计库的 reference 库。设置 reference 库可以<create_lib -ref_libs \$reference_lib>或<set_ref_libs \$reference_lib>（可执行设计库 reference 的更改，删除与增添，更新后需要执行<rebind_block>使设计库中 block 的新设置生效，或用<link_block -rebind>。可以使用<set_view_switch_list>控制顶层 block rebind 的顺序，默认为 {abstract design frame outline}.）。<set_ref_libs -rebind \$modified_reference>可 rebind 新的 reference 库，不影响未做更改的。

Reference 设置不具有传播属性，即 C 时 B 的 reference 库，B 时 A 的 reference 库，则在 A 中可以调用 B 做 instance，B 中可以调用 C 做 instance，但 A 中不能直接使用 C 作为 instance。

Library Packaging 功能（write_lib_package、read_lib_package）可以将 reference libraries 移动到设计库的文件夹下，并自动 rebind 库中的所有模块。

Reference 库和设计库的区别如下表

Feature	Design library	Reference library
Contents	Designs, tech data, reference lib list	Leaf-level cells
Creation tool	Implementation tool (icc2_shell)	Library manager tool (icc2_lm_shell)
Disk storage	A disk directory, libName[.extension]	A single binary file, libName.ndm
Name extension	Any extension (.ndm, .des, .xyz, ...) or no extension	Always has ".ndm" file name extension
Extension usage	Always refer to library using full name, including extension (if any)	Refer to library name without .ndm extension (except in open_lib command)
Usage as reference lib	Optionally serves as a reference library for higher-level design libraries	Serves only as a reference library for design libraries

载入层次化设计。示例过程如下：

```
create_lib mydesign.ndm -ref_libs {std_cell_lib.ndm} -technology mytech.tf
read_verilog bot.v      # Implicitly creates new block "bot"
read_def bot.def
...
read_verilog mid.v      # Implicitly creates new block "mid"
read_def mid.def
...
read_verilog top.v      # Implicitly creates new block "top"
read_def top.def
...
save_lib                (层次结构为 top → mid → bot )
```

3). Technology Data

工艺数据（.tf）支持两种方式载入：第一种<create_lib \$lib -technology techA2.tf >；第二种<create_workspace \$tech_lib -technology techA2.tf>或者<create_lib \$tech_lib -technology techA2.tf>，再<create_lib \$lib -use_technology_lib \$tech_lib>。

对已有工艺数据的库更新工艺文件，使用命令<read_tech_file /usr/TECH/my-tech2.tf>。工艺文件最基

本包含 Unit titles、Layers、Contact Codes，对应在.tf 文件分别为“Tile”、“layerNumber，maskName，taTypeNumber”、“ContactCode，cutLayer，owerLayer，pperLayer”。

4). Blocks

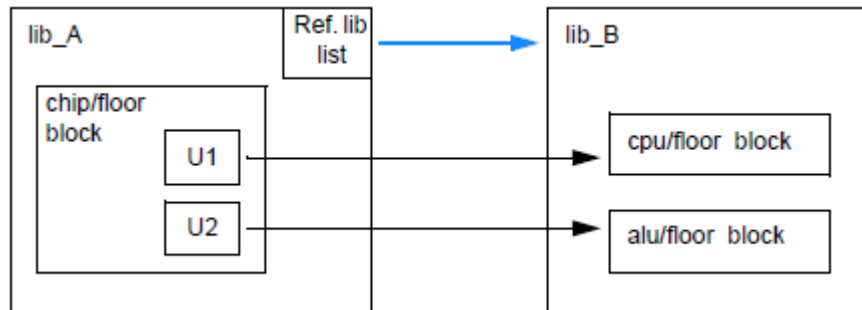
模块 block 的完整信息包括物理和功能设计数据。<read_verilog>隐含有<create_block>，创建载入 Verilog 后，读入约束（load_upf、ead_sdc、read_def），之后主要流程为<place_opt、clock_opt、route_opt>，保存<save_block>。

模块 block 可以拥有多个 views：Design view（.design）代表完整的设计数据；Frame view（.frame）代表用于 place 及 route 的数据；Abstract view（.abstract）代表只保留端口的 view；Outline view（.outline）代表用于 floorplan 的简化 view。对于 reference 库，有额外的 Timing view（.timing）包含单元的时序、逻辑、功耗描述；Layout view（.layout）包含单元的 GDSII 等价信息。

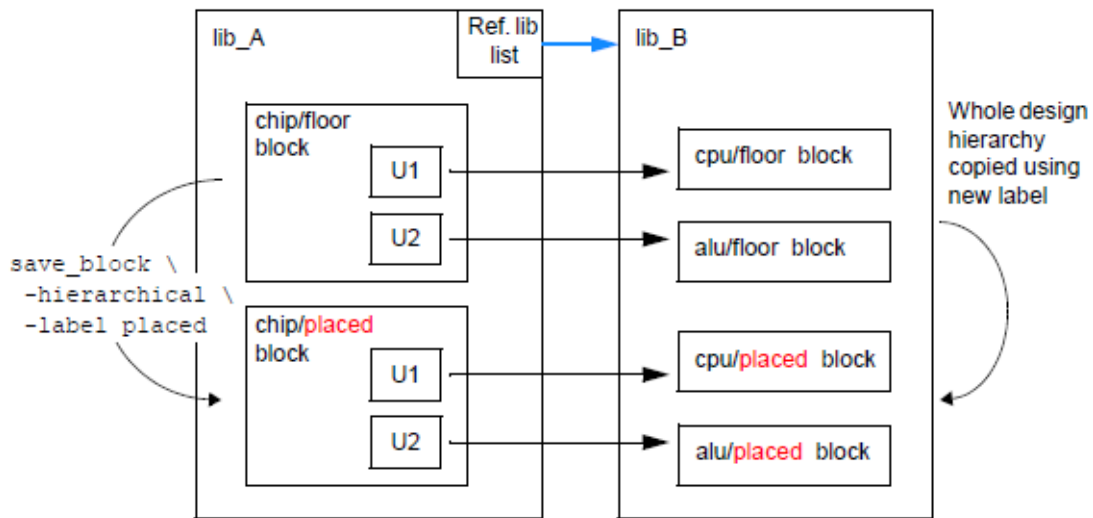
模块 block 的全称结构为[libName:][blockName][/labelName][.viewName]。示例如下：icc2_shell> open_block my_lib:MUX2/ver1.design。其中 label 用在层次化设计中区分不同版本模块，使用符号“/”分割。不同 label 的模块代表相互独立的模块。示例说明创建新 label 及与原 block 关系：

```
icc2_shell> current_block blkZ          (默认 block，无 label)
{libA:blkZ.design}
icc2_shell> save_block -label v1        (创建新 label，v1)
Information: Saving 'libA:blkZ.design' to 'libA:blkZ/v1.design' ...
icc2_shell> save_block -label v2        (创建新 label，v2)
Information: Saving 'libA:blkZ.design' to 'libA:blkZ/v2.design' ...
icc2_shell> current_block              (依旧在默认 block)
{libA:blkZ.design}
```

以下例子说明 label 的用处：设计库为 lib_A，顶层为 chip，reference 库为 lib_B。下图为 floorplan 阶段各库中各模块的状态：



Floorplan 后进行 place，之前留取备份。可以使用指令<save_block -hierarchical -label placed>或<save_block -hierarchical -as chip/placed>。示意图如下：



可以在读取 Verilog 时就指定 label, 命令为`<read_verilog mychip.v -design chip/floor>`。或者在读取 Verilog 之前设置选项, `<set_app_options -name file.verilog.default_user_label -value **>`, 后面`<read_verilog mychip.v -design chip>`就会把 Verilog 读入到 chip/**中。

创建同名新模块, 需要`<create_block -force **>`。

icc2_shell> list_blocks libA # List blocks in library lib_A only

icc2_shell> list_blocks -ref_libs # List blocks in ref libs also

icc2_shell> list_blocks -lib_cells # List lib_cell blocks also

`<save_block -as / -label **>`保存新版, 同时保留老版数据。

`<close_blocks -save **>`关闭时自动保存。

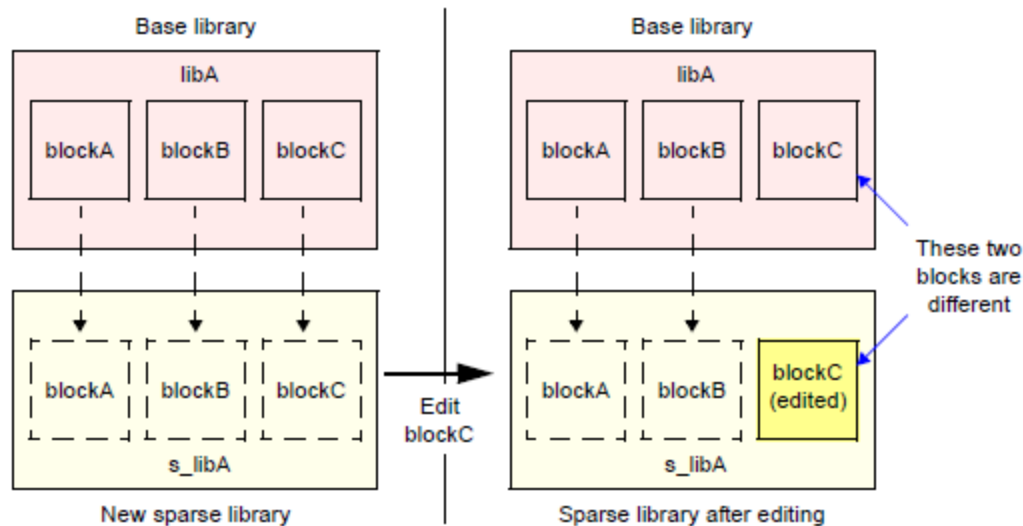
`<open_block -check **>`如果 block 已打开, 不做修改, 不增加 open count; 如果 block 没开, 返回 Error 信息。

5). Block Types

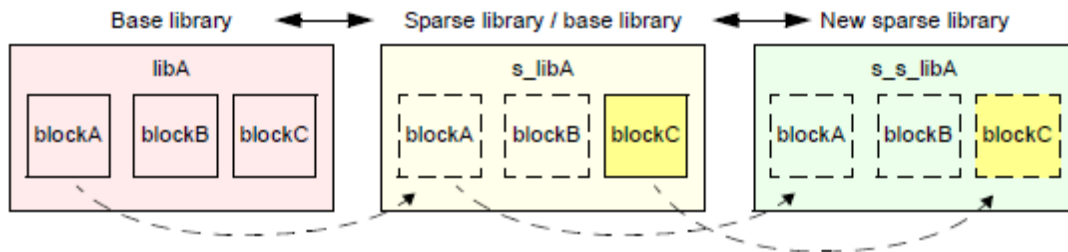
Block 有两种基本类型: design 和 library cell。

6). Sparse (稀疏、稀少) Libraries

Sparse 库用在多用户共享的数据库的使用上。Based library 是共享的数据基础, 不同用户使用时, 可根据需要对其进行修改个增减。使用命令`<create_lib -base_lib **>`创建相关的 based library。Sparse library 主要使用 technology data、reference library list 和 the base library 的模块。创建后可尽心修改。示意图如下:



当直接创建 Sparse Library 时，它指向 based library；若对其进行修改怎保存到本地，不影响 based library。当多用户使用未经修改的 based library 时，为共享状态。Based library 和 Sparse Library 的关系是可层次化的。示意图如下：



为找到 sparse library 模块 s_s_libA，工具将遍历调用层次直到找到逻辑上保存的模块。

创建命令为<create_lib -base_lib libA s_libA>，调用 based library “libA”创建 sparse library “s_libA”；保存命令为<save_lib>；可以通过属性“is_sparse”判断哪些库是 sparse 库，<get_attribute -name is_sparse [get_libs libA]>。

可以使用<create_block、open_block、save_block、copy_block、reopen_block>对 sparse library 进行编辑。直接调用 based library 的模块叫做“remote block”，数据保存在 sparse library 的模块叫做“local block”。可使用属性“is_remote”判断模块是 remote 还是 local，示例：<get_attribute -name is_remote [get_blocks BLK1]>。

已经保存到本地的 local block 可用<revert_blocks **>将之重新转换成 remote block。示例如下：

```
icc2_shell> create_lib -base_lib libA s_libA
Information: Creating Sparse View library 's_libA' with base library 'libA'. (NDM-103)
icc2_shell> open_block BLK1
Information: Updating library catalog of Sparse View library 's_libA'
Information: Adding block 'BLK1.design' to Sparse View library 's_libA'. (NDM-105)
icc2_shell> save_block BLK1
icc2_shell> get_attribute -name is_remote [get_blocks BLK1]
false
icc2_shell> close_blocks BLK1 # Must close block before reverting
icc2_shell> revert_blocks BLK1
Information: Reverted design 's_libA:BLK1.design' and removed it ...
1
icc2_shell> open_block BLK1 # Reopen remote block stored in base library
icc2_shell> get_attribute -name is_remote [get_blocks BLK1]
```

true

当需要重置 sparse Library 时，相当于在不影响 sparse Library 中 local 模块条件下，更新 sparse library 和 based library 的关系。在以下情况下使用：

Base library 变更（名字，工艺文件，reference 列表，base library 中 block 等）；sparse library 使用的变更的 base library，需要重置；将 sparse library 对应不同的 base library。

具体操作如下：（1）、关闭 library 中所有 block，< current_lib **, close_blocks * >;（2）、使用命令< **set_base_lib -library s_libA -base_lib libB** >，此步操作对 sparse library 中的 local block 无影响，但将移除所有旧 base library 的 remote block，添加新 base library 的 remote block。可用<get_attribute -name base_lib_path [get_libs s_libA]>报 s_libA 的 base library。

工具支持调用 base Library 新建 sparse library，在对 sparse library 中 block 修改后更新至 base library 中。具体命令如下：

```
icc2_shell> open_lib s_libA          # Open sparse library
icc2_shell> open_lib -edit libA      # Open base library for editing
Information: Promoting open_mode of library 'libA' from read to edit.    (LIB-033)
icc2_shell> copy_block -from_block s_libA:BLK1 -to_block libA:BLK1
Information: Overwriting block 'BLK1.design' in library 'libA'. (DES-025)
{libA:BLK1.design}
icc2_shell> save_lib libA # Save modified base library
Saving library 'libA'
1
```

7). File Attachments

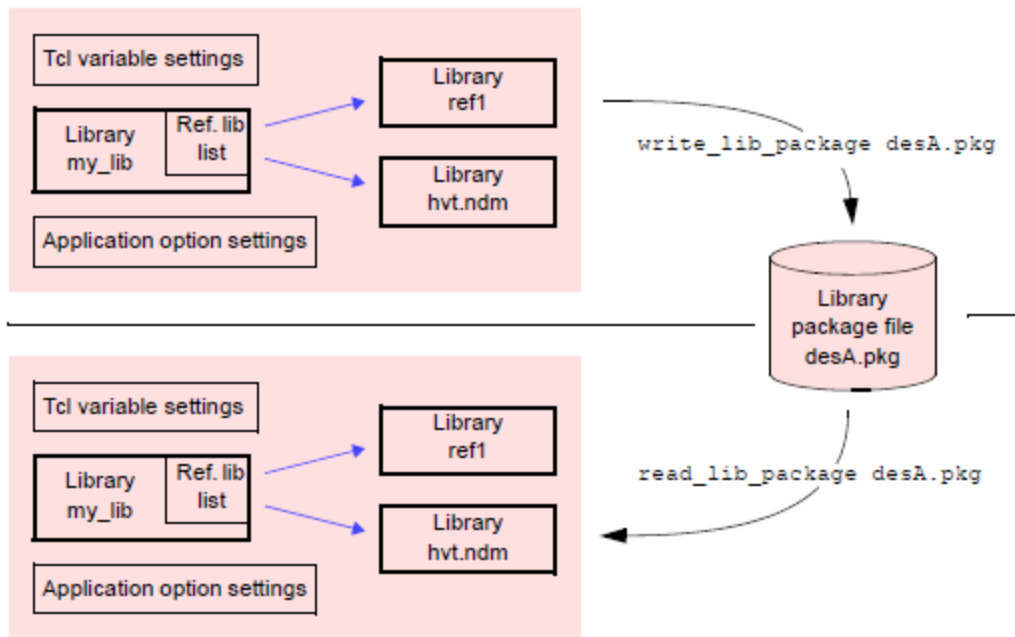
可以将任何格式的文件附在模块或库中。常用在存储重要说明，例如模块和库的描述，tcl 脚本，signoff 标准。

以下命令支持附加说明文件：

<add_attachment> / <report_attachments> / <open_attachment> / <remove_attachments>

8). Library Packaging

Library package 是一种文件，包含 library data 和 environment settings。具体命令为<write_lib_package>，读取为< read_lib_package >。下图为创建过程：



创建命令< write_lib_package >:

```
icc2_shell> current_lib    # Check current library
{my_lib}
icc2_shell> write_lib_package  desA.pkg    # Create package for current lib
1
```

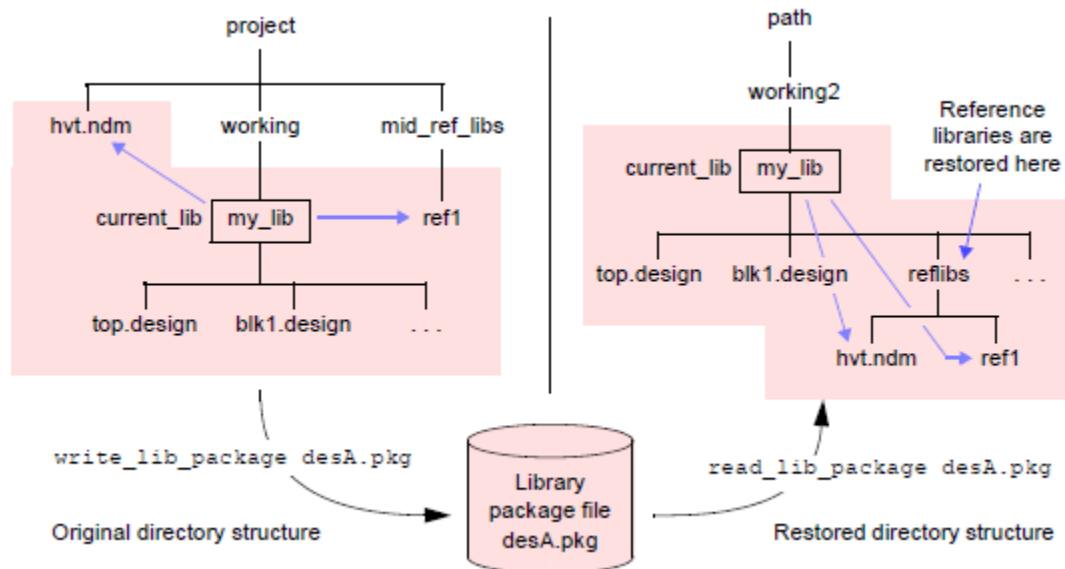
默认命令创建的 library package 包括 current library、its reference libraries、environment settings (Tcl variables、application options、current block)。写出的文件在当前 run 目录下

<write_lib_package>有选项-library */-blocks */-exclude_ref_libs *。

读取命令<read_lib_package >:

```
icc2_shell> read_lib_package  /path/pkgs/desA.pkg
Information: Loading library file '/path/working2/my_lib' (FILE-007)
Information: Loading library file '/path/working2/my_lib/reflibs/ref1'
Information: Loading library file '/path/working2/my_lib/reflibs/hvt.ndm'
Opening block 'my_lib:top.design'
1
icc2_shell> current_lib
{my_lib}
icc2_shell> report_ref_libs
Name Path Location
-----
* ref1 my_lib/reflibs/ref1 /path/working2/my_lib/reflibs/ref1
*+ hvt my_lib/reflibs/hvt.ndm /path/working2/my_lib/reflibs/hvt.ndm
"*" = Library currently open
"+ " = Library has technology information
1
```

默认<read_lib_package> 解析 pkg 文件到当前工作目录下,并应用其中的环境变量。<read_lib_package> 选项-destination 指定 pfg 应用到库的位置, -overwrite 意覆盖目标库。解析的 reference 库被打包成一个叫“reflibs”的库, 位置到设计库目录下。 以下示意图为创建/读取 pkg:



2. Data Import and Export

1). Reading a Verilog Netlist

(1). 读取 Verilog 命令 `<read_verilog>`。最基本为 `<read_verilog **.>`，直接创建模块 `top_module_name.design`；当设计存在多个未例化 module，需要指定顶层 module，例：`<read_verilog -top my_top **.>`；指定 Verilog 应用的模块，`<read_verilog -top my_top -design **.>`。

(2). 如果没有 UPF 指定点地结构，可以使用如下命令指定点地的 port 和 net（默认为 VDD/VSS）

```
set_app_options -name mv.pg.default_power_supply_port_name -value VD1
set_app_options -name mv.pg.default_power_supply_net_name -value VD1
set_app_options -name mv.pg.default_ground_supply_port_name -value VS1
set_app_options -name mv.pg.default_ground_supply_net_name -value VS1
```

(3). Link 模块使用命令 `<link_block>`；`<write_verilog>` 可以写出模块的 Verilog。

2). Allowing Inconsistent Design Data

当数据不完整或存在不匹配时，可以用 `<set_current_mismatch_config **>` 指定当前匹配的检查标准，默认为 Tapeout，即不允许任何不匹配；可自定义，代表修复/检测的 Error 类型由 `<create_mismatch_config>` 指定，这样工具会继续 link；设为 Prototype 时，工具会修复所有检测到的不匹配。在 `<link>` 后，可用 `<report_design_mismatch>` 报告模块 link 中检测/修复的 mismatches。

创建 mismatch 配置文件：

1、`<create_mismatch_config>` 默认生成的配置根据 tapeout 检查标准，即不允许 `<link>` 中有 Error，所有 Error 都被记录；若想生成全新的配置，需选项 `<-ref_config>`。

2、指定对特定的 Error 类型如何处置，可设置 error、ignore、repair。

3、如设置 repair，则对特定类型 Error 处置策略可选对应上图中方法。

例：create_mismatch_config user_def
set_attribute [get_mismatch_type missing_logical_reference] action(user_def) repair
set_attribute [get_mismatch_type missing_logical_reference] current_repair(user_def) create_blackbox (属性分别为 “action(user_def)”、“current_repair(user_def)”)
不匹配的类型：

Error type	Description
bus_bit_naming	A bus is bit-blasted in the reference cell, but is defined as a bus in a different module.
Supported repairs:	
bus_bit_blast_naming	Reconstructs the buses using the style specified in the link.bit_blast_naming_style application option.
missing_logical_reference	The netlist contains an unresolved reference.
Supported repairs:	
create_blackbox	Creates a physical black box for the cell, which enables the missing cell to be placed in the floorplan and retained.
create_empty_logic_module	Creates an empty logic module for the cell; you can create a physical black box for this module and apply a timing model to it.
record_unbound_instance	Records the unresolved reference, but does not perform any repairs.
missing_port	The number of ports differs between the netlist and the reference library.
Supported repairs:	
create_missing_port_mapping	Creates a placeholder port on the cell that is missing a port, either the library cell or the cell instance in the netlist.
port_name_case	Pin name mismatch between the netlist and the reference library due to case-sensitivity.
Supported repairs:	
record_port_name_case_insensitivity	Records the mismatch but does not perform any repairs.
reference_name_case	Cell name mismatch between the netlist and the reference library due to case-sensitivity.
Supported repairs:	
record_cell_name_case_insensitivity	Records the mismatch but does not perform any repairs.

```
< report_mismatch_configs -config_list tapeout >
```

```
...
Config : tapeout
```

Mismatch-Type Action Strategy Available Strategies

```
...
missing_logical_reference error null { create_blackbox record_unbound_instance
                                         create_empty_logic_module }
missing_port error null { create_missing_port_mapping }
port_name_case error record_port_name_case_insensitivity { record_port_name_case_insensitivity }
```

```
reference_name_case      error null {record_cell_name_case_insensitivity }
bus_bit_naming          error null {bus_bit_blast_naming }
1
```

3). Saving a Design in ASCII Format

- Verilog netlist

```
icc2_shell> write_verilog -hierarchy all my_design.v
```

- DEF file

```
icc2_shell> write_def my_design.def
```

- UPF file

```
icc2_shell> save_upf my_design.upf
```

- SDC files

```
icc2_shell> write_sdc -output my_design.sdc
```

- SDC + ENV files

```
icc2_shell> write_script
```

- GDS files

```
icc2_shell> write_gds -library libA -design blockB my_design.gds -layer_map tech12.map
```

Map 将 ICC2 的层映射到 GDSII 中；默认无其他选项则生成本模块的 GDSII。选项 hierarchy 意为写出当前模块下的所有子模块的 GDS。支持 merge。

- OASIS files

```
icc2_shell> write_oasis
```

4). ICC2 Layer Mapping File

命令<write_gds/read_gds/write_oasis/read_oasis>是数据流(stream)和 ICC2 数据库之间的转换。默认。转换过程中，层号保持不变。为使弥合 ICC2 数据库与 GDSII 之间某些层号的区分/缺失，需要进行 map 操作。以上命令都支持选项“-layer_map”。具体语法如下：

第一种：56 10 （左边为 ICC2 数据库中层号，右边为 GDSII 层号）

第二种：data 56:4:power 10:2 （左边为 ICC2，右边为 GDSII；power 意为“geometric (nontext) power-related data”）

详细语法结构为：“；”为注释符）

[object_type] tf_layer[:tf_purpose][:use_type][:mask_type] stream_layer[:stream_data_type]

object_type: 匹配的类型，可为 data（所有非 text）、text、all；

tf_layer: 工艺文件的层号

tf_purpose: The **purpose number** in the tf. (Integer or drawing keyword)

use_type: 几何学用法，可为 power、ground、signal、clock、boundary、hard_placement_blockage、soft_placement_blockage、routing_blockage、area_fill。

mask_type: The multiple-patterning mask constraint of the geometries。对金属层有效值为 mask_one、mask_two、mask_three、mask_same；对孔层支持额外的从 MASK_FOUR 到 MASK_FIFTEEN。

stream_layer: The layer number in the stream file

stream_data_type: The layer **data type number** in the stream file

在 layer mapping 文件中，有一些控制说明结构，例如：

read_always true|false [-mapped_only] [-ignore_missing_layers]，默认 true，即读取 GDS 时遇到 ICC2 中不存在层，则创建新层；设为 false，则在 tf 中没有对对应层号时报 Error；false -mapped_only 为仅读取

mapping 文件中层; false -ignore_missing_layers 为仅读取 tf 中有的层。

blockage_as_zero_spacing true|false。false 默认; true 将 block 处理 zero_spacing_blockage
cell_prop_attribute attribute_value。读/写保留 cell/net/pin 的 GDSII label 和 ICC2 属性的映射关系。
net_prop_attribute attribute_value。
pin_prop_attribute attribute_value。

可以直接设置在<write_gds -layer_map my_layers.map -instance_property 4 -net_property 5 -pin_property 6 my_design.gds >

默认<write_gds / write_oasis> 使用 tf 文件中 cut data type numbers, 例如;

```
Layer "VIA2"  
cutTblSize = 3  
cutNameTbl = (V1SM, V1BAR, V1LRG) # cut names  
cutWidthTbl = (0.05, 0.05, 0.10) # cut width  
cutHeightTbl = (0.05, 0.10, 0.10) # cut height  
cutDataTypeTbl = (5, 10, 15) # cut data types  
...
```

在上例中, 孔 V1SM、V1BAR、V1LRG 将映射成 data types 5、10、15。如果不要这种映射, 使用 “-ignore_cut_datatype_tbl_mapping”。

默认<write_gds / write_oasis>将 mapping 文件中不存在的层映射为 0。选项 “-keep_data_type” 可保留 tf 层号。

默认<write_gds / write_oasis>写出的 label 朝向为 R0, 为将 pin text 调整为 route 的方向, 命令如下:

```
set_app_options -name "file.gds.rotate_pin_text_by_access_direction" -value true
```

默认<write_gds / write_oasis>写出的通孔前缀为 “\$\$”, 可指定不同的前缀, 命令如下:

```
set_app_options -name "file.gds.contact_prefix" -value "MY_PREFIX"
```

可用<set_layer_map_file>保存 mapping 文件, 保存后相关转换默认都采用此 mapping 文件。

5). ICC Layer Mapping File Syntax

为兼容 ICC, 可选择<write_gds>读取 ICC 的 mapping 文件。

```
icc2_shell> write_gds -layer_map map_file -layer_map_format icc_default (1-255)
```

```
icc2_shell> write_gds -layer_map map_file -layer_map_format icc_extended (1-4095)
```

ICC 对应 Milkyway VS. tream 格式如下:

MilkywayObjType [MilkywayNetType][PinCode]

MilkywayLayer [:MilkywayDataType] StreamLayer [:StreamDataType]

MilkywayObjType: A→all, T→text, D→Data

MilkywayNetType: S→signal, P→power, G→ground, C→Clock

PinCode

MilkywayLayer: name 或 integer

StreamLayer: 整数

StreamDataType: 整数

```
T METAL:2 3:5 ; converts text on Milkyway layer METAL data Type 2 to GDSII Stream  
; layer #3 data type 5  
; Note: If you stream back into Milkyway database without using the layer  
; map file, this will switch your text on layer METAL to layer METAL5  
TS 16 31:6 ; converts text associated with signal nets on Milkyway layer #16  
; to GDSII Stream layer #31 and data type 6  
TP METAL3 16 ; converts text associated with power on Milkyway layer METAL3  
; to GDSII Stream layer #16  
TG 28 16 ; converts text associated with ground on Milkyway layer #28
```

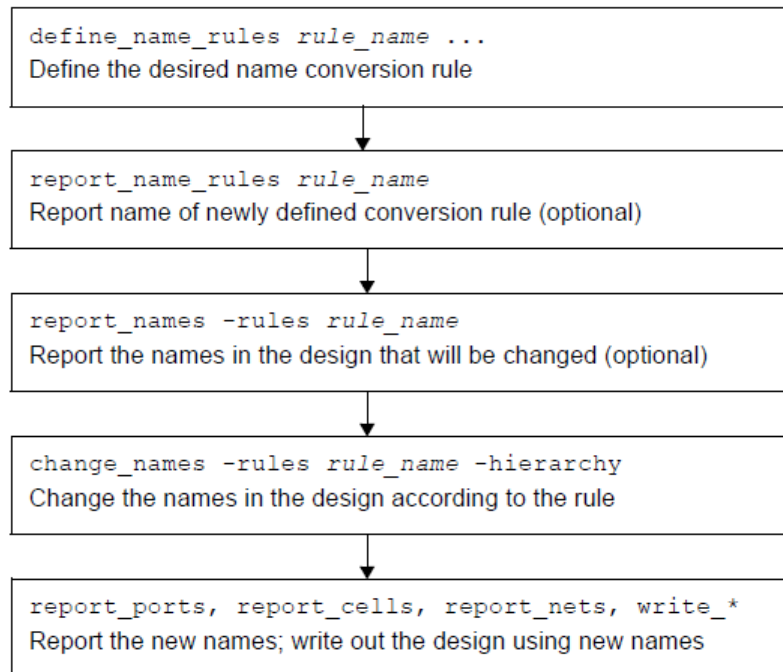
	; to GDSII Stream layer #16
D METAL2 45	; converts data on Milkyway layer METAL2 to GDSII Stream layer #45
DS 45:2 18:5	; converts data associated with signal nets on Milkyway layer #45
	; data Type 2 to GDSII Stream layer #18 data Type 5
A METAL6 3:4	; converts text and data on Milkyway Layer METAL6 to GDSII
	; Stream layer #3 and datatype 4
A METAL4 -	; 连号符阻止 Milkyway 中所有 METAL4 的 text and data

Milkyway 层号对应 blockage 摘要:

Milkyway layer number (default)	Milkyway layer number (extended)	Blockage layer
212	4025	polyBlockage
216	4029	metal4Blockage
217	4030	via3Blockage
218	4031	metal1Blockage
219	4032	metal2Blockage
220	4033	metal3Blockage
223	4036	polyContBlockage
224	4037	via1Blockage
225	4038	via2Blockage

6). Changing Object Names

有 ICC2 数据库输出文件需要对 port/pin/cell/net 名字进行约束，可以用< define_name_rules / change_names>。典型流程如下：



如下例子将 port 名中所有“address”、“data” 转化为“ADR”、“D”：

1. 定义转化规则

icc2_shell> **define_name_rules** UCports -type port -map {"address","ADR"},{"data","D"}}

2. (Optional) 报告 rule, 检查确认

```
icc2_shell> report_name_rules UCports
Rules Name: UCports
Object Max Repl Rem
Type Len Char Chrs Prefix Allowed Chars
-----
Port none '_' no P Mapping String: "{"address","ADR"},
{"data","D"}"
Cell none '_' no U
Net none '_' no N
```

3. (Optional) 报告该 rule 的应用

```
icc2_shell> report_names -rules UCports
Design Type Object New Name
-----
leon3mp port address[27] ADR[27]
leon3mp port address[26] ADR[26]
leon3mp port address[25] ADR[25]
...
leon3mp port data[31] D[31]
leon3mp port data[30] D[30]
leon3mp port data[29] D[29]
...
```

4. 将 rule 应用到整个 design

```
icc2_shell> change_names -rules UCports -hierarchy
Information: Using name rules 'UCports'.
Information: 62 objects (60 ports, 2 bus ports, 0 cells & 0 nets)
changed in design 'leon3mp'.
```

5. (Optional)报告该 rule 的应用

```
icc2_shell> report_ports
Pin Cap Wire Cap
Min Max Min Max
Port Dir rise/fall rise/fall rise/fall rise/fall
-----
ADR[0] out 0.00/0.00 0.00/0.00 0.00/0.00 0.00/0.00
ADR[10] out 0.00/0.00 0.00/0.00 0.00/0.00 0.00/0.00
ADR[11] out 0.00/0.00 0.00/0.00 0.00/0.00 0.00/0.00
...
D[0] inout 0.00/0.00 0.00/0.00 0.00/0.00 0.00/0.00
D[10] inout 0.00/0.00 0.00/0.00 0.00/0.00 0.00/0.00
D[11] inout 0.00/0.00 0.00/0.00 0.00/0.00 0.00/0.00
...
```

6. 当保存或输出时, 设计数据根据 rule 进行修改。

当<write_verilog>时需要用< change_names >设置以满足一些要求。下为示例:

```
icc2_shell> current_block
{my_lib:my_design/done.design}
icc2_shell> change_names -rules verilog -hierarchy
Information: Using name rules 'verilog'.
Information: 321 objects (0 ports, 0 bus ports, 0 cells & 321 nets)
changed in design 'my_design'.
1
icc2_shell> write_verilog my_design.v
```

1

为详细了解 Verilog naming rules, <report_name_rules>。

icc2_shell> report_name_rules verilog

```
...
Rules Name: verilog
Collapse name space: true
Equal port and net names: true
Equal inout port and net names: true
Check internal net name: true
Target bus naming style: %s[%d]
Remove irregular port bus: true
Check bus indexing: true
Check bus indexing use type info: false
Remove port bus: false
Case insensitive: false
Add dummy nets: false
Reserved words: not defined
...
```

3. Working With Design Data

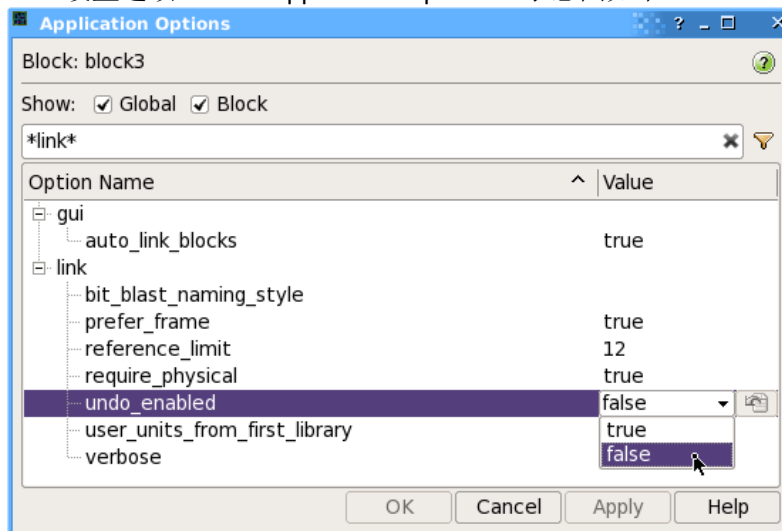
1). Application Options

设置选项命令<set_app_options>, 选项的格式为 “**category[subcategory].option_name**”, 示例如下:

set_app_options -name **place.coarse.max_density** -value 0.6

<report_app_options、get_app_options (报选项, 但无值)>支持通配符。

支持 GUI 设置选项, File > Application Options。示意图如下:



<get_app_option_value>报该选项具体的参数, 支持 “-block/system_default/details (报所有参数, 不如<report_app_options>可读性好)”

<help_app_options>默认报所有参数、选项的简要说明，以 categories 分类。可以指定 category，例如：“help_app_options -category link”；支持通配符；<man>指令报具体使用方法。

参数、选项有 3 级，为“value→User-default→System-default”，优先级由高到低（value 可覆盖后面所有值）。对“value”设置，<set_app_options -name ** -value **>；对“User-default”设置，需要<-as_user_default>；对“System-default”设置，需要<-as_system_default>。

<reset_app_options>重置 value 值，当有 User-default 时，System-default 不影响；当没有 User-default 时，重置 value 保留 System-default。

仅保留 System-default，<reset_app_options -user_default>。

2). Objects and Collections

<get_defined_attributes -return_classes> 报出所有 objects。对 objects 的操作有 <move/flip/rotate/remove_objects>，objects 可以为 block/net/tech 等。设计各类 subjects 指令如下：

get_pin_blockages、report_pin_blockages、get_placement_blockages、get_routing_blockages、get_bound_shapes、report_bounds、get_cells、report_cells、get_clocks、get_clock_groups、get_generated_clocks、report_clocks、get_core_area、get_io_guides、report_io_guides、get_pin_guides、report_pin_guides、get_io_rings、report_io_rings、get_nets、get_shapes、report_nets、get_pins、get_ports、report_ports、get_power_domains、get_domain_elements、report_power_domains、get_pg_regions、report_pg_regions、get_site_rows、get_supply_nets、get_related_supply_nets、report_supply_nets、get_supply_ports、report_supply_ports、get_voltage_areas、get_voltage_area_shapes、report_voltage_areas。

<list_attribute>列的 attribute 的属性有

A - Application-defined
U - User-defined
I - Importable from design/library (for user-defined)
S - Settable 可设置
B - Subscripted 下标

根据坐标找 objects。<get_objects_by_location -at { ** }/-within/-intersect（边界相交，贴边的不算）{ { * } { { * } } -classes { * } }>。

根据连接性找 objects。例如：<get_cells -of_objects [get_nets net32]、get_nets -of_objects [get_cells U83]>。

<get_*>类型的指令可以使用选项-physical_context。

3). Bus and Name Expansion

通配符使用规则：

get_nets a(0:3)	→ {a0 a1 a2 a3}
get_nets {a[0:3]}	→ {a[0] a[1] a[2] a[3]}
get_nets {a[0:1,6:8]}	→ {a[0] a[1] a[6] a[7] a[8]}
get_nets {a[0:4:2]}	→ {a[0] a[2] a[4]}
get_nets a(0:4:2)	→ {a0 a2 a4}
get_nets {a(0:2)b[0:4:2,3]}	→ {a0b[0] a0b[2] a0b[4] a0b[3] a1b[0] a1b[2] a1b[4] a1b[3] a2b[0] a2b[2] a2b[4] a2b[3]}

BUS 类型数据默认使用 “[]”，可通过<set_app_options design.bus_delimiters { } / () / < > >设置。

4). Design Hierarchy

默认 ICC2 对 hierarchy design 的物理实现是 Flat 摆放所有 hierarchy level 的 leaf-level cells。为了按照物理模块要求完成低层次 module，需要先<commit_block \$module_name>。

可在 GUI 中查看层次关系：View > Hierarchy Browser，T→Top、M→Uncommitted logic Module、B→Committed Block。

<current_blocks>、<current_designs>、<current_instance> 、<report_hierarchy>。

5). Technology Data Access

库载入工艺文件有两种途径：<read_tech_file -technology>、<create_lib>。对工艺文件中信息再定义/读取/报告使用如下命令：

get_techs、create_tech、report_attributes -application [get_techs]、remove_tech。get_site_defs、create_site_def、report_site_defs、get_layers、create_layer、report_attributes -application [get_layers]、get_via_defs、create_via_def、report_via_defs、get_density_rules、create_density_rule、report_attributes -appl [get_density_rules]、remove_density_rules。

<write_tech_file>可将库中工艺信息写出，但不支持“symmetry、is_default、routing_direction、track_offset”。

6). Reporting Design Information

<report_lib>有参数“-library/-netlist/-hieraechical/-floorplan/-rrouting”。示例如下：
icc2_shell> report_design -netlist

NETLIST INFORMATION

CELL INSTANCE INFORMATION

Cell	Instance	Type	Count	% of total	Area % of total	siteAreaPerSite
TOTAL	LEAF	CELLS	12973	100	9360583.005	100
Standard cells			12704	97	61696.760	0
Filler cells			0	0	0.000	0
Diode cells			0	0	0.000	0
Module cells			4	0	6117000.323	65
Soft macro cells			4	0	6117000.323	65
Hard macro cells			4	0	49885.923	0

...

REFERENCE DESIGN INFORMATION

Number of reference designs used : 88

Name	Type	Count	Width	Height	Area
------	------	-------	-------	--------	------


```
SDDFX1_RVT lib_cell 1729 5.17 1.67 8.641 ...
NBUFFX32_RVT lib_cell 1727 6.38 1.67 10.674 ...
AO22X1_RVT lib_cell 1364 1.52 1.67 2.541 ...
```

...

NET INFORMATION

NetType	Count	FloatingNets	Vias	Nets/Cells
Total	59276	45346	113837	4.569
Signal	59276	45346	113837	4.569
Power	0	0	0	0.000

...

NET FANOUT AND PIN COUNT INFORMATION

Fanout	Netcount	netPinCount	NetCount
<2	54241	<2	45346
2	2452	2	9123
3	979	3	2224
4	559	4	979
5	243	5	559

...

PORT AND PIN INFORMATION

Type	Total	Input	Output	Inout	3-st ...
Total	103285	86052	18407	1272	484 ...
Macro	600	344	256	0	256 ...
Ports	261	160	228	127	0 ...

...

7). Polygon（多边形） Manipulation

工具支持将多边形形状直接转化成实际的 objects。具体指令如下:

`compute_area`→Calculates the area of a geometric region

`compute_polygons`→ Performs a Boolean operation between two shapes to create a `geo_mask` object（对多个区域进行布尔计算，应用价值较大）

`copy_to_layer`→Copies a `geo_mask` object to create a collection of shapes in a mask layer

`create_geo_mask`→ Copies geometric objects to create a new `geo_mask` object

`create_poly_rect`→ Creates a collection of `poly_rect` objects from a list of coordinates and optionally associates a layer with the result（有一系列坐标点组成）

`resize_polygons`→ Pushes the edges of specified polygons inward or outward, creating a new `geo_mask` object

`split_polygons`→ Decomposes a geometric region into rectangles or trapezoids, creating a collection of `geo_mask` or `poly_rect` objects

由图形转化为具体物理信息的示例如下:

```
icc2_shell> current_block → {libA:block1.design}
```

```
icc2_shell> set myshape1 [create_poly_rect -boundary {{10 10} {30 20}}]
```

```
icc2_shell> set myshape2 [create_poly_rect -boundary {{15 15} {20 30}}]
```

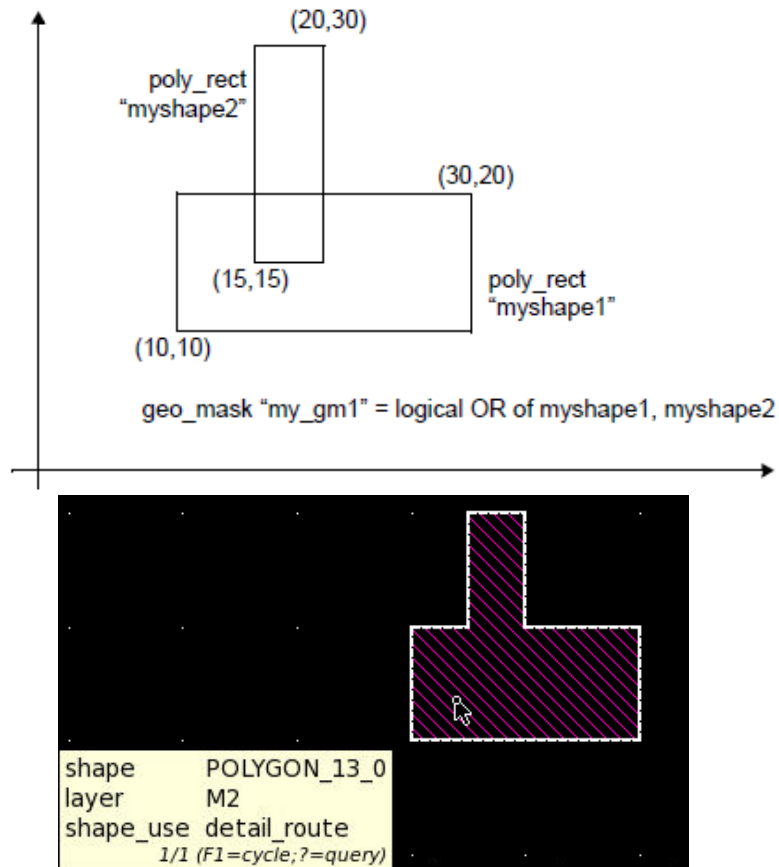
icc2_shell> set my_gm1 [compute_polygons -objects1 \$myshape1 -objects2 \$myshape2 -operation OR]

icc2_shell> report_attributes -application -class geo_mask \$my_gm1

Design	Object	Type	Attribute Name	Value
block1	geo_mask	boolean	is_empty	false
block1	geo_mask	string	object_class	geo_mask
block1	geo_mask	collection	poly_rects	{20 40} ...
block1	geo_mask	int	shape_count	1

icc2_shell> copy_to_layer -layer M2 -geo_masks \$my_gm1 → {POLYGON_13_0}

icc2_shell> save_block

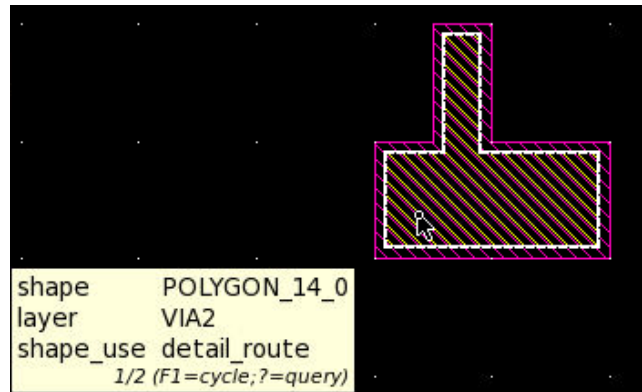


由物理信息抽取图形信息的示例如下：（接上例说明）

set sh2 [get_shapes -of_objects M2]

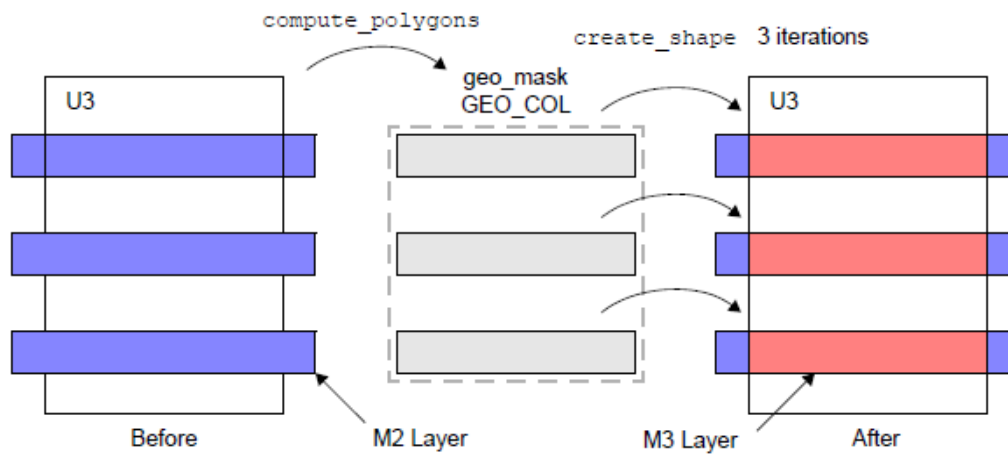
set gm2 [resize_polygons -objects \$sh2 -size -1]

copy_to_layer -layer VIA2 -geo_masks \$gm2



下例说明实际具体使用：

```
set SM [get_cells U3]
set GEO_COL [compute_polygons -operation AND -objects1 $SM -objects2 [get_layers M2] ]
foreach_in_collection PR \
    [get_attribute $GEO_COL poly_rects] {
        set PRPL [get_attribute $PR point_list]
        create_shape -shape_type polygon -layer M3 -boundary $PRPL
    }
}
```



8). Undoing and Redoing Changes to the Design

撤销操作<undo>；恢复撤销<redo>。不是所有指令都可以撤销，可以使用< get_undo_info -command create_block >判断。具体示例如下：

```
icc2_shell> undo -levels 3      # Reverse the 3 most recent changes
icc2_shell> redo -levels 2      # Reapply the 2 most recent reversed changes
icc2_shell> redo -all
```

可以约束<undo>的功能，< set_app_options -name shell.undo.enabled false >、< set_app_options -name shell.undo.max_levels -value 50 >、< set_app_options -name shell.undo.max_memory -value 500000000 >。

9). Schema (图解、概要) Versions (略)

二、ICC2 SDC 格式

1. 基础指令:

current_instance、expr、list、set、set_hierarchy_separator、set_units (-capacitance/-resistance/-time/-voltage/-current/-power 单位)

2. Object Access 指令:

all_clocks、current_design
all_inputs -level_sensitive (指代由 set_input_delay -level_sensitive 约束的输入, 延时来源 Latch)
/-edge_triggered (指代由 set_input_delay 默认约束的输入 ports, 延时来源于 DFF) /-clock *
all_outputs -level_sensitive/-edge_triggered/-clock *
all_registers -no_hierarchy/-hscseparator (指定分隔符, 默认是/) /-clock */-rise_clock */-fall_clock */-cells/-data_pins/-clock_pins/-output_pins (抓取 Register 的 data/clock/output pin) /-slave_clock_pins (抓取 register 中输出 pin 属性为 clocked_on_also 的 pin) /-async_pins (抓取 Register 的异步复位 pin) /-level_sensitive/-edge_triggered/-master_slave (仅考虑 master 或 slave register cells)
get_cells -hierachical/-regexp {patterns}/-nocase/-of_objects objects (支持在 objects 中抓取 cells, 并可对抓取结果再处理。例: <query_objects [get_cells -of_objects [get_pins o*/CP]]>, 提取 pin 信息中 cell 并排序)
get_clocks/get_lib_cells/get_lib_pins/get_libs/get_nets/get_pins/get_ports 类似。
create_clock (默认占空比 50%, 逻辑 1→逻辑 0); create_generated_clock (需要指定名字-name 和源时钟-source。有实际物理 pin, 需要指定时钟关联的 pin; 倍频-multiply_by *, 分频-divide_by *; -edge {**} 整数对应源时钟的第*个边沿, 从 1 开始计数, 先高电平; -combinational, 相当于一分频, source 时钟会在逻辑电路中传播, 到时序原件终止。)
group_path -name * /-weight * /-from/-rise_from/-fall_from/-to/-rise_to/-fall_to/-through/-rise_through/-fall_through *
set_clock_gating_check -setup * -hold * -rise/-fall (-high/-low \$object_of_pin/cell, 对复杂 cell, 工具无法判断检查在时钟的高或低电平进行检查, 需要指定)。Clock gating 的 setup 检查在时钟有效前, 控制信号已稳定; hold 检查当时钟有效期间, 控制信号保持稳定。
set_clock_groups -group * -logically_exclusive (逻辑互斥, 同时存在, 例如多时钟某一经选择器后耦合到选择器前段的另一时钟域) -physically_exclusive (物理互斥, 不能同时存在, 例如两个时钟经选择器后使用其一) -asynchronous -allow_paths (异步该选项有效, 分析异步逻辑之间影响) -name *
set_clock_latency -rise/-fall/-min/-max/-source/-late/-early * -clock *
set_clock_sense -logical_stop_propagation | -stop_propagation | -positive | -negative | -pulse \$pulse_type (rise_triggered_high_pulse, fall_triggered_high_pulse, rise_triggered_low_pulse, fall_triggered_low_pulse)
[-clocks *] \$pin ??什么含义??
set_clock_transition 在 pre-layout 阶段, 工具计算值过乐观, 可根据 post-layout 值进行设置
set_clock_uncertainty
set_data_check 检查逻辑单元的数据有效性, 例 AND 单元有 2 输入 A/B, 默认 “set_data_check -from *1 -to *2” 检查*1pin 对*2pin 的 setup 和 hold 检查; -setup/hold 为仅检查单项; -clock 指定逻辑单元所在时钟域。
set_disable_timing ??与 false path 区别??

set_false_path
set_ideal_network 屏蔽 DRC 检查 (latency 和 transition 默认为 0, 使用 set_ideal_latency、set_ideal_transition 设置)
set_input_delay
set_output_delay
set_max_delay

set_max_time_borrow 例: set_max_time_borrow * { \$latcha \$latchb }

set_min_delay

set_min_pulse_width ??没有查到??

set_multicycle_path 若无-setup/hold 指定, 默认为 setup 设置, hold 为同周期检查 (0); -rise/fall;
-start/end ??不明白意思??

例 1: set_multicycle_path 2 -from { latch1b } -to { latch2d }

检查 latch, latch 在 open edge 发射 (launch) 数据, 在 close edge 锁存 (latch) 数据。The example sets all paths between latch1b and latch2d to 2-cycle paths for setup. Hold is measured at the previous edge of the clock at latch2d.

例 2: set_multicycle_path -1 -from { latch1b } -to { latch2d }

The example moves the hold check to the preceding edge of the start clock.

例 3: set_multicycle_path 0 -from phi1 -to phi1

The example is a two-phase level-sensitive design, where the designer expects paths from phi1 to phi1 to be zero cycles.

例 4: set_multicycle_path 3 -start -from { clk50mhz } -to { clk10mhz }

The example uses -start to specify a 3-cycle path relative to the clock at the path startpoint. Clock sources are specified, affecting all sequential elements clocked by that clock, or ports with input or output delay relative to that clock.

例 5: create_clock -period 20 -waveform {0 10} CLK2

create_clock -period 10 -waveform {0 5} CLK1

set_multicycle_path 2 -setup -from ff1/CP -to ff2/D

例 6: 接上例, 当 CLK1 触发器在 10ns 时失效, 则原有 10ns hold 检查过约束, 可以设置 set_multicycle_path 1 -hold -end -from ff1/CP -to ff2/D, 等同于 set_min_delay 0 -from ff1/CP -to ff2/D

set_propagated_clock

set_case_analysis

set_drive/set_driving_cell

set_fanout_load report_port -design_rule 报告输出 port 的 fanout-load 属性 ??与 fanout 有什么区别??

set_input_transition

set_load

set_logic_dc 、 set_logic_one 、 set_logic_zero 、 set_max_area 、 set_port_fanout_number 、 set_level_shifter_strategy、set_level_shifter_threshold、set_wire_load_min_block_size、set_wire_load_mode、set_wire_load_model、set_wire_load_selection_group、set_max_dynamic_power、set_max_leakage_power 在 sdc 中被忽略 ??没有查到??

set_max_capacitance

set_max_fanout

set_max_transition

set_min_capacitance

set_operating_conditions 在 scenario 中设置

set_resistance

set_timing_derate

set_voltage

create_voltage_area

三、内嵌 TCL

1. 工具 TCL 指令

抑制报 information、warning、error，命令< suppress_message/ suppress_errors ID>；先关指令包括< print_suppressed_messages、unsuppress_message>。

内嵌 TCL 命令包括 which/getenv/setenv/printenv/exec（执行 shell 命令，no file name expansion is performed）/sh（执行 shell 命令，perform file name expansion）??设么意思??

基本 Shell 命令对应 TCL 命令列表：

ls	→	glob 匹配表达式	/	file
rm	→	file delete		
rm -rf	→	file delete -force		
mv	→	file rename		
date	→	date		
sleep	→	after		

例：shell 中 touch 命令的 TCL 写法：

```
proc touch {file_name} {  
    if {[file exists $file_name]} {  
        file mtime $file_name [clock seconds]  
    } else {  
        set fp [open $file_name "w"]  
        close $fp  
    }  
}
```

2. 基本 TCL 指令

File 类：file exists fname、file isdirectory fname、file isfile fname

Glob：glob pattern 抓取文件

Open：open fname access_mode（access_mode 分为 r 只读、需存在；r+读写、需存在；w 只写、覆盖或创建；w+读写、覆盖或创建；a 只写、需存在、附在后端；a+读写、附在后端、文件不存在则创建）

Close：常用格式 set f [open VDD.em w+]；close \$f

Flush：flush file_ID 将缓冲区数据输出

连续访问：

Gets：gets file_ID ?varName? 默认读取文件标示的下一行，若有\$varname，将数据存入该参数；命令返回字符数，文件结尾返回-1；set fid [open \$fname r]、set data_in [gets \$fid]、close \$fid、echo \$data_in

Puts：puts ?file_ID? string 把 string 写回到文件中。例：set fid [open \$fname w+]、puts \$fid "This is my line of text."、close \$fid

随机访问：

Seek：seek file_ID offset ?origin? 把标识的文件的访问点设置为相对于 origin 偏移量为 offset 的位置。origin 可以是 start，current，end，默认是 start。命令的返回值是一空字符串。例：seek file_ID 2000 改变标识的文件访问点，以便下次读写开始于文件的第 2000 个字节。

Tell：tell file_ID 返回标识的文件的当前访问位置。

Eof: eof fileId 如果到达 fileId 标识的文件的末尾返回 1，否则返回 0。

3. 流控制命令

If: 基本结构

```
if {expression} {  
    script  
}  
elseif {expression} {  
    script  
}  
else {  
    script  
}
```

Switch: 基本结构

```
switch test_value {  
    pattern1 {script1}  
    pattern2 {script2}  
    ...  
}
```

支持 3 中匹配方式: -exec (精确匹配)、-glob (通配符)、-regexp (正则表达式)。例:

```
switch -exact $vendor_library {  
    Xlib {set_max_delay 2.8 [all_outputs]}  
    Ylib {-}  
    Zlib {set_max_delay 3.1 [all_outputs]}  
    default {set_max_delay 3.4 [all_outputs]}  
}
```

While: 基本结构:

```
While {expression} {  
    Script  
}
```

For: 基本结构:

```
For {set p 0} {$p <= 10} {incr p} {  
    echo "$p squared is: [expr $p * $p]"  
}
```

Foreach: 基本结构:

```
Set x {a b c}  
Foreach member $x {  
    printvar member  
}
```

退出循环: continue→下个循环; break→跳出循环。

4. 自定义函数

定义: `proc name args body`。例:

```
proc max {a 0} {b 0} {          后面是默认值
    if {$a > $b} {
        return $a
    }
    return $b
}
```

写说明文档:

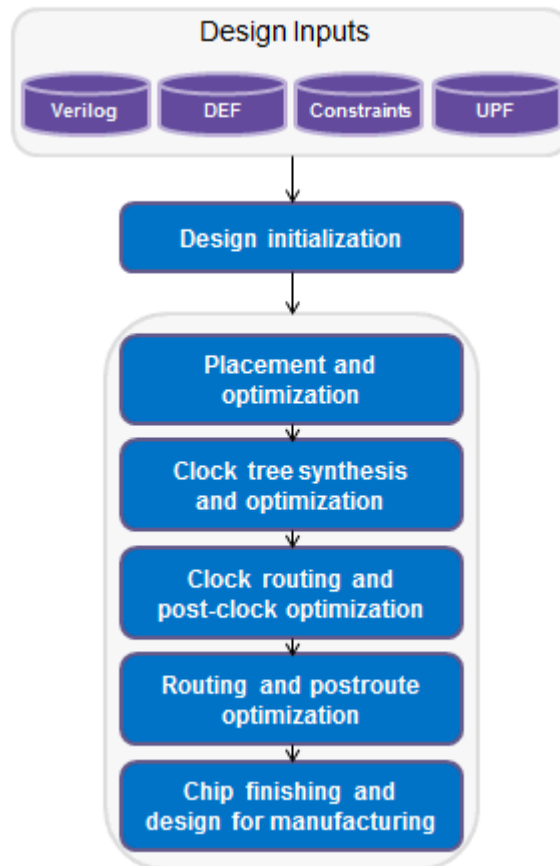
```
proc plus {a b} {return [expr $a + $b]}
define_proc_attributes plus -info "Add two numbers" \
    -define_args {
        {a "first addend" a string required} \
        {b "second addend" b string required} \
        {"-verbose" "issue a message" "" boolean optional} }
help -verbose plus
Usage: plus # Add two numbers
        [-verbose] (issue a message)
        a      (first addend)
        b      (second addend)
```

四、ICC2 Implementation

1. 准备阶段

1). 基本说明

下图说明 ICC2 的输入及基本流程。



分步骤说明:

1. 建库，并检查输入文件的齐备。
2. 创建 floorplan，创建电地。
3. 进行 placement 并优化 (place_opt)。
4. 进行时钟树分析并优化 (clock_opt)。
5. 进行 routing 和 postroute 优化。
6. 执行 Chip-Finish 并进行优化 DFT。

7. 保存。

2). 新概念（先进节点）

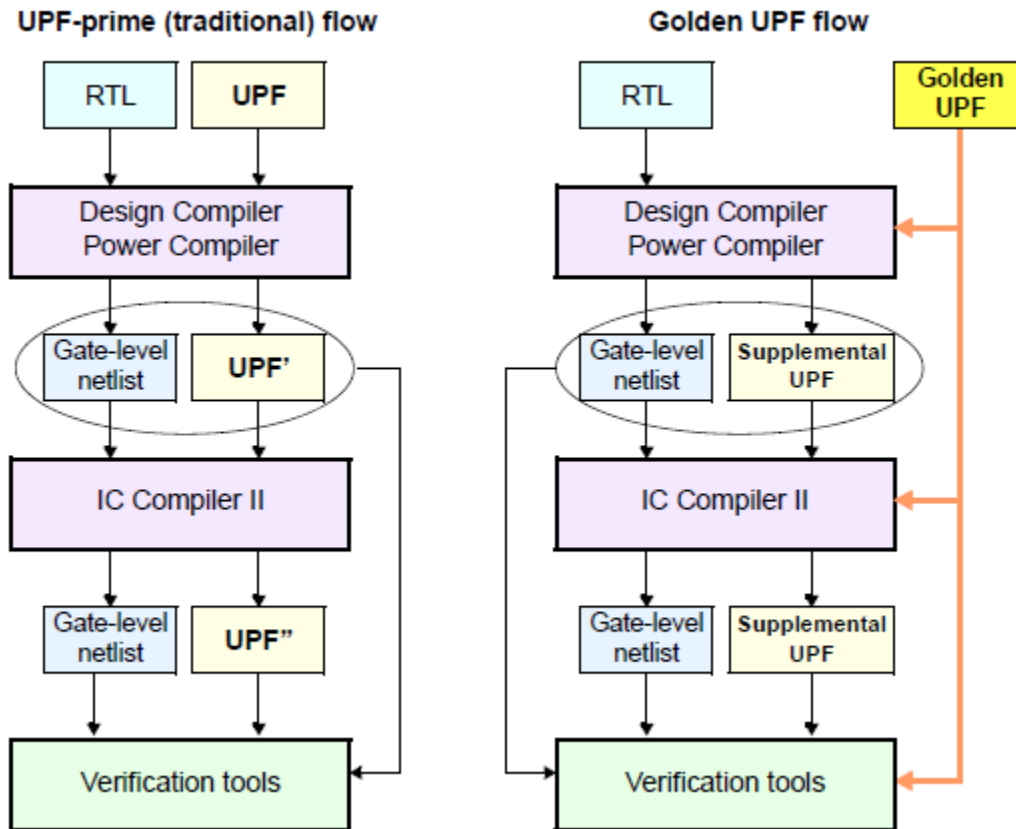
Power 相关概念：

(1). 基础概念：

低功耗设计和多电压域设计需要 UPF 约束文件。UPF 文件可描述供电网络（supply network），开关部件（switches），隔离单元（isolation），记忆单元（retention）信息等。UPF 语言并非直接描述 power 信息，而是通过描述 power 行为来约束。每个 power domain 都有 scope（范围）和 extent（内容）概念。Scope 指的是该 power domain 对应的逻辑层次；extent 指的是该 power domain 对应的单元。

(2). UPF 流程

ICC2 支持 traditional UPF flow 和 golden UPF flow。Golden UPF flow 是针对多电压域设计的流程，在分析、物理实现、验证过程中使用。示意图说明两种流程：



Golden UPF flow 较之传统流程最大的区别是：传统流程经 DC、ICC 分别会突出 UPF'和 UPF''文件给后续使用；但 Golden UPF flow 工具吐出的 UPF 只作为补充（supplemental）UPF，各步骤主体还是读取 golden UPF。

Golden UPF flow 有以下几个优点：

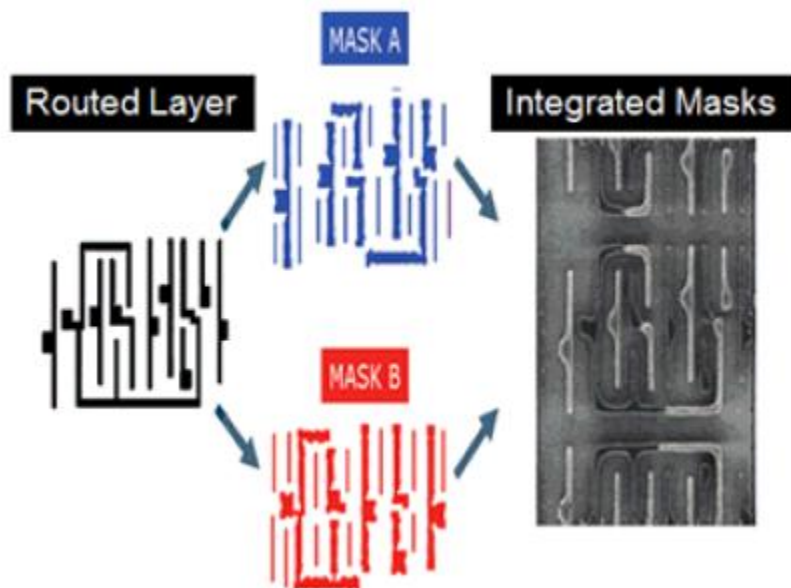
1. Golden UPF 在整体流程中未改变，保持结构，描述指令的一致性；

2. 支持条件分支语句，可以经由判断当前状态（DC/ICC/PT 等）来执行不同的语句，这种分支语句在传统 UPF 流程中失效；
3. 补充 UPF 记录、说明 power intent 的变更，更方便 debug；
4. 可以使用 Verilog 网表来存储 PG 连接性信息，不再需要<connect_supply_net>，这可以有效降低 UPF 文件大小。

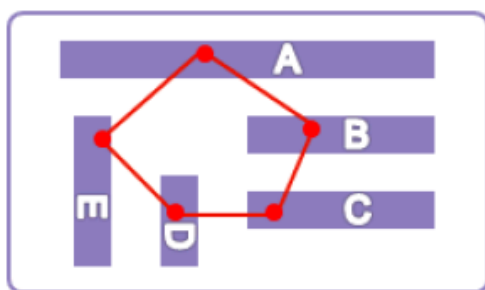
(2). Multiple-Patterning Concepts

在 20nm 之后的工艺节点上，因为特征尺寸过小，会因光源的波长限制产生衍射等现象。为解决先进工艺节点光刻精度的问题，提出新技术——multiple patterning。

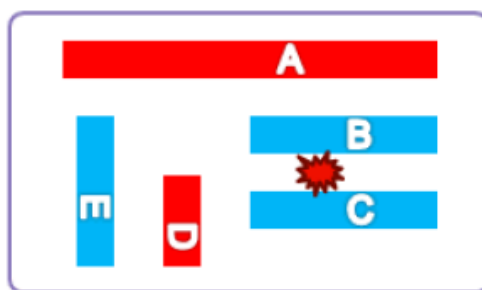
简要说明就是把实际需要光刻图形拆分成多个掩模板，以保证在每个掩模板上都满足光刻的最小尺寸要求。下图为拆成 2 份 pattern 的示意图：



拆分时常见错误：某一区域相邻图形（小于最小距离，无法单 mask 光刻）成环式，且个数为奇数；这种错误叫“odd cycle”。示意图如下：

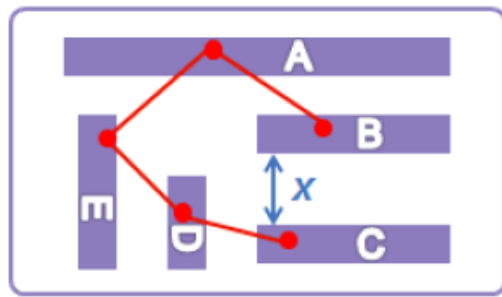


Layout with an odd cycle

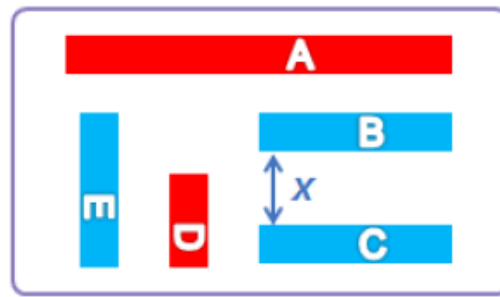


Spacing violation prevents decomposition

如果该“环”中某一级之间距离大于最小距离，则不会报 Error，因为此时可以完整拆分。示意图如下：



Layout with no odd cycle



Layout can be decomposed

ICC2 工具会根据工艺设置检查这种问题，并尽量确保不会发生（约束来源.tf）。一般对低层（金属）层进行 multiple patterning。

未说明哪些层次进行 multiple pattern 的设计称为“uncolored”。对此可设置“Mask Constraints”。对线，mask constraint 被定义在参数“mask_constraint”中。对通孔，mask constraints 被定义在 lower_mask、upper_mask 和 cut_mask 参数中。这些参数可设置的值为如下：

same_mask → 说明 mask color 没有被定义。这种线条需要满足单独光刻的最小尺寸限制。

mask_one → 定义为 color-1，对应一套掩模板。

mask_two → 定义为 color-2，对应一套掩模板。

mask_three → 定义为 color-3，对应一套掩模板。

any_mask → 说明 mask color 没有被定义。这种线条需要满足单独光刻的最小尺寸限制。不分析。

3). 调用基本命令

启动命令为 shell/Unix 下：icc2_shell。“-f”接执行脚本；“-x”接执行命令；“-output_log_file”接 log 文件；ICC2 延续 ICC，开启时先读取.synopsys_icc2.setup。

ICC2 精简命令，拓展参数，参数格式为“category [.subcategory] . option_name”；报告相关参数：< get_app_options *.*>。可使用参数“sh_command_log_file”定义 log 文件。

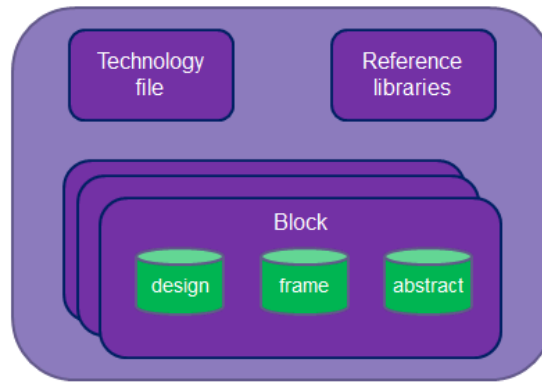
4). 设置 search_path

```
set_app_var search_path ""
lappend search_path ./mylibdir
```

5). Design library 类型说明

Design view：全部设计信息；Frame view：包含 place 及 route 需要信息；Outline view：层次化设计创建 floorplan 中使用；Abstract view：包含 place、route 和时序信息。

下图简要说明 design library 的关系：



相关命令为：create/open/close/save **lib**。

6). Floorplan 基础说明

Block 相关指令： create/open/close/save/current **blocks**。

反标 floorplan 信息：

(1). read_def 默认不会重置已有 floorplan 信息，若需要重置已经存在的信息，使用 “-no_incremental”；默认不会创建根据 def 创建 object，如需要使用 “-add_def_only_objects cells/nets/ports/all”，但不会创建 PG 相关 objects。

(2). 若 def 的 site 名与工艺文件中不符，需要指定 mapping。例：“read_def -convert_sites { {CORE unit} } block.def”。这里 CORE 对应 def 的 site 名；unit 对应工艺文件的 site 名。

(3). 如果仅验证 def 的可读性，使用 “read_def -syntax_only block.def”。

(4). 如果 def 文件没有 port 相关定义，ICC2 工具继承 pad cell 位置当做 port 位置（顶层）。为保证可以准确继承，需要设置。

```
set_attribute -objects [get_lib_cells */IOCell] -name pad_cell -value true
```

```
set_attribute -objects [get_lib_pins */IOCell/PADpin] -name is_pad -value true
```

(5). 读取 scan chain 信息：read_def ./myscan.def（不会影响已有的 scan chain 信息）

(6). 读取 Power Intent 信息

```
load_upf block.upf
```

```
read_name_map block.nmf （如果使用 Golden UPF 流程，存在和 DC 的 name 对应）
```

```
resolve_pg_nets -check_only （检查 UPF 内部描述有没有矛盾或 UPF 和已有 floorplan 有矛盾）
```

```
commit_upf （此步不仅会检查，而且还会解析、修正潜在问题）
```

```
report_mv_path （报上步的变动或不能修复的地方）
```

在此之后不能再设置 UPF 命令，除了 “set_related_supply_net、connect_supply_net、set_design_attributes、set_port_attributes、find_objects、set_scope”。如果需要设置，要先 reset_upf，再按上步骤操作。

7). 时序基础说明

ICC2 延续使用 OCV。对 setup 分析，launch path 按照 maximum delays 计算，capture path 按照 minimum delays 计算；Hold 检查相反。

时序分析考虑 scenario，它由 corner 和 functional mode 组成，在其中定义的不同的 RC 寄生参数计算模型、工艺变量、环境变量等。

8). 电地基础说明

(1). 创建逻辑电地连接

首先确认设计中存在完整的逻辑电地连接，如果不存在使用“connect_pg_net”来创建。就算是单电压域也需用“resolve_pg_net”来解析存在电地冲突，而且在存在冲突时，该命令可以修复部分。

connect_pg_net 有两种模式。第一种: connect_pg_net -automatic(根据 UPF); 第二种: connect_pg_net -net VDD [get_pins */vdd] ; connect_pg_net -net VSS [get_pins */vss] (手工设置)。

(2). 检查电地定义

check_mv_design 检查电地设置的“inconsistent、conflicting library settings、missing isolation cells、incorrect voltage shifting across power domains”。默认不检查电地连接，使用选项“-power_connectivity”开启检查；默认最多报 20 个违例，使用“-max_message_count ** ”设置报告数量。

9). 优化基础说明

(1). 设置单元的使/禁用

set_lib_cell_purpose -include/exclude none lib_cells

-include/exclude 后接阶段，可为“optimization (延时/DRC 等)、hold、cts、power (Leakage)、all、none”

(2). 设置 CTS/CTO 单元选项

1. **set_target_library_subset**。设定优先单元。默认应用范围为顶层和其下所有层次，可用“-objects *”指定具体层次；默认对 data path 和 clock path 都有效，可用“-clock/-data 指定”；“-dont_use”禁用；“-only_here”只能在此处用，其他 objects 不能应用。

2. set_app_options -name **opt.common.enable_target_library_subset_opt** -value 1。使(1)设置生效；

(3). 阻止优化

设置 dont_touch 属性: set_dont_touch [get_cells/nets cell/net_name] true; 默认仅对当前 design 有效，一条线在其他层次处没有该属性，为解决可以[get_nets -segments net_name]或者使用 set_dont_touch_network (默认影响 clock/data path，可用“-clock_only”指定；移除时用参数“-clear”)。可用“report_dont_touch -all”报 dont_touch 属性 objects。

(4). 限制单元优化

设置: `set_size_only [get_cells cell_name] true`
报告: `report_size_only -all`

(5). 限制 size 时 pin name

优化时保持等价性时可能出现 pin name 变化的情况。可通过以下命令设置:
`set_app_options -name opt.common.preserve_pin_names -value always` (不允许变更)
`set_app_options -name opt.common.preserve_pin_names -value never`

(6). 对输入/输出隔离

命令: `set_isolate_ports $port;` 插的单元为 buffer, 距离 port 很近, 尊重约束 (placement blockage 等)。默认插入 buffer 名字格式为 “pi_n”。为阻止底层模块优化时删掉 isolate buffer, 工具设置该单元 `size_only` 属性。以下情况不插入 isolate buffer:

1. 端口 net 设置为 dont_touch
2. 端口非输入、输出类型, 工具不支持 inout 和三态格式端口。
3. 端口为时钟或是电地属性 port。

报告: `report_isolate_ports -all`

移除: `set_size_only [get_cells pi_*] false`
`set_isolate_ports [all_outputs] false`

10). 翻转率基本说明

命令: `read_saif *.SAIF` ; 不同 SAIF 文件对应不同 scenario。
命令: `set_switching_activity -mode, -corner, -scenario`; 相关命令: `report/reset_switching_activity`。

11). 绕线资源基本说明

(1). 全局约束

设置绕线层: `set_ignored_layers -min_routing_layer M* -max_routing_layer M*` (可多次设置)
对超出绕线层的 pin 允许绕线: `set_app_var -name route.common.global_max/min_layer_mode -value allow_pin_connection`
全局约束为 hard: `set_app_var -name route.common.global_max/min_layer_mode -name hard`
忽略 RC 和 congress: `set_ignored_layers -min/max_routing_layer M* -rc_congestion_ignored_layers {M2}`

报告: report_ignored_layers

移除: remove_ignored_layers -max/min_routing_layer | -all | -rc_congestion_ignored_layers

(2). Net 约束

设置绕线层: set_routing_rule [get_nets n1] -max/min_routing_layer M*

设置线约束类型: set_app_var -name route.common.net_max/min_layer_mode -value soft | hard | allow_pin_connection

设置线约束力度: set_app_var -name route.common.net_max/min_layer_mode_soft_cost -value low | medium | high (力度越高 router 越尊重, 会花费更长时间修复)

移除: set_routing_rule -clear

(3). 时钟树约束

普遍约束: set_clock_routing_rules。

设置绕线层: set_routing_rule [get_nets n1] -max/min_routing_layer M*。默认最小层约束为 soft, 最大层为 hard。

设置线约束类型: set_app_var -name route.common.net_max/min_layer_mode -value soft | hard | allow_pin_connection

设置线约束力度: set_app_var -name route.common.net_max/min_layer_mode_soft_cost -value low | medium | high (力度越高 router 越尊重, 会花费更长时间修复)

(4). 优先绕线方向约束

工具会根据库中信息判断优先绕线方向, 如果没有设置, 则 M1 横向, M2 竖直, 之后层依次正交。

命令: set_attribute -objects \$layer -name routing_direction -value vertical | horizontal。

命令 “create_routing_guide -switch_preferred_direction” 对优先绕线方向的设置高于上指令。

12). 并行基础说明

(1). 管理多进程 (多核同一机器)

每个 license 支持 8 个进程。下列命令支持并行处理 “place_opt、clock_opt、refine_opt、route_opt、route_auto、route_global、route_track、route_detail、route_eco”

基本配置命令: set_host_options -max_core 8

报告: report_host_options

(2). 管理分布式进程（不同机器）

使用于下列指令：“signoff_check_drc、signoff_fix_drc、signoff_create_metal_fill、signoff_fix_isolated_via”。

基本配置命令：`set_host_options -name dp_config -submit_command [list qsub -P bnormal
-cwd`

2. Place

1). 约束

ICC2 place 分为两阶段 “coarse placement 和 legalization”。

(1). Placement Blockages

延续 ICC 基本类型，hard | soft | Hard macro | **Partial**。

命令：create_placement_blockage

命令：create_keepout_margin -outer/inner {lx by rx ty} -tracks_per_macro_pin **（默认 0.5，越大 margin 越大，根据 pin 密度，track 宽度设置） -max/min_padding_per_macro ** -type *

(2). Placement Bounds

基本bound类型：Soft move bound、Hard move bound、Exclusive move bound、Soft group bound、Hard group bound、Dimensionless group bound。

Group bound对比move bound主要区别是没有具体区域坐标，而是设置一个矩形区域来约束单元，默认是soft；例：create_bound -name b4 -dimensions {100 100} {INST_1 INST_2}。

Dimensionless group bound不设矩形区域坐标，而是设置“吸引力力度”，默认medium，可设为low | high | ultra。

add_to_bound / remove_from_bound → 将单元加入/移除bound

其他相关指令：get_bounds、**get_bound_shapes**

应用：对Clock Gating Cells自动生成Group Bound

set_app_options -name place.coarse.icg_auto_bound -value true；该设置令ICG和其驱动时序单元尽量近；该group bound在place前自动创建，在place结束时删除；对已在其他bound中cell不约束。

set_app_options -name place.coarse.icg_auto_bound_fanout_limit -value 30；默认40。

(3). Voltage Areas

Voltage Area 是 power domain（逻辑）对应的物理区域。Voltage Area 允许相连，嵌套，重合等。重合区按照优先级设置。

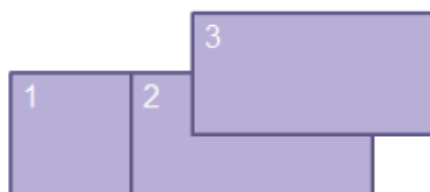
创建：create_voltage_area -power_domains **（可多电压域，但必须拥有相同的 primary supply net。需要指定名字，-name；为单电压域时，名字即为 power domain 名） -region *

创建基础命令：**create_voltage_area**、**create_voltage_area_shape**、**set_voltage_area**。

Voltage merge:

例：`create_voltage_area -power_domains {PD1} -region { {0 0} {10 10} { {10 0} {30 10} } { {15 5} {35 15} } }`

`get_voltage_area_shapes -of_objects PD1` → {VOLTAGE_AREA_SHAPE_1
VOLTAGE_AREA_SHAPE_2 VOLTAGE_AREA_SHAPE_3}



Original voltage area shapes

`set_voltage_area PD1 -merge_regions`

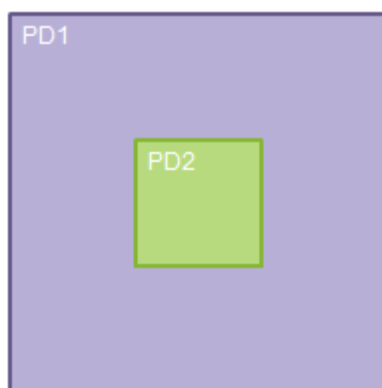
`get_voltage_area_shapes -of_objects PD1` → {VOLTAGE_AREA_SHAPE_3}



Merged voltage area shapes

Voltage area 嵌套使用“stacking order”决定谁有效。默认，按照定义 voltage area 的顺序设定 stacking order，最后定义的即为 top 层。

例：先定义 PD1，后定义 PD2，则报 PD1 区域则为 PD1–PD2 区域；先定义 PD2，后定义 PD1，则报 PD1 区域则为 PD1 区域。



`create_voltage_area -power_domains PD1 -region { {0 0} {30 30} }`

`create_voltage_area -power_domains PD2 -region { {10 10} {20 20} }`

`get_attribute -objects [get_voltage_areas PD2] -name effective_shapes`

→ {{10.0000 10.0000} {20.0000 20.0000}}

`get_attribute -objects [get_voltage_areas PD1] -name effective_shapes`

→ {{0.0000 0.0000} {10.0000 0.0000} {10.0000 20.0000} {20.0000 20.0000}

{20.0000 10.0000} {10.0000 10.0000} {10.0000 0.0000} {30.0000 0.0000}

{30.0000 30.0000} {0.0000 30.0000}}

报告 stacking order: `set_voltage_area_shape -verbose`

设置 stacking order: `set_voltage_area_shape -raise/lower -top/bottom -above/below`

Voltage area 报告命令: get_voltage_areas、report_voltage_areas

Voltage area 变更命令: 修改对应 power domain → set_voltage_area -add_power_domains / remove_power_domains、修改名字 → set_voltage_area -name、修改区域 → create/remove_voltage_area_shape、删除 → remove_voltage_areas -all

(4). Guard Bands

Guard bands 是 voltagearea 周围一圈区域禁止放 cell (包括 level shifters and isolation cells)。Guard bands 主要是保证不同 power domain 之间绝对隔离, 防止短路。默认 voltage area 是没有 Guard bands。设置命令: create_voltage_area / create_voltage_area_shape -guard_band {{ 横向 纵向 }} (若-region 中为多区域, 则此处也需设置多组)。嵌套时, 扣除重合区域 voltage area 也要考虑其 guard bands。

报告设置值, 报告 voltage area 的 “effective_guard_band_boundaries” 参数。

(5). Density Constraints

密度约束, 可以对全局, 模块, 区域设置。可用 partial blockages 设置。

全局设置: set_app_options -name place.coarse.max_density -value **

(6). Cell Spacing Constraints

Spacing Constraints 是标准单元间或标准单元与边界之间的距离设置。首先设置 group (边界 group 名字 SNPS_BOUNDARY) 到单元/边界 (包括 die/macro/voltage/macro boundary 或电地线), 再设置 group 与 group 之间的位置约束。legalizer 和命令 check_legality 忽略标准单元和电地线 “complete blockage” 的 spacing 设置; 检查 “partial blockage” 的 spacing 设置。

步骤:

1. 设置 label

set_placement_spacing_label -name -lib_cells -side right | left | both

2. 约束 label

set_placement_spacing_rule -labels {X SNPS_BOUNDARY} {0 0} → 花括号中含

义: 两个 label 的距离小于第一个, 大于第二个; 该例中{0 0}意为不允许相连; {2 2}意为不允许间距为 2; {2 4}意为小于 2 或大于 4。

Spacing Constraints 不会存储在设计中, 需要在进入 session 时定义。

报告 → report_placement_spacing_rules

移除 → remove_placement_spacing_rules -all / -label / -rule

(7). Inserting Multivoltage Cells

多电压域设计需要一些特殊单元, 例如 level shifters、isolation cells 等。通常这些单元在

逻辑综合阶段插入 (compile_ultra 阶段), 但流程也支持在 ICC2 阶段插入。在做等价性验证时, 需要考略 multivoltage cell 在 RTL 中是否有逻辑存再。关于 multivoltage cell 的具体说明参照 Synopsys Multivoltage Flow User Guide。

1. 插 Level Shifters

Level Shifters 的作用为衔接两个不同电压, 保证跨电压后, 输出信号能驱动之后的电压域单元。

命令: create_mv_cells。默认工具将 Level Shifters 设为 size_only, 将 port 到 Level Shifters 的线设为 dont_touch 属性。创建单元类型及位置根据 UPF 中 mapping 和 insertion strategy 设置。

2. 插 Isolation Cells

Isolation Cells 可选择性关断输入端电压域输入。

3. 调整已有 Multivoltage cells 的 power strategy

命令 associate_mv_cells 、 commit_upf 会自动为已存在 Multivoltage cells 分配 power strategy。如果不满意, 可手动调整: set_power_strategy_attribute; 查看: get_power_strategies。

4. 报告、检查

report_mv_path 报告 level shifters、isolation cells 的详细信息或是不能插入的原因; 检查: check_mv_design

(8). Marking the Clock Networks

在 CTS 之前, place 及优化应该在时钟为 ideal 的前提下。

```
foreach_in_collection mode [all_modes] {  
    current_mode $mode  
    set_ideal_network [all_fanout -flat -clock_tree]  
}
```

为弥补 ideal 时钟的乐观分析, 需要设置 uncertainty、latency、transition 约束 (set_clock_uncertainty、set_clock_latency、set_clock_transition)。

在 CTS 时, 需要先 remove_ideal_network。

2). 执行及优化

(1). 主流程简介

Place 阶段分为 coarse placement 和 legalization。支持 set_host_options -max_cores **。
全流程: place_opt; Coarse: create_placement (包括 high-fanout 和优化; Legalization: legalize_placement

place_opt 主要分为以下 6 个阶段:

1. Initial placement (initial_place)

执行 coarse placement, 仅做 datapath timing requirements 分析。也做 scan chain 优化。

2. Initial DRC violation fixing (initial_drc)

移除已存在 buffer trees 并执行 high-fanout 分析和 electrical DRC violation fixing。

3. Initial optimization (**initial_opto**)
执行 timing、area、congestion、leakage-power 优化。
4. Final placement (**final_place**)
执行 incremental placement 来优化 timing 和 routability。
5. Final optimization (**final_opto**)
6. Legalization(**无**)

Place_opt 设置及影响参数:

1. place_opt -from / -to 控制 place_opt 的运行阶段, 可选前 5 项, legalization 必做。
2. 考虑时钟树结构。设置参数 place_opt.flow.**clock_aware_placement** 为 true。优点: 改善 clock gating cell 的 enable pin; 减少 CTS 阶段 clock buffer 的数量。
3. 创建临时 CTS 为 place 及优化。设置参数 place_opt.flow.**trial_clock_tree** 为 true。之后的 clock_opt 或 synthesize_clock_trees 删掉之前的时钟树。
4. 优化 ICG 单元。设置参数 place_opt.flow.**optimize_icgs** 为 true。该参数设置将使用 clock-aware placement, 创建临时时钟树, 将 ICG 和相关单元放在更合适的地方, 无论 place_opt.flow.**clock_aware_placement/trial_clock_tree** 设为什么; 若 clock gating cell 的 enable 端是 关键路径, 会拆分复制 clock gating cell。可用参数 place_opt.flow.**optimize_icgs_critical_range** 控制在什么范围内对 clock gating cell 起效。
place_opt 在优化阶段只拆分复制 clock gating cell 单元, 不能合并。可在 place_opt 前先 **merge_clock_gates**。
5. 默认在 high-fanout 分析和 electrical DRC 修复时使用 virtual **route**, 可以使用 global **route** 引擎分析。set_app_options -name place_opt.**initial_drc.global_route_based** -value **1**; 为在 initial placement 阶段分析两次, set_app_options -name place_opt.**initial_place.two_pass** -value true。这样会进行 wire-length-driven placement、trial high-fanout synthesis、timing-driven placement。对“**碎片式**”floorplan, 结合上述两种策略可以优化时序, 减小串扰。
6. 设置 congestion 约束。set_app_options -name place_opt.**congestion.effort** -value low | medium | high。
7. 使用更高金属层计算。Place 阶段默认使用 min/max 之间可用金属层参数的均值, 对于长线或是关键路径, set_app_options -name place_opt.flow.**optimize_layers** -value true, 工具使用高层计算, 对应绕线也使用高层。
8. 默认 refine_opt 使用 Path Optimization, place_opt 不使用。可用“set_app_options -name place_opt.flow.**do_path_opt** -value true”开启。

(2). Power 优化简介

功耗主要分动态和静态功耗。动态功耗取决于 cell 数和频率; 静态功耗主要取决于亚阈值漏电, 一般沟道越短漏电越大。一般, 越先进节点静态功耗占比更大。

ICC2 中功耗选项主要分为 Low-Power Placement、Leakage-Power Optimization、Total-Power Optimization。

1. Low-Power Placement 步骤

反标翻转率 → 至少设置一个 scenario: set_scenario_status -dynamic_power true → 设置参数 place.coarse.low_power_placement 为 true。

2. Leakage-Power Optimization 步骤

至少设置一个 scenario：set_scenario_status -leakage_power true → 设置 **opt.leakage.effort** 参数为 low | medium | high → place_opt/clock_opt（preroute 优化）→ 设置 route_opt.flow.enable_power 为 true → route_opt（postroute）。

3. Total-Power Optimization 步骤

设置 **opt.power.mode** 为 total，其他设置同上。

(3). Scan Chain 优化简介

在 DC 阶段，时序原件可能会根据 chain 需求 replace。通常设置不同测试信号，scan cell 可抓取功能逻辑信号或是测试数据。

如果使用 SCANDEF，place 阶段自动优化 scan chain。Place 阶段先优化内部逻辑，在对 scan chain 进行 repartitioning、reordering。Repartitioning 交换跨 scan chain 的内容，使每个 chain 的内部组分尽量近。这样减少线长和拥塞，保证绕线成功率。

(4). Preroute 阶段自动使用 Nondefault Routing Rules 优化

设置参数 **place_opt.flow.optimize_ndr**、**clock_opt.flow.optimize_ndr**、**refine_opt.flow.optimize_ndr** 为 true，分别对应 place_opt、clock_opt、refine_opt 阶段有效。

例：set_app_options -name place_opt.flow.optimize_ndr -value true

(5). 定义优化单元的前缀名

```
set_app_options -name opt.common.user_instance_name_prefix -value "PO_"
place_opt
set_app_options -name opt.common.user_instance_name_prefix -value "CO_"
clock_opt
set_app_options -name opt.common.user_instance_name_prefix -value "PO_"
route_opt
```

定义前缀应用于所有优化阶段，故在不同阶段，需要分步设置。

CTS 单元前缀参数：cts.common.user_instance_name_prefix

(6). SPG 流程（使用 DCG 的 Physical Guidance）

place_opt 可以使用 DCG 的 Physical Guidance，以此作为 placement 基础起点。优点：减少运行时间；获得更好的 DCG 与 ICC2 的一致性。

为保持一致性：使用相同约束（时序+UPF）；使用 DCG 阶段 placement（DCG 阶段：write_def；ICC2 中 read_def -add_def_only_objects {cells} -convert_sites）。

DCG 写出的 DEF 默认包含 Die area、site rows、tracks、ports、macros、standard cells、physical only cells、Route guides、prerouted nets、Placement blockages。

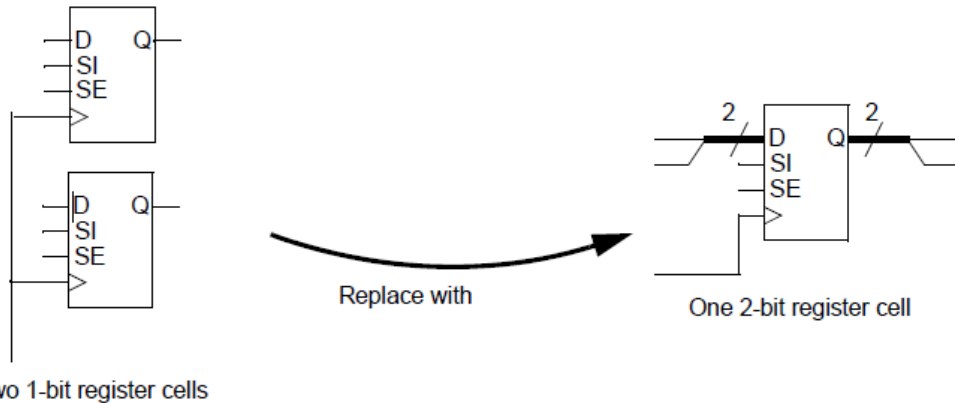
在 ICC2 中需要额外定义的（DCG-DEF 不能写出）：Voltage area、Layer constraints、Special route guides with utilization、Special blockages with blocked layers。

开启 SPG 流程：set_app_options -name place_opt.flow.**do_spg** -value true

(9). 执行 Multibit Banking

在 multibit cells 可用情况下，ICC2 可以将单位寄存器（register）或是单位多电压域单元（例如 isolation、level-shifter cells）替换成等价的多位单元。

例：



Two 1-bit register cells

这种策略可以减小总时钟线长度，时钟 buffer，和相关单元面积。

具体操作步骤：

1. create_placement → 执行 initial placement
2. identify_multibit → 识别可以被替换的 multibit cell，并生成替换脚本。
3. 修改“identify_multibit”脚本，确认后 source。其中已包含替换命令“create_multibit”。
4. place_opt -from initial_drc → 优化设计
5. （选）如果没有改善 Qor，将某些 multibit banks 拆成更小的 multibit banks 或是单独的 register。

具体指令说明：

1. identify_multibit -register / mv_cell（只能选择对一种 cell 有效） -output_file ** -input_map_file **（与-register 同时使用） -slack_threshold **（忽略比之更恶劣的路径相关触发器） -slack_threshold_file -exclude_instance ** -exclude_library_cells **

Slack 文件格式： path_group_name \$slack

Map 文件格式： \$group_name { 可以 group 的 reference }

数字（说明几个单位寄存器打包成一个多位单元） {rank 中该多位单元的个数 多位单元的 reference}{ }
{ }

举例： reg_group_1 { REGX1 REGX2 REGX4 }

2 {1 MREG2}

4 {1 MREG4}

6 {1 MREG2}{1 MREG4}

说明： group 名字叫 reg_group_1，可以打包单元 “REGX1 REGX2 REGX4”；第一行含义：两个一位寄存器可以打包成一个 register bank，该 bank 由 MREG2 组成；第三行含义：六个一位寄存器可以打包成一个 register bank，该 bank 由 MREG2 和 MREG4 组成。

2. split_multibit \$instance_name -lib_cells { 拆分的组合 }

例： split_multibit reg_group_0 -lib_cells { MREG2 MREG2 } → 将一个 instance 名为 reg_group_0 的四位 register bank 拆分成两个两位的 multibit banks。

(10). Incremental Placement 及 Optimization

Incremental placement & Optimization 命令：refine_opt。该命令主要分为 5 步骤：

1. Initial path optimization (initial_path_opt)

在此阶段，工具沿 timing paths 移动寄存器，已达到修复时序的效果。如果影响其他时序，放弃。

2. Incremental placement (inc_place)

降低拥塞并提高可绕线性。

3. Incremental optimization (inc_opt)

进一步优化 timing、area、congestion、leakage-power 优化

4. Final placement (final_path_opt)

最终微调 path 上单元以改善时序。

5. Legalization

类似 place_opt，可以指定运行阶段。可以对某些 endpoint 优化 (refine_opt -end_points)，默认对整个 design 优化；开启层优化设置：set_app_options -name refine_opt.flow.optimize_layers -value true。

(11). 执行 Magnet Placement

主要命令：magnet_placement。

选项：-move_fixed、-move_legalize_only → 可以移动 fix 属性的 cell

-mark_fixed、-mark_legalize_only → 在 magnet placement 后将移动的 cell 设为 fix 属性

-logical_levels ** -exclude_buffers -stop_by_sequential_cells -cells ** (指定被 magnet 的 cell)

(12). Refining Placement

在拥塞问题很严重时使用该策略，命令：refine_placement。默认对整体设计有效，可以指定区域 (-coordinates)、placement 力度 (-effort low|medium|high)、减小拥塞力度 (-congestion_effort low|medium|high)、对原有 placement 的扰动 (-perturbation_level min|medium|high|max)。

(13). 处理多电压域模块

Placement 时考虑 voltage area 约束。Placement 阶段，会据此摆放，并在边界插 level shifters、isolation cells；优化阶段，根据对“可用供电线”和“双供电 buffer 最少”的动态分析插 buffer，为理解 buffer 插在哪些线中，使用命令 check_bufferability；根据 power state tables 执行 always-on buffering。

3). 分析

(1). 使用属性 “status”

属性 “status” 存在于 instance 中。下列出 Placement 阶段中存在 “status” 和对应 DEF 格式:

unplaced	→	+UNPLACED
placed	→	+PLACED
fixed	→	+FIXED
legalize_only	→	N/A
locked	→	+COVER

查看: `get_attribute -objects [get_cells INST1] -name status`

设置: `set_placement_status fixed [get_cells INST1]`

(2). 报告命令

1. 多电压域报告: `check_bufferability`。主要检查 multivoltage rules, 说明为什么不能在某条线上插 buffer; 或可以插, 插什么类型 single-rail / dual-rail buffer。

`check_bufferability -driver ** -loads ** (或-net) -voltage_area **`

例: `check_bufferability -driver top_reg_1/Q -loads top_U1_9/B -voltage_area PD1 →`

Information: Cannot be buffered because required supplies are not available (MV-472)

2. 基础全面报告: `report_placement`。涵盖 Qor 信息和摆放违例报告等。

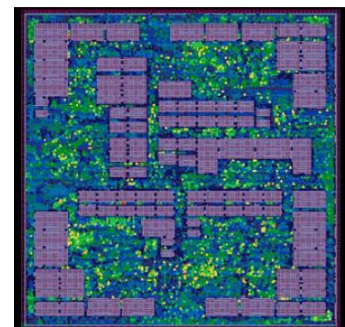
`report_placement -hierarchical -wirelength interface|hard_macro|IO|none` (默认报所有线长) `-physical_hierarchy_violations internal|external|none` (分别为报与边界重合单元、与子模块重合单元、不报; 默认报所有) `-swimming_pool_area` (意为被 macro、子模块等包围的区域, 这种区域若果摆单元, 后期绕线困难, 最好用 blockage 盖住) `-thin_channel_area` (最好用 blockage 盖住) `-hard_macro_route_over` (macro 摆放存在绕线交叉问题) `-hard_macro_overlap`

(3). GUI 分析

Cell Density Map → Placement > Cell Density Map

Pin Density Map → Placement > Pin Density Map

Global Route Congestion Map → Placement > Global Congestion Map



(4). 时序分析

报告命令: `report_timing` (更具体 `report_delay_calculation`)、`report_qor`、`report_constraints` (报 logic DRC violations)

分析指令: `analyze_design_violations -type max_trans / setup / hold / only_lib` (指定分

析某一类 violation) -fanout * (默认分析大于 30 的 violation) -slack * (默认分析小于 5ps 的 violation) -drc_nets -endpoints ** -output ** (默认输出 analyze_design_violations.txt)

(5). 功耗分析

报告命令: report_power 只分析静态功耗 (使用 library 中 cell_leakage_power 计算, 无则使用 cell 对应的 default_cell_leakage_power 值)。泄露功耗支持一下几种模式: Average 模式 (按个模式几率加权计算); Unconditional; State (需要指定状态, set_case_analysis)。切换使用命令 “set_app_options -name power.leakage_mode -value average (默认)”。

3. CTS

1). CTS 前提条件 (准备)

CTS 前需检查: 时钟定义 (create_clock、create_generated_clock), placement 是否存在隐含问题 (基本问题: check_legality -verbose; Congestion 问题: 如果 place 阶段有 congestion 问题, CTS 后会更严重, 可使用 create_placement -congestion -congestion_effort high 重新 place; Timing 问题; Logic DRC 问题: max cap/transition; 电地问题: 是否指定 CTS scenario; 是否需要先进行 high-fanout 线分析)。

2). 分析时钟基本结构

判断时钟基本结构, 需要了解:

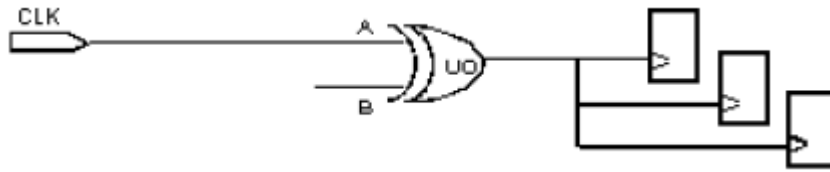
1. 时钟根节点;
2. 时钟 sink 点是否存在例外 (exceptions);
3. 是否存在预布 cell (ICG cells 等);
4. 是否存在多时钟共享路径 (注意 SDC 设置要有 -add 选项);
5. 不同时钟之间是否存在联系 (同步检查、skew 要求等)

3). 时钟定义

时钟遍历由根节点到所有时序单元的时钟端; 当经过的时序单元是 ICG 或后面存在 generated clock, 时钟继续传播。

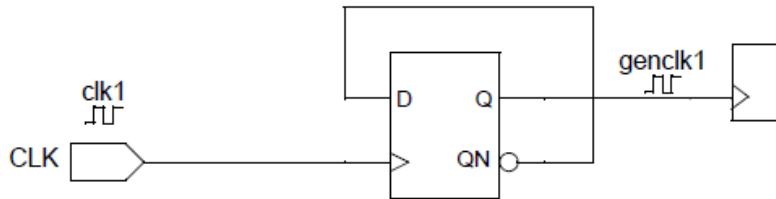
注意事项:

1. 如果 clock-gating 逻辑使用 non-unate 单元, 例如 XOR/XNOR 门, 工具 tracing the clock Path 时同时使用 negative-unate 路径和 positive-unate 路径计算。如果跟实际逻辑需求不符, 使用 set_case_analysis 强制使 clock-gating 单元一端置位。示例如下:



说明：此例中 clock-gating 单元为 UO。B 端可置 1 或 0，A 端 CLK 的上升沿在 B 不同置位情况下，输出可为上升沿或是下降沿。这种单元为 non-unate 单元，即非单边变化。根据逻辑需要，可化为单边变化：set_case_analysis 0 U0/B。

2. Generated clock：以二分频为例：



创建的 generated clock 以 master clock “clk1” 为参照，并平衡这两个时钟 sink 点的 skew。使用 check_clock_trees 检查。

3. 创建时钟在 port 上（不连接 I/O pad cell），要设置驱动力（set_driving_cell -lib_cell mylib/CLKBUF* [get_ports **]），设置过小将导致多插 buffer，如不设工具默认驱动能力无限大。

4. 创建时钟在 port 上（连接 I/O pad cell），需要设置输入 transition time。命令：set_input_transition -rise / -fall ** [get_ports *]

5. sink 点的例外情况：

ignore pins：在时钟平衡时忽略，只在前面插个 guide buffer；但会修 DRC。以下情况为 ignore pin：

该时钟 sink 点作为其他时钟的源端；

三态使能 pin；

输出 port；

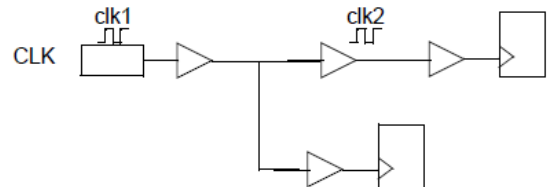
时序器件的非时钟输入；

被 set_case_analysis 定为常值；

逻辑单元输入；

没有输出的 ICG 或没有没有是使能的 ICG。

人工设置 set_clock_balance_points



6. 检查 “report_disable_timing” 和 “report_case_analysis”，是否存在非计划设置。

7. 定义例外的 sink 点

→ 定义为 sink 点：set_clock_balance_points -clock * -balance_points * -consider_for_balancing true [-corners *]

不使用 -clock，应用于所有时钟；默认按照 skew 为 0 平衡，使用 -delay * [-rise] [-fall] [-early] [-late] 可以控制该点相对的延时差。

→ 定义 ignore 点：set_clock_balance_points [-clock clock] -consider_for_balancing false -balance_points *

报告、移除：report_clock_balance_points、remove_clock_balance_points

→ 定义 Don't Touch Exceptions 点：set_dont_touch_network -clock_only [get_pins pin_name] 或 set_dont_touch [get_cells cell_name] true、set_dont_touch [get_nets -

segments net_name] true

如果没有-clock_only, 则也会影响 data path; 清除设置: set_dont_touch_network -clock_only [get_pins pin_name] -clear; 报告: report_dont_touch -all

→定义 size-only Exceptions 点: set_size_only [get_cells cell_name] true

报告: report_size_only -all

8. 存在多个 CTS mode 时, 可以跨 mode 复制 exception 点: set_clock_tree_options -copy_exceptions_across_modes -from_mode {*} -to_mode {*}

9. 如果对某些 sink 点有 set_clock_latency 设置, 对 Exceptions 点需要一步 derive_clock_balance_points, 在 CTS 之前执行。

例如: set_clock_latency -source 0.5 [get_clocks clk]

set_clock_latency 1.0 [get_clocks clk]

set_clock_latency 1.5 [get_pin reg1/CK]

set_clock_latency 0.5 [get_pin reg2/CK]

对上述设置, derive_clock_balance_points 效果等同以下命令: 为避免冲突, 要避免同时使用这两个命令。

set_clock_balance_points -delay -0.5 -balance_points [get_pin reg1/CK] -clock clk -corner worst

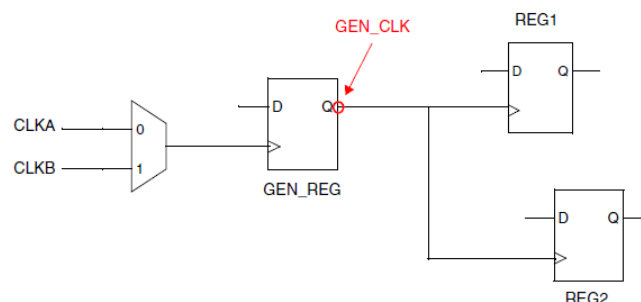
set_clock_balance_points -delay 0.5 -balance_points [get_pin reg2/CK] -clock clk -corner worst

derive_clock_balance_points 也可使用选项调整, -clocks / -corners / -reference_latency / -output

4). 检查

命令 check_clock_trees 检查以下事项:

时钟是否有 sink 点; 时钟结构是否存在循环路径 (Loop); 多时钟到同一触发器; 忽略的 clock tree exceptions; 定义在输出 pin 上的 Stop pin or float pin; Generated clock 缺少源时钟/源时钟与之无路径/在源时钟 pin 上存在多个时钟, 只指定一部分。示例:



对 MCM 设计, 还检查 CTS scenario 设置的冲突。

5). Logic DRC 设置

set_max_capacitance -clock_path ** (默认 0.6pF) [-corners * -modes *]

set_max_transition -clock_path ** (默认 0.5ns)

例: set_max_capacitance 0.6 -clock_path [get_clocks -mode mode2] -corners

[get_corner [get_attribute [get_mode mode2] associated_corners]]

6). CTS 通用设置

1. 设置 CTS 使用单元: **set_lib_cell_purpose -include cts [get_lib_cells *]**。

注意事项: 至少包含 1 个 buffer 和 inverter; 应该包含 preexisting cell, 否则 CTS 器件无法 size_cell; 多电压域设计要有 always-on cells; cell 不能有 don't_touch 属性。

例: 设置 CTS 使用单元, 并该单元在其他步骤不使用:

```
set cts_cells list_of_cells
set_lib_cell_purpose -exclude cts [get_lib_cells]
set_lib_cell_purpose -include none [get_lib_cells $cts_cells]
set_lib_cell_purpose -include cts [get_lib_cells $cts_cells]
```

2. 将与 preexisting cell 逻辑等价的 cell 列入 clock references cell 中。命令:

derive_clock_cell_references

基本步骤:

- (1). **set_lib_cell_purpose -include cts {tech_lib/clk_buf* tech_lib/clk_inv*} /-output ****
- (2). **derive_clock_cell_references /source cts_leq_cells.tcl**

- (3). **synthesize_clock_trees**

第一步必不可少。

3. 局部限制 target library 使用范围。示例如下:

```
set_lib_cell_purpose -include cts {HVT_lib/buf1 HVT_lib/buf2 LVT_lib/buf1 LVT_lib/buf2}
set_target_library_subset -clock {LVT_lib} -objects [TOP/TSK_BLK]
set_app_options -name opt.common.enable_target_library_subset_opt -value 1
```

说明: 全局允许使用 LVT 和 HVT 的 buf1、buf2, 在 TOP/TSK_BLK 模块中只是用 LVT 的。

4. skew / latency 设置

```
set_clock_tree_options -target_skew
set_clock_tree_options -target_latency
```

5. 设定 CTS 主 corner

使用参数 **cts.compile.primary_corner**。

6. 保留原有时钟树 (默认删除)

将 **cts.compile.remove_existing_clock_trees** 置为 false

7. 保留层次设计的时钟 port

示例: **set_freeze_ports -clock [get_cells I2] true**。说明: 综合时 I2 的时钟 port 不会处理。

8. 降低电迁移

设置 **set_clock_cell_spacing** (balance_clock_groups 也遵守该设置), 防止时钟树单元过密摆放; 综合时钟 (synthesize_clock_trees、clock_opt); 检查摆放 **check_legality -verbose**。

9. 早期评估选项

早期评估, 约束往往不精确、悲观, 建议 “**set_app_options -list {cts.common.enable_dirty_design_mode true}**”。该设置会忽略 dont_touch 属性; 工具认为 max cap/tran 过严时, 会自动忽略约束; 如果最长线约束小于 50 微米, 移除; 若果横向间距大于 3 倍, 纵向大于 2 倍时, 忽略 clock cell 距离设置; 如果线宽减 DRC 间距值大于 10 倍默认线宽减 DRC 间距值, 忽略非默认绕线规则; 报告会引入大延时的平衡点。

10. clock cell 间距设置


```
set_clock_cell_spacing -x_spacing * -y_spacing * -lib_cells
```

默认两个间距设置为 5 的单元相邻摆放，距离至少 10。将参数 **cts.placement.cell_spacing_rule_style** 设为 maximum，则两单元最大设置有效。

11. 创建 skew group

当某部分 sink 只局部平衡，创建 skew group。命令：create_clock_skew_group -name **
-objects {*/CK */CK }

12. 设定 CTS 阶段单元名字前缀

```
set_app_options -name cts.common.user_instance_name_prefix -value "CTS_"
```

13. 在 Initial CTS 阶段使用 Global Router During

设置 cts.compile.enable_global_route 为 true。

14. 设置非默认绕线规则

7). 报告时钟设置

使用 report_clock_settings，-type 后不同参数代表报告的不同设置。Configurations 意为 skew、latency、DRC 等；还有 references、spacing_rules、routing_rules 等。

8). 综合时钟树

建议综合前先保存，以免综合后结果不理想迭/代重做。

synthesize_clock_trees: 时钟树综合，优化。该命令使用 global route 引擎评估。结束后令时钟传播，对一些 CTS 无效的 scenario，需要先设为有效，再 synthesize_clock_trees -propagate_only（在时钟综合之后，只是令该 scenario 时钟传播，不再综合时钟）。

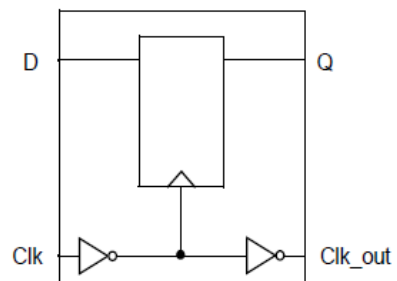
clock_opt: 时钟树综合（build_clock），时钟绕线（route_clock），优化（final_opto）。可以用 -from、-to 控制阶段。

某些设计中，存在结构上就无法平衡的 endpoint，ICC2 自动将一点忽略以解决平衡的矛盾。默认开启，关闭设 cts.common.enable_auto_exceptions 为 false。

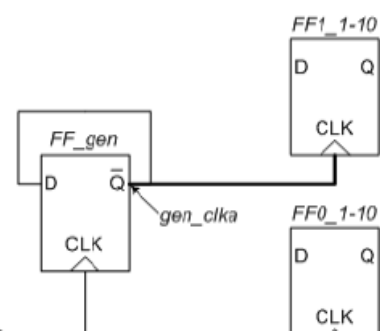
下分两例说明结构平衡矛盾：

1. modules/cells 的输入 pin 和输出 pin 都是同时钟域，并且之间有延时（逻辑路径）

例：顶层调用如图 module，顶层不可能平衡 Clk 和 Clk_out 端，故自动将 Clk 忽略。



2. Generated clock 相关平衡，忽略 master clock。正常理解平衡关系为：FF_gen/CLK 和 FF0_1-10/CLK 在 clkb 中平衡，



FF1_1-10/CLK 和FF0_1-10/CLK 在clka
中平衡。实际中，忽略在clkb中忽略
FF_gen/CLK。

9). 时钟树优化

1. CCD (Concurrent Clock and Data Optimization) 优化 (concurrent, 一致、并行)

useful skew 技术使用前后拍的时序余量来修复时序。普通 useful skew 技术通过调整 clock path 延时来优化时序；同时调整 data path 延时的技术即为 CCD。

开启: set_app_options -name clock_opt.flow.enable_ccd -value true, 对 clock_opt 有效, 不再绕线 (略过 route_clock 阶段), 需要通过命令 route_group 绕线; 或通过以下选项继续绕线:

```
set_app_options -name clock_opt.flow.enable_ccd_route_clock -value true
```

```
set_app_options -name clock_opt.flow.enable_ccd_route_clock_eco -value true
```

开启: set_app_options -name route_opt.flow.enable_ccd -value true, 对 route_opt 有效。

通用选项:

以下选项分别制定 CCD 优化对边界路径无效; 对某些 port 相关路径无效; 对有些 path group 无效 (默认为所有 scenario, 可指定某些 scenario)

```
set_app_options -name ccd.optimize_boundary_timing -value false
```

```
set_app_options -name ccd.ignore_ports_for_boundary_identification -value  
{ $port_name }
```

```
set_app_options -name ccd.skip_path_groups -value { $path_group1 { $path_group2  
$scenarios } }
```

10). 平衡不同时钟

工具可以平衡不同时钟 (可控制之间 skew)。不能平衡 generated clock 和其他时钟。为平衡不同时钟, 需要 interclock delay balancing 约束。

1. 约束:

```
create_clock_balance_group -name -objects [ get_clocks { $clk1 $clk2 } ]
```

或选取该 mode 中所有时钟 -objects [get_clocks -mode \$mode]

代表平衡时钟之间的延时关系 -offset_latencies { 0 -10 }

自动设置 interclock delay balancing 约束: derive_clock_balance_constraints 默认平衡所有有跨时钟域的时钟, 可以指定 “-slack_less_than ** ” 只平衡 slack 此值以下的时钟。

报告: report_clock_balance_groups

2. 平衡: balance_clock_groups

11). CTS 后优化

clock_opt -from final_opto, 不改动时钟树, 仅优化。优化 timing、logical DRC violations、area、power、routability。

12). 时钟详尽绕线

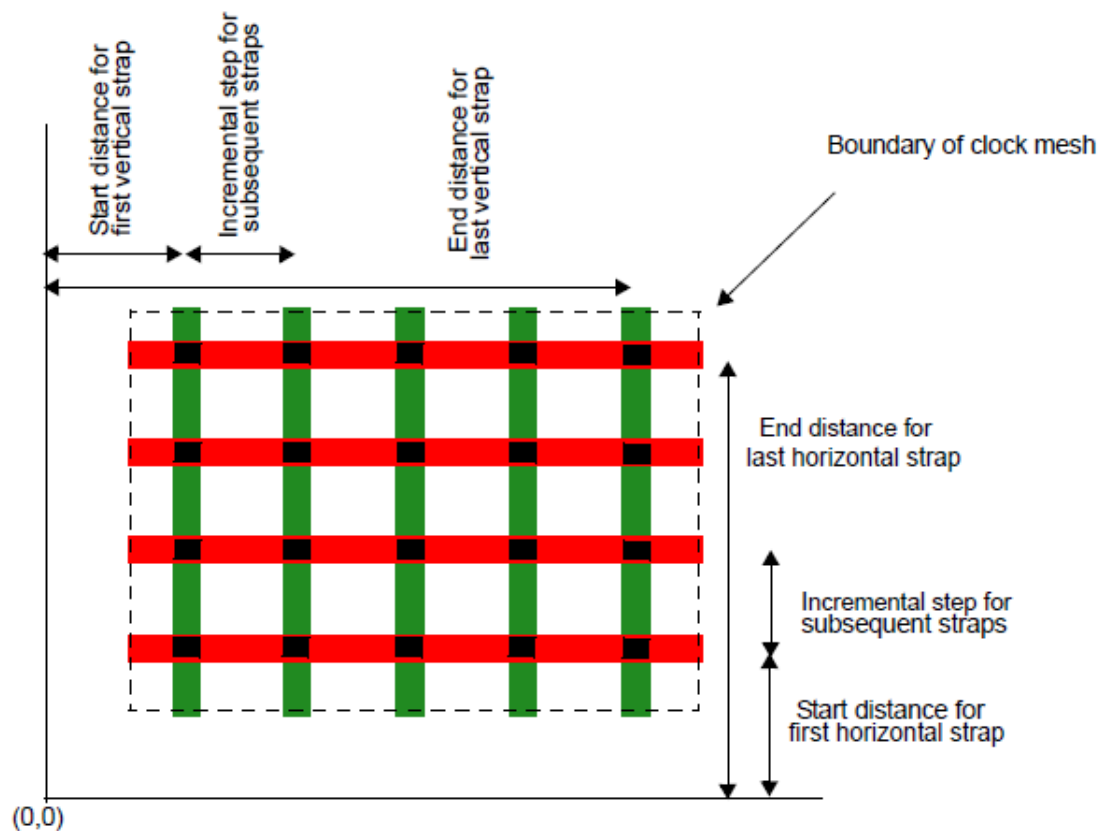
默认 CTS 使用 global route 绕线。可以使用 detail route 引擎。有两种方式:

1. CTS 后, **route_group -all_clock_nets (-reuse_existing_global_route true)**
2. clock_opt -from route_clock -to route_clock

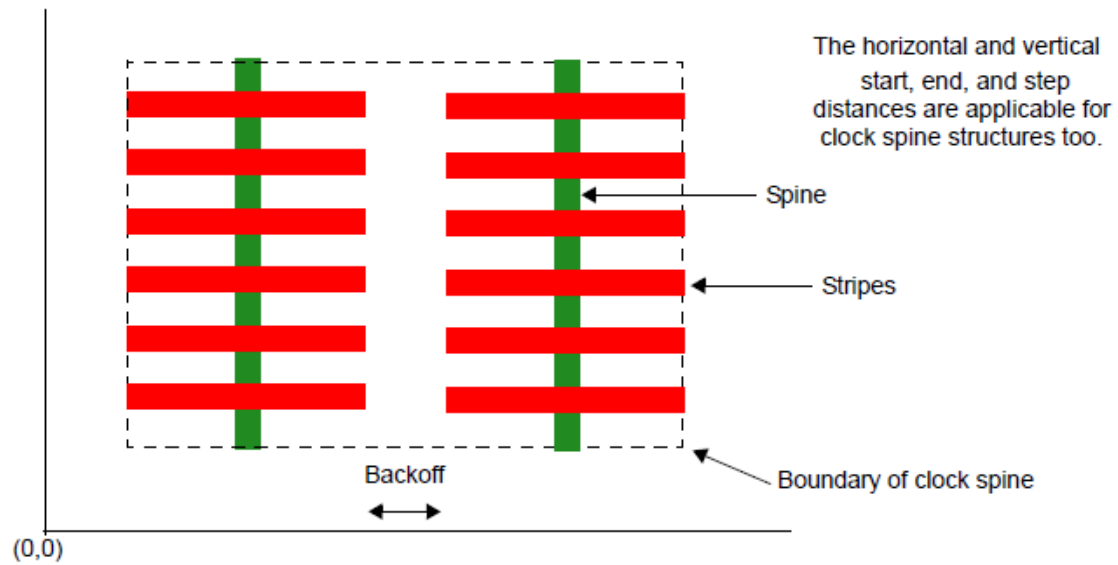
13). 创建 Clock Straps

命令: create_clock_straps。应用在 mesh 和 spine (脊髓型) 时钟结构。下图说明基本结构:

MESH 结构:



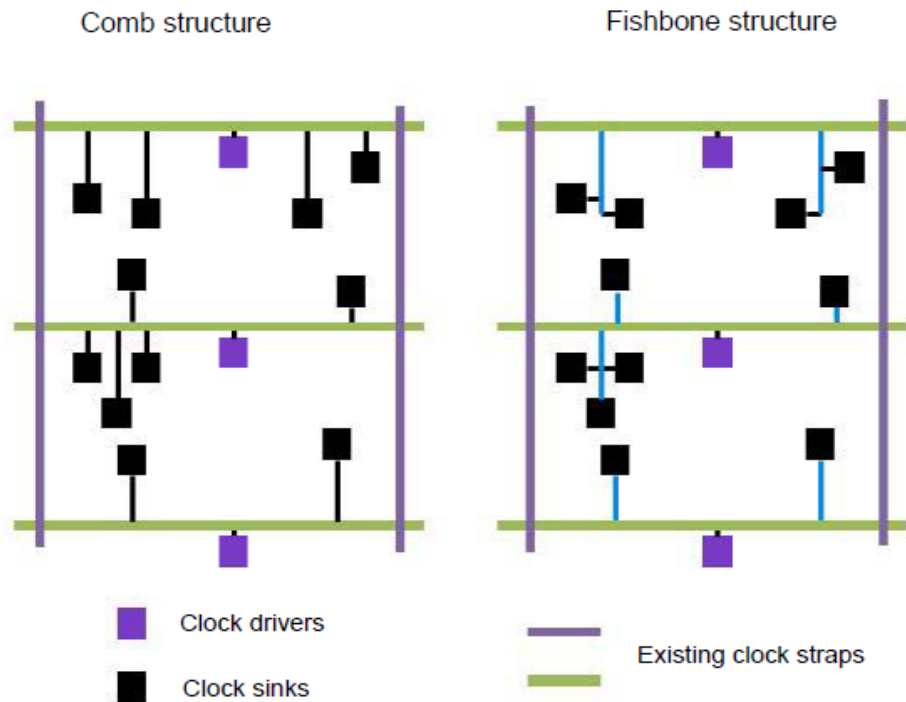
SPINE 结构:



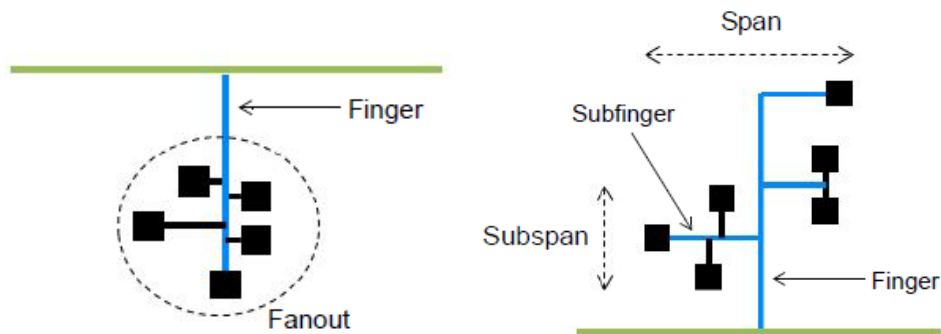
使用选项: `create_clock_straps -net ** -boundary **` (时钟结构生成在此区域) -keepouts (默认遇到 keepout 会打断 strap, 可用 `-allow_splitting false` 阻止) -layers ** -width `-grid x- | y-axis` (指定创建的起始点及方向, 若某方向多 strap, 要使用迭代列表, 其中标注起始点, 终点, 步进) -margins ** (默认是 0, 指定遇到 obstructions 或 keepouts 时可偏离的最大距离) -create_ends (在 boundary 创建 strap) (-allow_floating true 默认 false) -type `user_route | stripe | detect` (详细说明见 man) -length ** (主干最大长度) -backoff ** (上图红色 strap 的最小间距) -spine_direction -bias (屏蔽线) -bias_to_nets ** (用指定线屏蔽) -bias_margins ** (屏蔽线距离) -clear (移除)

14). 绕线连接 Clock Drivers 和 Sinks 到 Clock Straps

命令: `route_clock_straps -nets ** -topology comb` (默认鱼骨型)。支持梳状和鱼骨型连接。示意图如下:



鱼骨型选项: `-fishbone_fanout ** -fishbone_span ** -fishbone_sub_span ** -fishbone_layers`



通用型选项: `-stop_after_global_route true` (对 comb 型, 默认使用 global 和 detail route, 该选项意为在 global route 后停止; 对 fishbone 无效)
`-max_detail_route_iterations 0-1000` (默认 40)

15). 绕线后优化

使用 detail route 对时钟树绕线, 可能 QoR 会较 global route 差 (因为综合引擎是 global route)。可以针对此优化。

`synthesize_clock_trees -postroute -routed_clock_stage detail`
 已有时钟、信号绕线信息 `-routed_clock_stage detail_with_signal_routes`

16). 标注已存在时钟树

对已有的时钟树设计, 可以标注并对此不进行任何操作。命令: `mark_clock_trees`。

17). 删除时钟树

命令：remove_clock_trees。不影响 shape_use 属性设为 user_route 的 shape。对具体时钟树相关单元影响说明：

Object	Impact on clock tree removal
Boundary cell	Removed.
Cells on don't touch net	Preserved.
Don't touch cell	Preserved.
Fixed cell	Preserved.
Generated clock	Preserved, if generated clock is defined on buffer/inverter pin. Traversal and clock tree removal continue past the generated clock.
Guide buffer	Removed.
Integrated clock-gating (ICG) cell	Preserved. Traversal (and clock tree removal) continues past the integrated clock-gating cell.
Block abstraction model	Preserved. Traversal (and clock tree removal) continues past the block abstraction model.
Inverter	Removed in pairs only. If a clock tree contains a single inverter, it is not removed.
Isolation cell	Preserved.
Level shifter	Preserved.
Three-state buffer	Preserved. Traversal (and clock tree removal) stops at the three-state buffer.
Buffer or inverter added beyond exception	Removed.

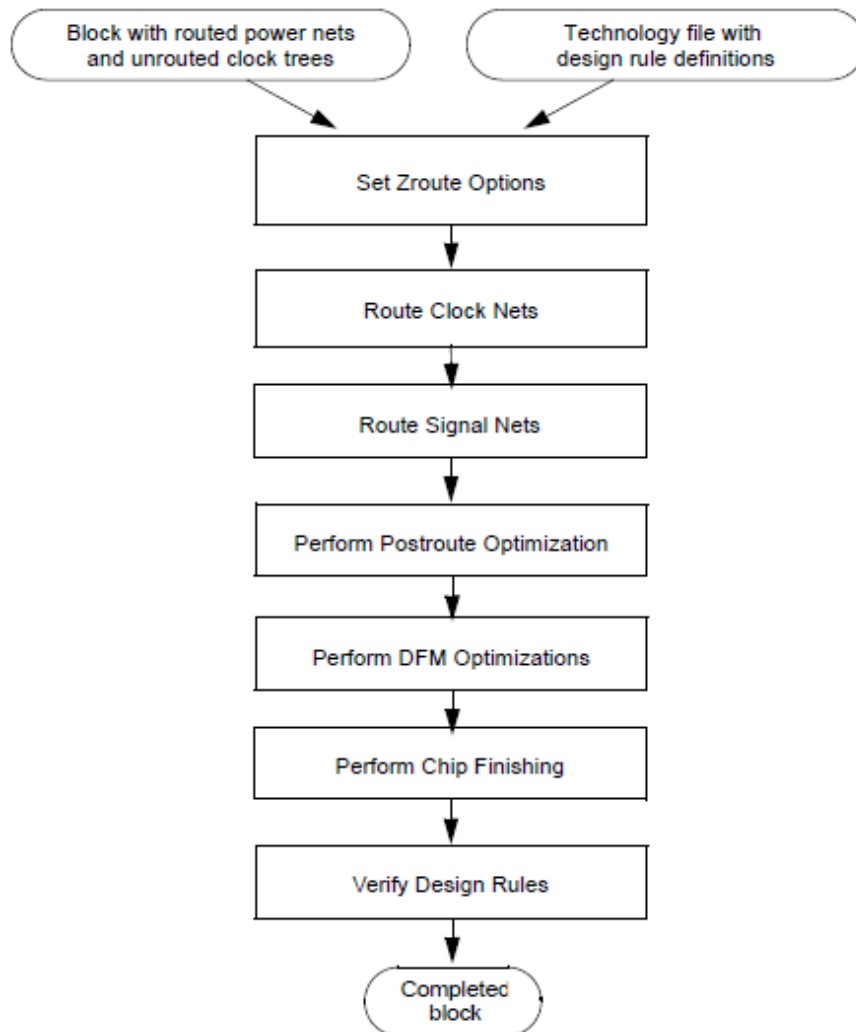
18). 分析时钟树

- 1. 分析 QoR: **report_clock_qor**。该命令报告涵盖：Summary (latency、skew、DRC violations、area、buffer count)、robustness (强健性, 需要有对应的-robustness_corner, -type robustness)
- 2. 收集指定的 sink 点: 命令 **get_clock_tree_pins**。例如: 找 ICG 单元的时钟输入 pin。
get_clock_tree_pins -filter **is_on_ICG** -clocks \$clk
- 2. 分析 Timing: 命令 report_clock_timing。可用选项: -type skew | interclock_skew | latency | transition。
- 3. GUI 分析
Clock Tree Analysis Window: Window > Clock Tree Analysis Window。基本说明。
Clock Tree Schematics: Select the clock on the Clock Browser panel→Click Clock Tree Schematic。可用此查看关键路径的时钟路径及源头, 分析时钟的影响。
Clock tree graph: Select the clock on the Clock Browser panel→Click Clock tree graph。类似 Clock Tree Schematics 的高级抽象。

4. Zroute

1). 基本流程

Zroute 有 5 个引擎，global routing、track assignment、detail routing、ECO routing、routing verification。



2). 准备检查

确认 tf 文件中要使用的孔属性 “isDefaultContact” 为 1，使用非默认设置时，注意 create_routing_rule、add_via_mapping、create_via_def。

模块的 congestion、timing、maximum capacitance 应该基本 clean。

4). 定义 Vias

略，详见 Implementation 文档。

5). 检查 Routability

命令: **check_routability** -standard_cell_search_range ** (默认 10, 检查 cell pin 是否被遮挡。可取消-check_standard_cell_blocked_ports false) -connect_standard_cells_within_pins true (默认 false) -blocked_range ** (默认 40, macro pin 在该层是否遮挡, 可取消-check_non_standard_cell_blocked_ports false) -blocked_range_via_side ** (默认 40, 检查 macro 临近层) -obey_direction_preference true (默认 false) -check_out_of_boundary false (默认 true) -check_min_grid false (默认 true)

额外选项: -check_pg_blocked_ports true -check_frozen_net_blocked_ports true -check_no_net_pins true -check_shield true 等

默认选项即可。

6). Routing 约束

1. routing blockage (控制是否可以走线)

命令: **create_routing_blockage** -boundary { * } -layers * -name * -cell \$reference_cell (应用于所有 reference cell 类别) -zero_spacing (否则 blockage 和线之间有最小间距) -nets * / -net_types **analog_ground | analog_power | analog_signal | clock | deep_nwell | deep_pwell | nwell | pwell | power | ground | reset | scan | signal | tie_high | tie_low** -reserve_for_top_level_routing。在模块级间隙有效, 防止绕线; 在顶层调用 FRAM 时, 忽略模块级 blockage, 即预留出该区域给顶层绕线。

相关指令: get_routing_blockages -of_objects [get_nets *]

2. Routing Guides (控制走线方向, 密度等信息)

命令: **create_routing_guide** -boundary { * } -preferred_direction_only -switch_preferred_direction (应用于 macro 区域效果较好。优先级更高) -max_patterns (限制指定区域边界上某层、在某种走线方向上线的数量。例: {{M2 non_pref_dir_edge 2}{M4 3}} 即为 2 层上非默认方向上边界金属最多有 2 条, 4 层默认方向上边界金属最多 3 条。超出报错 violation) -horizontal_track_utilization (0-100) -vertical_track_utilization (0-100) -access_preference (指定某区域/点绕线优先级。格式: {layer [{wire_access_preference 区域 力度 (0-1)}] [{via_access_preference 点 力度}] ...}。例: {M1 {wire_access_preference {{0 0} {2 1}} 0.2} {via_access_preference {{0 0} {5 5}} 1.0} {via_access_preference {{40 40} {45 45}} 0.5}}) -river_routing (绕线资源紧张时或需要小间距绕线时使用, 尽量压缩绕线空间) -name * -cell \$reference

相关指令: get_routing_guides

3. Routing Corridors (可对关键信号设置)

命令: **create_routing_corridor** -boundary * -path * -width * -min/max_layer_name * -name * (初始创建)

create_routing_corridor_shape -boundary * -routing_corridor *

add_to routing_corridor * [get_nets *]

检查 corridor 连续性及 pin 是否在两端范围内: check_routing_corridors。

4. 非默认绕线约束

非默认约束可以加紧线间距、宽度设置, 自定义 pin tapering distance (线到 pin 逐级缩小) 约束, 对特殊线使用特殊绕线规则, 设置 multiple-patterning mask 约束, 自定义屏蔽线规则。

最小线宽: create_routing_rule \$rule_name [-default_reference_rule | -reference_rule_name ref_rule] (选择参照 rule) -multiplier_width 0.001-2000 (最小线宽为各层默认线宽的倍数。)

最小线宽: create_routing_rule \$rule_name -widths {layer1 width1 ... layern widthn} (直接定义各层线宽) (同时使用-width 和-multiplier_width, 则参照线宽为-width 值)

最小线间距: create_routing_rule \$rule_name [-default_reference_rule | -reference_rule_name ref_rule] -multiplier_spacing 0.001-2000

最下线间距: create_routing_rule \$rule_name -spacings { layer1 {spacing1 spacing2 ... } ... } -spacing_weight_levels { layer1 {weight1 weight2 ... } ... } (权重可为 low | medium | high | hard。默认 Zroute 只修 hard) (同时使用-multiplier_spacing、-spacings 类似上)

可使用 route.common.soft_rule_weight_to_effort_level_map 配置不同 spacing weight 对应的 Zroute fix effort。Effort 可为 off (不修)、low、medium、high (尽量修)。

例: set_app_options -name route.common.soft_rule_weight_to_effort_level_map -value { {low low} {medium medium} {high high} }

孔间距设置: create_routing_rule \$rule -via_spacings {{ \$V1_ref \$V2_ref \$spacing }}

自定义孔: 略。

自定义 multiple-patterning: 可对关键路径设置, 但先进节点对 DRC 要求严格, 不建议使用。create_routing_rule * -mask_constraints {layer constraint}

自定义屏蔽线约束: create_routing_rule * -shield_widths {layer width ... } -shield_spacings {layer spacing ... } [-snap_to_track]

报告: report_routing_rules

移除: remove_routing_rules

5. 对非默认绕线约束检查设置

设置 route.detail.var_spacing_to_same_net 为 true, 对相同信号线图形使用非默认设置检查;

设置 route.common.ignore_var_spacing_to_shield 为 false, 对所有信号线检查 shield 距离;

create_routing_rule -ignore_spacing_to_shield false, 对某条信号线检查 shield 距离;

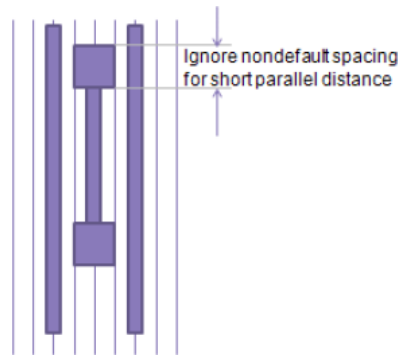
设置 route.common.ignore_var_spacing_to_pg 为 true, 对所有信号线不检查到 PG 距离;

create_routing_rule -ignore_spacing_to_pg true, 对某条信号线不检查到 PG 距离;

设置 route.common.ignore_var_spacing_to_blockage 为 true, 不检查所有信号线到 blockage 距离;

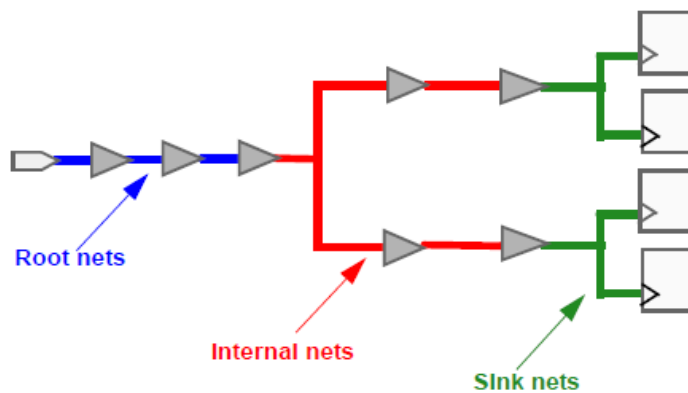
create_routing_rule -ignore_spacing_to_blockage true, 不检查某些信号线到 blockage 距离;

例: 如图可以忽略非默认绕线的短线的检查。可以减少运行时间, 不影响串扰。



6. 应用非默认绕线规则

命令: `set_clock_routing_rules`。应用在 CTS 综合之前。选项: `-clocks * -net_type root | sink | internal -nets *` (root: 在根节点到时钟线分支之前都算。) `-default_rule` (相当于移除)



优先级: 1. 由 `set_routing_rule` 设置; 2. 由 `set_clock_routing_rules -nets` 设置; 3. 由 `set_clock_routing_rules -clocks` 设置; 4. `set_clock_routing_rules` 设置。

命令: `set_routing_rule`。应用在 CTS 综合之后。选项: `-default_rule -no_rule -clear`
报告: `report_routing_rules -of_objects [get_nets *]`

7. 限制 Off-Grid 绕线

通常绕线在 grid 上。可以通过限制绕线在 track 上或增加 Off-Grid 绕线的 cost。

Tech 文件中 `onWireTrack` 或 `onGrid` 为 1 时, wire/via 在 track 上。可通过 `route.common.wire_on_grid_by_layer_name` 和 `route.common.via_on_grid_by_layer_name` { {layer (metal、via) true | false} ... } 控制。

可通过设置 `route.common.extra_via_off_grid_cost_multiplier_by_layer_name` { { layer multiplier (0-20) } ... } 来增加 Off-Grid 绕线的 cost。

8. 控制到 pin 绕线

`set_app_options -name route.common.connect_within_pins_by_layer_name -value` { {layer off | via_standard_cell_pins | via_wire_standard_cell_pins (仅适用于 STD cell) | via_all_pins | via_wire_all_pins} (使用 STD, macro 等) } (via/wire 到 pin 连接要在 pin 形状之内)

9. 控制 Pin Tapering (逐渐缩小)

默认 Zroute 使用 Distance-based, 在 pin 该距离内使用默认绕线规则, 在范围外使用由定义的 `create_routing_rule` 非默认规则。对先进节点, 使用 Layer-based tapering 效果更好。Distance-based 和 layer-based tapering (优先级更高) 默认就相斥。

Distance-based 默认 tapering 距离是 10 倍的该层 track。使用 `create_routing_rule -`

taper_distance * -driver_taper_distance * (应用于驱动 pin) 设置。

Layer-based tapering 使用 create_routing_rule -taper_over_pin_layers * (整数, 对于 pin taper 可使用 “pin + *” 金属层) -taper_under_pin_layers * (整数, 对于 pin taper 可使用 “pin - *” 金属层)

默认 taper 是由非默认绕线规则逐渐缩小到默认绕线规则。设置 route.detail.pin_taper_mode 成 pin_width 则逐渐缩小到 pin 宽度。

设置 route.detail.use_wide_wire_effort_level 为 low 或 high, 设置仅在避免非默认绕线导致 DRC 时使用 tapering。

设置 detail.use_wide_wire_to_input_pin、route.detail.use_wide_wire_to_output_pin、route.detail.use_wide_wire_to_macro_pin、route.detail.use_wide_wire_to_pad_pin、route.detail.use_wide_wire_to_port 为 true, 则禁止此类 port/pin tapering。

create_routing_rule -taper_distance 0。禁止说有线的 tapering。

10. 设置 Rerouting Mode

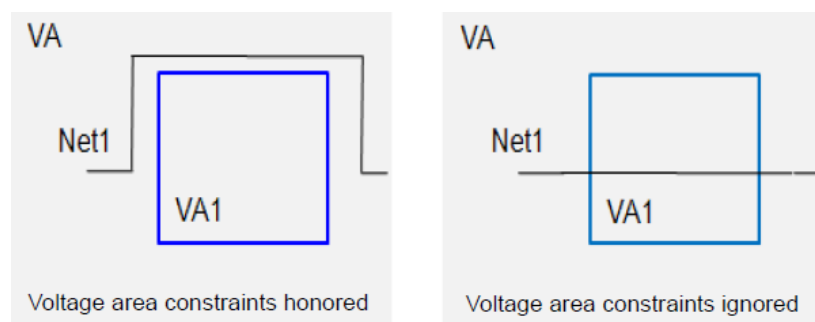
默认 Zroute 会根据需要 reroute, 可以设置 physical_status 来控制。Locked (不改)、minor_change、unrestricted。例: set_attribute -objects [get_nets *] -name physical_status -value locked | minor_change | unrestricted

7). 绕线选项基本分类

绕线选项基本分类: route.common_options、route.global_options、route.track_options、route.detail_options。默认绕线时, 先会报设置的选项, 但不会报默认的、没有修改的选项。set_app_options -name route.common.verbose_level -value 1, 可以报告所有。

8). 多电压域设计

Zroute 支持 voltage area constraints, 默认会遵照 voltage area 绕线。但为修拥塞或 DRC 可以通过非本地 voltage areas 绕线。例右下图:



9). 对 Clock Nets 绕线

使用 “route_group -all_clock_nets” 支持以下几种模式:

Balanced 模式 : synthesize_clock_trees → route_group -all_clock_nets -reuse_existing_global_route true

Comb 模式 : synthesize_clock_trees → set_app_options -name

route.common.clock_topology -value comb → route_group -all_clock_nets
默认 detail router 迭代 40 次，可通过-max_detail_route_iterations 设置。

10). 对 Critical Nets 绕线

使用 route_group -nets * / -from_file *。如果有 routing corridor，建议对其先绕线。
默认 route_group 忽略指定线的 global routes，复用 dangling wires 和 existing detail。

通过“-reuse_existing_global_route true”复用 global routes，但这样会使 routing corridor 无效。通过“-utilize_dangling_wires false”不再复用 dangling wires。通过“-stop_after_global_route true”在 global route 后停止。

默认 route_group 不会修复 soft DRC，设置“set_app_options -name route.common.post_group_route_fix_soft_violations -value true”强制修复。

11). Routing Secondary Power and Ground Pins

多电压域设计中 always cell 在该电压域关断后仍需供电，Secondary Power and Ground Pins 即为始终供电的电源。

1. 首先确定 reference library (frame view) pin 属性正确；

is_secondary_pg 为 true，port_type 为 power 或 ground。使用“report_attributes -application [get_lib_pins -of_objects reflib/cell/pin -all -filter "name == attr_name"]”判断。

2. 设置绕线约束；

set_routing_rule 定义金属层范围；设置到 pin 绕线方式
“route.common.single_connection_to_pins/route.common.connect_within_pins_by_layer_name”

set_app_options -name route.common.number_of_secondary_pg_pin_connections -value *。默认为 0，意为没有限制多少 secondary power or ground pins 连接到同一 power strap 或 ring 上。

设置非默认绕线规则。例：create_routing_rule wideSVDD -widths { M1 0.3 M2 0.3 }
→ set_routing_rule -rule wideSVDD {VDD2}

route.common.separate_tie_off_from_secondary_pg true

默认设置对 Secondary Power and Ground Pins 和 tie-off connections 有效，为使设置只对 Secondary Power and Ground Pins 有效，设置 route.common.separate_tie_off_from_secondary_pg application 为 true。

3. 绕线

connect_pg_net -automatic
route_group -nets *

12). 对 Signal Nets 绕线

可使用分步命令“route_global、route_track、route_detail”或自动绕线命令“route_auto”。

1. route_global

选项为 route.common_options 和 route.global_options 系列。分两步：先 initial routing，再 rerouting 以降低拥塞。-effort_level low | medium (three rerouting) | high (four rerouting) | ultra (six rerouting)。

在 floorplan 阶段，可用 “route_global -floorplan true -virtual_flat all_routing (加上效果更好)” 评估拥塞。

默认 route_global 不是 timing-driven，开启设置 route.global.timing_driven 为 true。

默认 route_global 不是 crosstalk-driven，开启设置 route.global.crosstalk_driven 为 true。

2. Track Assignment

类似。

3. Detail Routing

Detail Routing 在以上基础上评估。将 design 划分区域，分区域评估。在此期间，Zroute 也会按照 routing design rules 和 antenna rules 来优化 via count 和 wire length。

设置 “set_app_options -name route.detail.drc_convergence_effort_level -value high” 强制尽量修复所有 DRC。

设置 “set_app_options -name route.detail.drc_convergence_effort_level -value high” 允许工具使用高层线修复 critical path。

设置 “set_app_options -name route.detail.repair_shorts_over_macros_effort_level -value high” 让工具尽量修 macro 上的短路；或者设为 low，macro 上使用 river routing 的 routing guides，对短路使用 detail route 后处理 (route_detail -incremental true)。

在 detail route 后设置 “set_app_options -name route.common.post_detail_route_fix_soft_violations -value true” 修复 soft DRC，再 route_detail -incremental true (默认不修复。不能修复 open 线，需要 ECO)。

可以通过 “set_app_options -name route.detail.report_ignore_drc -value \$list_of_drcs” 忽略一些 DRC。

4. route_auto

选项为 route.common_options、route.global_options、route.track_options、route.detail_options 系列。常用参数-reuse_existing_global_route true。

对分步命令有效的所有选项在此同样有效。

13). 屏蔽线

(1). 独立设置屏蔽

基本流程：

1. 创建 rule。例：create_routing_rule shield_rule -shield_widths {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3} -shield_spacings {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3}

2. set_clock_routing_rules 对时钟线应用 rule (需在 CTS 前，CTS 后用 set_routing_rule 设置)；set_routing_rule 对信号线应用 rule。

3. create_shields

屏蔽线属性：设置 route.common.shielding_nets (优先级高) 或 create_shields -with_ground (使用地线)；默认屏蔽线与该线之间留 1 个 track。

设置：

屏蔽线宽、间距：设置 route.common.min_shield_length_by_layer_name { {layer

wire_length} ... }

同轴线（上、下）层屏蔽：create_shields -coaxial_above/below true（-coaxial_above/below_skip_tracks 0~7,跨层屏蔽线与该线的水平间距 Track 数,“-1”为失效）

如若屏蔽线违反 DRC, 默认会按照 DRC 设置修改。

阻止屏蔽线连接到 std cell pin/rail: -ignore_shielding_net_pins/rail true

按照 shielded 线形状屏蔽: -align_to_shape_end true

屏蔽线按照各层默认方向绕线: -preferred_direction_only true

报告:

默认 create_shields 后就会报告 shield 情况。例如:

Shielded 82% side-wall of (reset)

Shielded 96% side-wall of (clk)

Shielded 2 nets with average ratio as follows.

1) 89.00% (total shield ratio/number of shielded nets)

2) 87.92% (total shield length/total shielded net length)

报告指令: report_shields (-per_layer true, 按层报), 以此为准, 与 “create_shields” 之间可能有 3% 的偏差。

默认的电地结构不算屏蔽线, 可以将之也算做屏蔽线, 报整体的屏蔽率。设置距离多大范围内的算屏蔽线。set_app_options -name route.common.pg_shield_distance_threshold -value ** (单位: 微米)

(2). 绕线过程中屏蔽

Zroute 在 signal route 时将 shield rule 算作 soft rules。故在 Zroute 时, 识别出某条线有 shield rule 但未屏蔽, 在 global routing、track assignment、detail routing 阶段会预留出空间, 在 detail routing 阶段末尾, 会对此检查并报告 violation, 但不会自动插屏蔽线, 仍需使用 “create_shields”。报告示例:

```
DRC-SUMMARY:
@@@@@@@ TOTAL VIOLATIONS = 13
Diff net var rule spacing : 2
Same net spacing : 2
Less than minimum area : 4
Short : 1
Soft spacing (shielding) : 2
```

设置 route.common.allow_pg_as_shield 为 true 可以减少 Zroute 为屏蔽线预留空间。

默认的 Zroute 不会对 detail route 中改动的线（由外部变更的不算, 例读 DEF）重新绕线, 设置参数 route.common.resield_modified_nets（默认 off）可以控制。

设 route.common.resield_modified_nets 为 unshield → 只删存在的屏蔽线

设 route.common.resield_modified_nets 为 resield → 删之后中心绕线

(3). 绕线后屏蔽

绕线后屏蔽, 使用 “create_shields”。对于因此引出的 DRC, 会调用 detail routing 修 5 次。若仍存在 DRC, 使用 route_detail -incremental true 调用 detail routing 继续修, 更好的选

择是“route_eco”，因为 route_detail 会打断 tie-off 连接。

(4). 示例脚本

```
# Define shielding rule
    create_routing_rule shield_rule -shield_widths {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6
0.3} -shield_spacings {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3}
# Assign shielding rule to clock nets
    set_clock_routing_rules -clocks CLK -rule shield_rule
# Perform clock tree synthesis
    synthesize_clock_trees
# Route the clock nets, reusing the global routing result
    route_group -all_clock_nets -reuse_existing_global_route true
# Perform preroute shielding for the clock nets
    create_shields -nets $clock_nets -with_ground VSS
# Assign shielding rule to critical nets
    set_routing_rule -rule shield_rule $critical_nets
# Route signal nets using shielding soft rules
    route_auto
# Perform postroute shielding for critical nets
    create_shields -nets $critical_nets -with_ground VSS
```

(5). 检查

设置 route.common.check_shield 为 true，Zroute 阶段检查屏蔽情况。若果设置某条线有屏蔽而实际没有，检查线的属性。

信号线的 shape_use 属性应为 shield_route；

电地网络的 shape_use 属性应为 detail_route。

14). 绕线后优化

绕线后优化的基础：已绕线(route_auto 或分立的 route_global、route_track、route_detail)，延时传播（**compute_clock_latency**）。

绕线后优化指令：**route_opt**。

主要操作如下：

计算寄生参数，并按照传播延时更新时序信息；

努力取得更号的 QoR（setup、hold、area、logical DRC violations）；

Legalization、ECO 绕线（默认使用 detail route，可设 route_opt.eco_route.mode 选择 track）；

默认不优化串扰（优化：设 route_opt.flow.xtalk_reduction 为 true）；

默认不优化泄露功耗（优化：设 route_opt.flow.enable_power 为 true）；

默认仅优化 data path（同时优化时钟路径（concurrent clock and data：CCD），设 route_opt.flow.enable_ccd 为 true）

15). ECO 绕线

ECO 缩写 engineering change order。

ECO 绕线指令: `route_eco`。

本质上对修改的事项进行 `global routing`、`track assignment`、`detail routing`。

主要操作如下:

默认考虑时序和串扰, 所以需要计算寄生参数并传播延时更新时序信息, 这步花费较长时间, 可以考虑不进行此步骤 (设 `route.global.crosstalk_driven`、`route.global.timing_driven`、`route.track.crosstalk_driven`、`route.track.timing_driven`、`route.detail.timing_driven` 为 `false`);

默认对多有开路线操作 (可设 `-nets *`);

默认忽略已有 `global route` (`-reuse_existing_global_route true`, 在源基础上进行 `global route`);

默认使用浮空线来缓解拥塞 (`-utilize_dangling_wires false`, 禁用);

最大迭代 40 次修复 (自定义 `-max_detail_route_iterations`);

默认对多有违反 `hard DRC` 的线有效 (只修开路, `-open_net_driven true`; 约束应用到那类线, `-reroute modified_nets_only` (最常用) | `modified_nets_first_then_others`);

默认不对 `soft DRC` 修复 (设 `route.common.post_eco_route_fix_soft_violations` 为 `true`);

默认报修改的 100 条线 (自定义: `-max_reported_nets`, “-1” 代表不报)。

16). GUI 操作

主要针对线的修改操作。

Area Push tool → 。功能: 在保证连接性的前提下, 令指定线远离画出的矩形。

Spread Wire tool → 。功能: 在保证连接性的前提下, 令指定线在某区域内均匀排布。

Stretch Connected tool → 。功能: 在保证连接性的前提下, 拉伸线。

Quick Connect tool → 。功能: 快速连接线到 `pin`、`port`、`terminal` 或其他线。

17). 删除冗余绕线

命令: `remove_redundant_shapes`

默认会根据存储的 `DRC` 信息删除 `dangling and floating` 线。使用选项 “`-initial_drc_from_input false`”, 调用 “`check_routes`” 重新生成 `DRC` 信息。

其他选项: `-nets ** -layers ** -route_types fixed_route/nonfixed_route -remove_dangling_shapes false -remove_floating_shapes false -remove_loop_shapes true` (默认 `false`) `-report_changed_nets true` (默认不会报删掉哪些线)

不会影响 `terminal`, 也不会对开路线和存在 `DRC` 的线操作。

18). 分析绕线结果

(1). 拥塞报告

命令: `report_congestion`。

选项: `-rerun_global_router -boundary ** -layers **`。

GUI 观察效果更好、直观。View > Map > Global Route Congestion。对应命令: `route_global -congestion_map_only true`。

主要设置检查的层（尊重 `set_ignored_layers`）和设置 “Max threshold”。例如设置 7。2 层实际需要 13 条 track，超出 6；3 层需要 9 条 track，超出 2；4 层需要 5 条 track。则计算拥塞时，拥塞值为 “6+2”。

所以设置 “Max threshold” 应小于 GRC 每边的最大允许 track。

(2). DRC 报告

命令: `check_routes`（GUI: Route > Verify Route）

默认除了 user nets、frozen nets、PG 属性的线外，检查 DRC violations、unconnected nets、**antenna rule violations**、**voltage area violations**。

选项: `-drc false -open_net false -antenna false -voltage_area false -check_from_user_shapes true -check_from_frozen_shapes true -coordinates -nets`

当今检查某区域时，值检查 DRC violations、voltage area violations。

(3). Signoff DRC 检查

调用 ICV 工具。检查命令: `signoff_check_drc`；修复命令: `signoff_fix_drc`。

(4). LVS（Layout-Versus-Schematic）检查

命令: `check_lvs`。

主要检查:

短路线（默认不检查顶层和模块等线的短路，设 `-check_child_cells true` 检查，设 `-exclude_child_cell_types abstract、analog、black_box、corner、cover、diode、end_cap、fill、filler、flip_chip_driver、flip_chip_pad、lib_cell、macro、module、pad、pad_spacer、physical_only、well_tap` 排除对部分类型模块的检查）；开路线；Floating net shapes（物理上没有连接到相应线属性的线/pin 上）

默认报告 20 条违例。

(5). 整体报告

命令: `report_design -routing`。

默认仅报顶层，需要报告模块级，-hierarchical。

19). 保存 route 信息

命令：write_routes。该命令生成一脚本，对所有线和孔描述，包括其设置、参数。但不会写出 blockage。如需 blockage 信息，需要 write_floorplan。

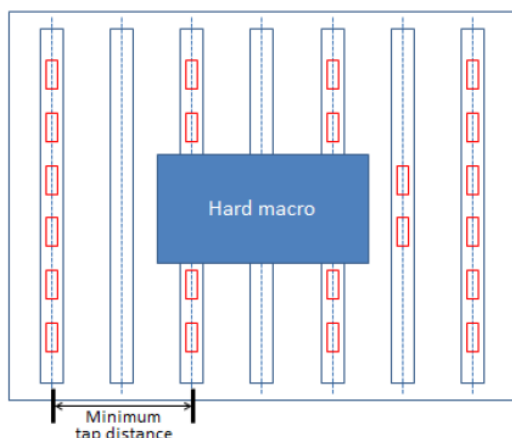
5. DFM

1). 插 Tap Cells

Tap cell 一般分 well 连接和衬底连接（或兼有）这几种类型。通常对应工艺有相应 DRC 约束最大 tap cell 之间距离。最好在 place 前，floorplan 阶段预先插好 tap cell。

命令：create_tap_cells -lib_cell \$myreflib/\$mytapcell -distance **

默认行为为每行都会插，遇到 macro，在 macro 右侧插。示意图如下：



控制选项：插的模式：-pattern every_row (the default)、every_other_row、stagger（推荐）；-at_distance_only（只能在整数，二分之一，四分之一 distance 上插，违反有 DRC）；-voltage \$area

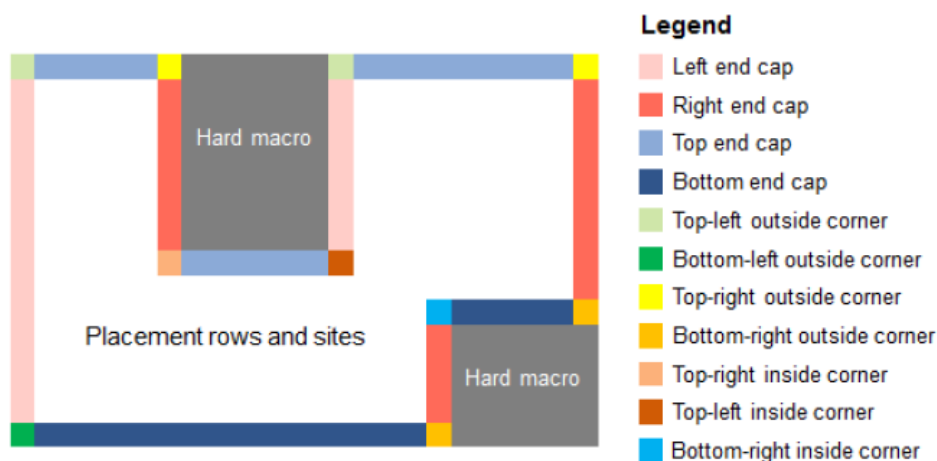
默认 tap cell 的命名原则：tapfiller ! library_cell_name ! number

2). 插 Boundary Cells

Boundary Cells 包含 **end-cap cells**（插在 site rows 边界和 core area、hard macros、blockages、voltage areas 左右周围）和 **corner cells**（连接水平和竖直方向的 end-cap cells）。Boundary Cells 也应该在 floorplan 阶段预插。单元作用：提供去耦电容，冗余的标准单元也可起到去耦的作用。

命令：create_boundary_cells -left/right/top/bottom_boundary_cell -left/right/top/bottom_left_outside/inside_corner_cell

示意图：



控制选项: `-at_va_boundary` (在 voltage area 两侧); `-min_row_width **` (阻止在较短的 row 中插); `-do_not_swap_top_and_bottom_inside_corner_cell` (阻止倒装芯片 top-bottom 互用); 旋转选项 (例: `-mirror_left_boundary_cell`、`-mirror_right_boundary_cell`、`-mirror_left_outside_corner_cell`、`-mirror_right_outside_corner_cell`、`-mirror_left_inside_corner_cell`、`-mirror_right_inside_corner_cell`, 不能旋转 top 和 bottom 的)。

默认命名规则: `boundarycell ! library_cell_name ! number`

3). 修复天线效应

天线效应是由于芯片多层金属工艺中逐渐积累的静电荷会击穿栅氧, 最终导致芯片失效。为避免该效应, 工具会对输入端检测:

$$(\text{antenna-area}) / (\text{gate-area}) < (\text{max-antenna-ratio})$$

(1). 定义天线效应约束

天线效应约束分为全局约束和各层约束。

全局约束: `define_antenna_rule -diode_mode 0~12 -metal_ratio * -cut_ratio * -mode 1~6`

“-mode”代表计算金属面积的方式。下图为各模式说明:

Antenna mode value	Area calculation mode	Antenna recognition mode
1	Surface	Single-layer
2	Surface	Accumulative ratio
3	Surface	Accumulative area
4	Sidewall	Single-layer
5	Sidewall	Accumulative ratio
6	Sidewall	(累计) Accumulative area

其中 Surface area: $W \times L$; Sidewall: $(W + L) \times 2 \times \text{厚度}$ (需要确定工艺文件中存在 `unitMinThickness`、`unitNomThickness`)。

“-diode_mode”各项含义如下图:

Diode protection mode	Definition
0	The diodes do not provide any protection.
1	The diodes provide unlimited protection. In this mode, the highest metal layer is not checked for antenna violations because the input-to-output connection is completed when the highest metal layer is formed.
2	Diode protection is limited; if more than one diode is connected, the largest value of the maximum antenna ratio for all diodes is used.
3	Diode protection is limited; if more than one diode is connected, the sum of the maximum antenna ratios for all diodes is used.
4	Diode protection is limited; if more than one diode is connected, the sum of the diode-protection values for all diodes is used to compute the maximum antenna ratio.
5	Diode protection is limited; the maximum diode-protection value for all diodes is used to calculate the equivalent gate area.
6	Diode protection is limited; the sum of the diode-protection values for all diodes is used to calculate the equivalent gate area.
7	Diode protection is limited; the maximum diode-protection value for all diodes is used to calculate the equivalent metal area.
8	Diode protection is limited; the sum of the diode-protection values for all diodes is used to calculate the equivalent metal area.
9	Diode protection is limited; scaling is based on maximum diode protection.
10	Diode protection is limited; scaling is based on total diode protection.
11	Diode protection is limited; scaling is based on maximum diode protection.
12	Diode protection is limited; scaling is based on total diode protection.

各层约束：define_antenna_layer_rule（优先级更高）-layer * -mode * -diode_ratio *（默认{0 0 1 0}）

“-diode_ratio” 普通模式：{v0 v1 v2 v3 [v4]}; 11、12 模式：{{v0 v1 v2 v3 [v4]} {s0 s1 s2 s2 s4 s5}}。具体说明如下图示意：

Diode mode	Calculation
0, 1	Not used
2, 3, 4	allowable max-antenna-ratio - If $dp > v0$ and $v4 < 0$, allowable max-antenna-ratio = $\min(((dp + v1) * v2 + v3), v4)$ - If $dp > v0$ and $v4 = 0$, allowable max-antenna-ratio = $(dp + v1) * v2 + v3$ - If $dp \leq v0$, allowable max-antenna-ratio = layerMaxRatio
5	$antenna_ratio = metal_area / (gate_area + equi_gate_area)$ - If $max_diode_protection > v0$ and $v4 < 0$, $equi_gate_area = \min(((max_diode_protection + v1) * v2 + v3), v4)$ - If $max_diode_protection > v0$ and $v4 = 0$, $equi_gate_area = (max_diode_protection + v1) * v2 + v3$ - If $max_diode_protection \leq v0$, $equi_gate_area = 0$
6	$antenna_ratio = metal_area / (gate_area + equi_gate_area)$ - If $total_diode_protection > v0$ and $v4 < 0$, $equi_gate_area = \min(((total_diode_protection + v1) * v2 + v3), v4)$ - If $total_diode_protection > v0$ and $v4 = 0$, $equi_gate_area = (total_diode_protection + v1) * v2 + v3$ - If $total_diode_protection \leq v0$, $equi_gate_area = 0$
7	$antenna_ratio = (metal_area - equi_metal_area) / gate_area$ - If $max_diode_protection > v0$ and $v4 < 0$, $equi_metal_area = \min(((max_diode_protection + v1) * v2 + v3), v4)$ - If $max_diode_protection > v0$ and $v4 = 0$, $equi_metal_area = (max_diode_protection + v1) * v2 + v3$ - If $max_diode_protection \leq v0$, $equi_metal_area = 0$ - If $equi_metal_area > metal_area$, $equi_metal_area = metal_area$
8	$antenna_ratio = (metal_area - equi_metal_area) / gate_area$ - If $total_diode_protection > v0$ and $v4 < 0$, $equi_metal_area = \min(((total_diode_protection + v1) * v2 + v3), v4)$ - If $total_diode_protection > v0$ and $v4 = 0$, $equi_metal_area = (total_diode_protection + v1) * v2 + v3$ - If $total_diode_protection \leq v0$, $equi_metal_area = 0$ - If $equi_metal_area > metal_area$, $equi_metal_area = metal_area$
9	$antenna_ratio = scale * metal_area / gate_area$ - If $max_diode_protection > v0$, $scale = \max(1 / ((max_diode_protection + v1) * v2 + v3), v4)$ - If $max_diode_protection \leq v0$, $scale = 1.0$

- 10 antenna_ratio = scale * metal_area / gate_area
- If total_diode_protection > v0,
 scale = max (1 / ((total_diode_protection + v1) * v2 + v3), v4)
 - If total_diode_protection <= v0,
 scale = 1.0
- 11 antenna_ratio = scale * metal_area / gate_area
- If max_diode_protection < v0,
 scale = s0
 - If max_diode_protection < v1,
 scale = s1
 - If max_diode_protection < v2,
 scale = s2
 - If max_diode_protection < v3,
 scale = s3
 - If max_diode_protection < v4,
 scale = s4
 - If max_diode_protection >= v4,
 scale = s5
- 12 antenna_ratio = scale * metal_area / gate_area
- If total_diode_protection < v0,
 scale = s0
 - If total_diode_protection < v1,
 scale = s1
 - If total_diode_protection < v2,
 scale = s2
 - If total_diode_protection < v3,
 scale = s3
 - If total_diode_protection < v4,
 scale = s4
 - If total_diode_protection >= v4,
 scale = s5
-

示例:

```
set lib [current_lib]
define_antenna_rule $lib -mode 1 -diode_mode 4 -metal_ratio 300 -cut_ratio 20
define_antenna_layer_rule $lib -mode 1 -layer "M1" -ratio 300 -diode_ratio {0.09 0 123 16880}
define_antenna_layer_rule $lib -mode 1 -layer "M2" -ratio 300 -diode_ratio {0.09 0 123 16880}
define_antenna_layer_rule $lib -mode 1 -layer "VIA1" -ratio 20 -diode_ratio {0.09 0 110 500}
```

(2). 天线效应属性

默认 std 和 macro 的天线效应属性在 FRAM 中。对于缺少默认属性的,可以手工自定义。
天线效应相关属性:

```
route.detail.default_diode_protection
route.detail.default_gate_size
route.detail.default_port_external_antenna_area
route.detail.default_port_external_gate_size
route.detail.macro_pin_antenna_mode
```

route.detail.port_antenna_mode

层次化设计使用 block abstract 的，要使用 derive_hier_antenna_property 抽取模块的天线效应参数，以便顶层使用。

如果模块的天线效应参数不完整，则 Zroute 会跳过天线效应分析，报 **ZRT-311** warning。遇到这种情况要 debug，详见《SolvNet article 027178, “Debugging the ZRT-311 Message”》。

(3). 分析、修复天线效应

默认 Zroute 的 detail routing 会分析、修复天线效应。不考虑天线效应，设 route.detail.antenna 为 false。

默认对所有信号、时钟线分析。若某条线不考虑，设 route.detail.skip_antenna_fixing_for_nets；默认不分析电地线，若需要分析，设 route.detail.check_antenna_on_pg 为 true；默认在第二遍迭代时修复天线效应，此时 DRC 已基本修复，设 route.detail.antenna_on_iteration 决定修复天线效应在第几次迭代；默认使用跳线（hop）修复；默认也会使用二极管修复（由 route.detail.insert_diodes_during_routing 控制），默认优先使用跳线修（优先级可改为插二极管设 route.detail.antenna_fixing_preference 为 use_diodes）；

在 detail route 阶段结束会报基本概况，示意图如下：

DRC-SUMMARY:

@@@@@@ TOTAL VIOLATIONS = 506

@@@@ Total number of instance ports with antenna violations = 1107

在 **detail route 阶段期间**，在栅附近插二极管消除天线效应。需要设 route.detail.insert_diodes_during_routing 为 true；设 route.detail.antenna_fixing_preference 为 use_diodes；设 route.detail.hop_layers_to_fix_antenna 为 false。

设置使用什么类型的二极管（新插/spare cell）：设 route.detail.diode_preference new/spare/none(重置)，设 route.detail.diode_insertion_mode new/spare/new_and_spare(重置)。

设置使用二极管 reference：设 route.detail.diode_libcell_names。

默认工具插二极管时复用 一个位置点，取消此设置，设 route.detail.reuse_filler_locations_for_diodes 为 false。

默认工具识别电压域和层次，不会混淆。如果顶层分析，工具可以插在同一电压域的下一层次，设 route.detail.use_lower_hierarchy_for_port_diodes 为 true。

在 **detail route 阶段后**，可以手动修复某些点。使用命令：create_diodes {port_name instance_name diode_reference number_of_diodes max_routing_layer max_routing_distance}。

影响控制参数：route.detail.diode_insertion_mode、route.detail.reuse_filler_locations_for_diodes、route.detail.use_lower_hierarchy_for_port_diodes。

示例：create_diodes {{A CI Adiode 2 M5 2.5}}，意为距离 CI 单元 A 端 2.5 插 2 个二极管，最高使用绕线层为 M5。

检查天线效应违例：check_routes -antenna true

4). 打多孔

Zroute 全过程都优化线长和孔数。最后阶段的打多孔可以提高 DFM 的成品率，对于设计中不能打多孔的地方要关注原因。

(1). 时钟线打多孔

1. 设置时钟线的绕线规则（create_routing_rule 包含多孔规则）→2. 应用到时钟线上（set_clock_routing_rules -rule *）→3. 分析时钟树（synthesize_clock_trees）→4. 对时钟树绕线（route_group -all_clock_nets -reuse_existing_global_route true）

Zroute 在 global route 阶段预留多孔的空间，在 detail route 阶段实际插多孔。

(2). 信号线打多孔

在信号线上打多孔有以下几种方式：Postroute阶段插多孔；Concurrent soft-rule-based 打多孔；hard-rule-based 打多孔（几乎 100%多孔率）。

通常使用 postroute 打多孔，以达到 80%左右的多孔率；为达到更高多孔率，使用 Concurrent soft-rule-based 方式；为更高多孔率，最后再尝试 hard-rule-based 方式。

过高的多孔率会导致 DRC 问题和串扰问题（也可能导致 floorplan 额外需要一些面积），故只有一些特殊需要的模块可能达到 100%的多孔率。

查看 Via Mapping Table:

默认 Zroute 读取工艺文件中孔的信息，并以此产生优化的 Via Mapping Table。查看该对应表，命令：add_redundant_vias -list_only true。通常自定的 mapping table 优于默认的。下图为 mapping table 示意图：

```
...
Redundant via optimization will attempt to replace the following vias:
VIA12SQ_C -> VIA12SQ_C_2x1    VIA12SQ_C_2x1(r) VIA12SQ_C_1x2    VIA12SQ_C_1x2(r)
VIA12SQ_C(r) -> VIA12SQ_C_2x1 VIA12SQ_C_2x1(r) VIA12SQ_C_1x2    VIA12SQ_C_1x2(r)
...
VIA89(r) -> VIA89_C_1x2(r)    VIA89_1x2(r)    VIA89_1x2    VIA89_C_1x2
              VIA89_C_2x1(r)    VIA89_2x1(r) VIA89_2x1    VIA89_C_2x1
VIA9RDL -> VIA9RDL_2x1 VIA9RDL_1x2
```

自定义 Via Mapping Table:

命令：add_via_mapping -from * -to * -weight 1~10 -transform all（默认在插多孔过程中，Zroute 可以旋转、镜像）|rotate|flip|none -force（重复的强制覆盖）

示例：add_via_mapping -from {VIA12 1x1} -to {VIA12T 2x1} -weight 5

add_via_mapping -from {VIA12 1x1} -to {VIA12 2x1} -weight 1

自定义孔：create_via_def

报告：report_via_mapping（默认报自定义）

移除：remove_via_mappings

Postroute 阶段插多孔:

命令：add_redundant_vias -nets * -effort *（high 较 medium 有 3~5%多孔率的提升，

但在 45nm 后工艺会造成光刻良率下降)

设置 `route.detail.optimize_wire_via_effort_level` 为 `true` 也会提升多孔率, 通过减小线长的方式进行。

在 `postroute` 阶段后, 设 `route.common.post_detail_route_redundant_via_insertion` 为 `low|medium|high`, 原始默认为 `off`, 该选项开启后续 ECO route 中打多孔。

Concurrent soft-rule-based 打多孔:

这种策略会提升多孔率, 一般用在 `initial routing` 和 `ECO routing` 阶段, 以增长运行时间、预留空间为代价。一般设计不建议, 只有设计要求多孔率很高时 (近乎 100%) 使用。使用的前提是 `postroute` 打多孔, 多孔率能达到 80% 以上。

基本流程:

1. 定义 via mapping table;
2. 开启 `concurrent soft-rule-based` (默认关闭)

在 `initial` 阶段开启 (`place_opt` 前设置): `set_app_options -name route.common.concurrent_redundant_via_mode -value reserve_space`

使用 `route.common.concurrent_redundant_via_effort_level` (默认 `low`) 控制程度。

在 `ECO` 阶段开启: `set_app_options -name route.common.eco_route_concurrent_redundant_via_mode -value reserve_space`

使用 `route.common.eco_route_concurrent_redundant_via_effort_level` 控制程度。

3. 绕线
4. 执行 `postroute` 打多孔。

Hard-rule-based 打多孔:

这种策略把多孔当做硬性约束, 能达到几乎 100% 的多孔率。`ECO` 阶段不支持该种方式。这种方式会导致运行时间很长, 且绕线拥挤。这种策略的前提是 `Concurrent soft-rule-based` 方式能达到 90% 以上的多孔率。

基本流程:

1. 开启 `Hard-rule-based` (`place_opt` 前)

`set_app_options -name route.common.concurrent_redundant_via_mode -value insert_at_high_cost`

用 `route.common.concurrent_redundant_via_effort_level` 控制程度。

2. 绕线

多孔对时序的影响:

对短线, 多孔增大的电容使延时边长; 对长线, 多孔减小的电阻使延时变短。`Zroute` 提供控制选项, 控制打多孔对时序影响的程度。

`add_redundant_vias -timing_preserve_setup_slack_threshold * -timing_preserve_hold_slack_threshold * -timing_preserve_nets *`

这种策略可以保证打多孔对时序的影响在可控的范围内。前提是时序基本收敛。

报多孔率:

在打多孔后, `Zroute` 会报一下信息:

非默认孔的转化率 (单孔 → 多孔); 每层的转化率; 各权重孔的比例; 整体转化率。示例如下:

```
Total optimized via conversion rate = 96.94% (1401030 / 1445268 vias)
Layer V01      = 41.89% (490617 / 1171301 vias)
Weight 10      = 9.64% (112869 vias)
Weight 5       = 32.25% (377689 vias)
```

Weight 1 = 0.01% (59 vias)
 Un-optimized = 58.11% (680684 vias)
 Layer V02 = 76.20% (1567822 / 2057614 vias)
 Weight 10 = 43.51% (895270 vias)
 Weight 5 = 28.62% (588805 vias)
 Weight 1 = 4.07% (83747 vias)
 Un-optimized = 23.80% (489792 vias)
 Layer V03 = 81.87% (687115 / 839297 vias)
 Weight 10 = 64.50% (541369 vias)
 Weight 5 = 10.75% (90224 vias)
 Weight 1 = 6.62% (55522 vias)
 Un-optimized = 18.13% (152182 vias)
 Layer V04 = 81.60% (226833 / 277977 vias)
 Weight 10 = 81.45% (226418 vias)
 Weight 1 = 0.15% (415 vias)
 Un-optimized = 18.40% (51144 vias)
 ...
 Layer V09 = 85.47% (1329 / 1555 vias)
 Weight 10 = 85.47% (1329 vias)
 Un-optimized = 14.53% (226 vias)
 Total double via conversion rate = 46.69% (2158006 / 4622189 vias)

5). 优化线长、孔数

在 detail routing 和 redundant via insertion 阶段后，可以进一步优化线长和孔数以提高良率。命令：**optimize_routes** -nets * -reroute_all_shapes_in_nets（对所有相连线重新绕线） -max_detail_route_iterations *（默认修 40 次）

6). 降低关键区域（Critical Areas）

关键区域代表该区域出错会导致整个芯片失效，一般分短路型和开路型。

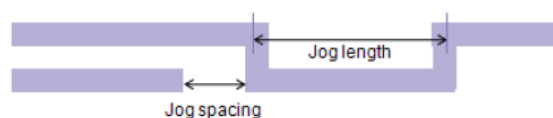
降低由短路决定的关键区域的方法是线摊开放；

降低由开路决定的关键区域的方法是增大线宽；

在 detail routing 和 redundant via insertion 阶段后，使用 **spread_wires** 将线摊开放。

spread_wires -pitch 1.5 → 按照之前线 pitch 的 1.5 倍摆放；

spread_wires -min_jog_length 3（pitch 的倍数） -min_jog_spacing_by_layer_name {{M1 0.07（微米）} {M2 0.08}} →



有时序控制选项：-timing_preserve_setup/hold_slack_threshold、-timing_preserve_nets。

在 detail routing、redundant via insertion 和 spread_wires 阶段后，使用 **widen_wires** 增大

线宽。

示例：widen_wires -widen_widths_by_layer_name {{M1 0.07 0.06} {M2 0.08 0.07}}。
最多每层支持 5 种线宽。

有时序控制选项：-timing_preserve_setup/hold_slack_threshold、-timing_preserve_nets。

7). 插 Filler Cells

一般工艺库支持有金属、无金属、单倍高度、多倍高度 Filler。

create_stdcell_fillers -lib_cells {从大到小} -bboxes -voltage_area → 添加；

connect_pg_net -automatic → 连接 Filler pin 电地属性；

remove_stdcell_fillers_with_violation → 移除有 Violation 的 Filler；移除后还需要再插，反复几次至全部 violation 解决；-check_only true 仅检查，不移除，生成报告 block_fillers_with_violation.rpt；-check_between_fixed_objects true 检查所有临近的物体。

命名规则：xofiller!filler_library_cell_name!number

8). 插 Metal Fill

插 metal fill 是为保证金属密度。过低的金属密度会导致应力失衡，造成良率下降。此步应在时序、DRC clean 的基础上进行。

命令：signoff_create_metal_fill（需要 ICV license）

6. IC Validator

1). ICV 准备设置

环境设置：setenv ICV_HOME_DIR /root_dir/icv

set path = (\$ICV_HOME_DIR/bin/LINUX.64 \$path)

分布式设置（例）：set_host_options -name dp4 -num_processes 2 {machineA machineB}；每台机器上最多运行两个进程。

Map 设置：工艺文件和检查的 runset 文件中层号不一致。设置 signoff.physical.layer_map_file。

Map 文件格式支持两种：

1). techfile_layer_list > runset_layer_list（list 以空格分隔）

2). object_type tf_layer[:tf_purpose] runset_layer[:runset_data_type]（object_type 默认为 all，可设 data、text）

支持注释符：“#”。

2). Signoff DRC 检查

基本流程：

1. 设置 signoff.check_drc.runset (最低设置要求);
2. save_block (检查数据来源为模块保存在硬盘中信息, 不检查内存中信息, 故需线保存);
3. signoff_check_drc。

相关选项设置:

signoff.check_drc.always_read_fill → 默认 false, 只有 fill data 时间比模块时间新时 ICV 才读取 fill 数据;

signoff.check_drc.auto_eco_threshold_value → 默认 20, 指定 “-auto_eco true” 时 incremental design rule checking 最大可修复百分比;

signoff.check_drc.excluded_cell_types → 默认 {}, 可设 lib_cell、macro、pad、filler, 为不检查的事项;

signoff.check_drc.ignore_blockages_in_cells → 默认 true, 只从 fram 中读取 pin 信息; false 时读取 pin & blockage 信息;

signoff.check_drc.ignore_child_cell_errors → 默认 false;

signoff.check_drc.max_errors_per_rule → 默认 1000;

signoff.check_drc.run_dir → 默认 signoff_check_drc_run;

signoff.physical.layer_map_file → 工艺文件到检查规则文件层号 map

signoff.physical.merge_stream_files → 变更读取 reference cell 数据来源由 design/fram view 到 gds。

“signoff_check_drc” 步骤说明:

1. 读取模块信息。默认 ICV 读取顶层和 std cell 的 design view, 从 fram view 中读取 IO、macro 的 pin 信息。

令 std cell 信息从 fram 中读取, 设 signoff.check_drc.ignore_blockages_in_cells 为 false;

指定读取 design view, 设 signoff.check_drc.read_design_views; 当 design view 不存在时, 读取 fram view;

指定 reference cell 数据来源自 gds, 设 signoff.physical.merge_stream_files;

2. 执行检查; 默认检查所有类型 cell, 可用 signoff.check_drc.excluded_cell_types 排除部分 (lib_cell、filler、macro、pad; 对整个设计检测; 检测所有规则; 检测所有层; 报 1000 条 violation;

选项 “-auto_eco” 控制只检测变更处。这种策略用在之前已执行过 signoff_check_drc, 而且变化幅度不大于 20%。选项 “-pre_eco_design” 指定比较的 version。这种策略默认只报变更处的问题, 设 signoff.check_drc.merge_incremental_error_data 为 true, 将也会报之前的问题 (从 signoff_check_drc.err 中读取之前信息, 可设 signoff.check_drc.merge_base_error_name 指定 err)(需要在执行 “signoff_check_drc-auto_eco” 前)。

对规则的控制: -select_rules、-unselect_rules;

对层的控制: -check_all_runset_layers true、-select_layers;

对报告条数的控制: signoff.check_drc.max_errors_per_rule、signoff.check_drc.ignore_child_cell_errors (true 时不检查 child cell 问题)

为自动修复的设置:

设 signoff.check_drc.ignore_child_cell_errors 为 true。(下层的报了工具也修不了, 影响时长);

设 signoff.check_drc.user_defined_options 为 { -holding_cell }, 令工具层次化优化, 在各自层次优化各自的问题; 在运行时, 命令最后加上该选项。

3. 生成相关报告:

生成 error data, signoff_check_drc.err; ICV 生成 block.LAYOUT_ERRORS (包含检测到的 Error)、block.RESULTS (运行简报)、signoff_check_drc.log、icv_config_out (数据来源)、icv_sdrc.conclude (Error 简报)、layer.map、signoff_check_drc.rc (runset 的环境参量)

3). 自动修复 DRC violation

指令: signoff **fix**_drc。(至少需要运行两次, 第一次修改, 第二次评估修改后的影响。使用选项 “-max_number_repair_loop” (1-10), 该选项比较大时, “-start_repair_loop” 也要随之加大点)

Signoff_fix_drc 第一遍使用 signoff_fix_drc.init_drc_error_db 中数据或 signoff_check_drc; 之后的修复循环读取前一次的数据。最后保存一个修改后的 design view, 名为“block_ADR_#”。

默认不检查以下规则: { “M*.DN.*:*”, “VIA*.DN.*:*”, “USER_GUIDE.*:*”, “CSR.*:*”, “DM.*:*”, “Rule*DEN.*:*” }

控制选项:

signoff_fix_drc.check_drc → 默认 local (global), 即检查修改后的周边影响;

signoff_fix_drc.config_file → 默认 auto, 使用检测的配置;

signoff_fix_drc.custom_guidance → 默认无。设为 **dpt** 专修 double-patterning odd-cycle violations;

signoff_fix_drc.fix_detail_route_drc → 默认 global (local), 修复范围;

signoff_fix_drc.init_drc_error_db → 指定检测 error db 文件; 默认重新执行“signoff_check_drc”

signoff_fix_drc.last_run_full_chip → 默认 false, 设置最后一遍修复后, 检查修改周边 (false) 还是全局 (true);

signoff_fix_drc.max_detail_route_iterations → 默认 5, DRC 修复绕线检查次数;

signoff_fix_drc.max_errors_per_rule → 默认 1000;

signoff_fix_drc.run_dir → 默认 signoff_fix_drc_run

signoff_fix_drc.target_clock_nets → 默认 false (修数据线); true 为修时钟线;

signoff_fix_drc.user_defined_options → 与检查的同样意义。

专注 Double-Patterning Odd-Cycle Violations 的修复:

1. 先检测、修复非 double patterning 类型 DRC; signoff_check/fix_drc -unselect_rules {list_of_dpt_rules}

2. 只检测、修复 double patterning 类型 DRC; set_app_options -name signoff_fix_drc.custom_guidance -value dpt; signoff_check/fix_drc -select_rules {list_of_dpt_rules}

3. 重新检测所有 DRC 类型, 确认修复效果。

4). 插 Metal Filler

signoff_create_metal_fill。详情见 user guide。

5). 自动修复孤立孔

signoff_fix_isolated_via。详情见 user guide。

7. 使用 Relative Placement 处理 Physical Datapath

基本类似 ICC，详情见 ICC/ICC2 user guide。

8. 顶层收敛

1). Abstract Views 流程

Abstract Views 保留模块的端口逻辑（可设置保留 critical path）。

ICC2 流程中可以对模块的任何阶段创建 abstract view。命令：create_abstract。选项 “-timing_level” 控制模型保留的时序类型（none、compact、full_interface）；-blocks；-all_blocks；-force_recreate，默认已有的不会重复创建覆盖。

“create_abstract” 之前设置 “set_app_options -name time.si_enable_analysis -value true；set_app_options -name abstract.include_aggressor_nets -value true” 则创建的 abstract 包含串扰信息，顶层调用时会考虑串扰。

不建议在 “create_abstract” 之后再对设计进行修改。

2). Frame View 流程

Frame View 保留模块的绕线信息。使用 “create_frame”。调用时需要 “link”，可以用 “set_ref_libs -add ./xsm_blk.ndm”。

当同时存在多个 View，且均可 “link”，默认顺序是 1. Abstract view → 2. Frame view → 3. Design view → 4. Outline view。使用 “set_view_switch_list” 更改顺序，之后应用 “rebind_block 或 link_block -rebind -force”。

可以对调用的模块 view 进行切换，并将下/上层设置应用到上/下层。例：change_abstract FSB -view design -promote_constraints。即将 FSB 的 abstract view 切换到 design view，并模块级设置应用到顶层。

3). 使用 Abstracts 顶层收敛

在调用模块 abstract view 前，首先需要做以下步骤：

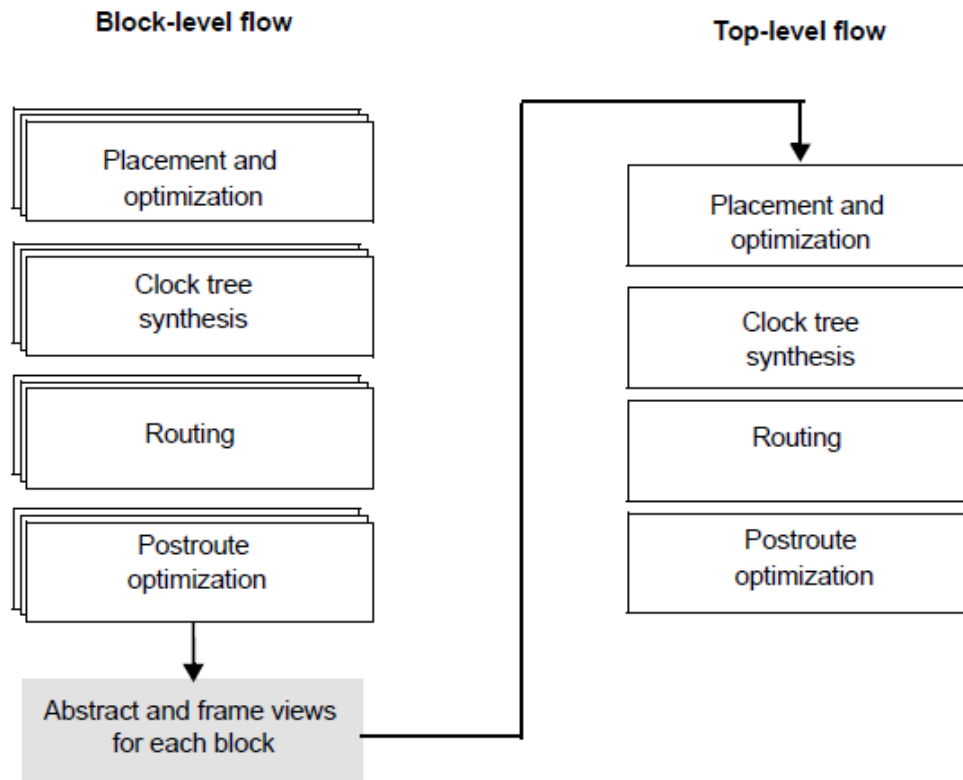
1. 设置顶层设置（由全片级约束切分出顶层约束，split_constraints）；
2. 设置顶层时钟树设置，由 “promote_constraints -auto_clock_connected -cts” 将对应模块时钟约束；
3. 应用顶层 UPF 约束（由全片级约束切分出顶层约束，split_constraints）；
4. 设置 placement 约束（选）；顶层经常会设置比较大的 bound，这通常会导致在边缘处比较拥塞，建议设 “place.floorplan.density_aware_hard_movebounds” 为 true。强制 cell 放

在内；顶层利用率较低时，建议设 “**place.coarse.max_density**” 使单元尽量聚拢；

5. 为降低时间，优化精度，设 “**set_app_options -name {time.virtual_route_top_level} -value true**”。

检查命令：check_hier_design。选项：-reference；-stage timing（默认）|pre_placement（检查时序及 place）。

基本流程示意图如下：



4). PT 创建 ETMs

参考脚本：

#读网表、link

```
read_verilog ./AMS_BLK.v
```

```
link
```

#读取时序约束

```
read_parasitics ./AMS_BLK.spf
```

#多多电压域，读取 UPF 设置和其他设置

```
load_upf ./AMD_BLK.upf
```

```
set extract_model_include_upf_data true
```

#读取 scenario 设置（mode 和 corner 设置）

```
source m1_constraints.tcl
```

```
source c3_constraints.tcl
```

Enable clock latencies for designs with synthesized clock trees

```
set extract_model_with_clock_latency_arcs true
```

```
set extract_model_clock_latency_arcs_include_all_registers false
```

#创建 ETM

```
extract_model -library_cell -format db -output AMS_BLK_m1_c3
```

5). 在 Library Manage 中创建 ETM Reference Libraries

参考脚本:

创建空的 reference libraries

```
create_workspace -flow etm_moded AMS_BLK (flow 种类还有很多)
```

#读取物理信息

从 frame view : `read_ndm -views frame AMS_BLK.ndm`

从 LEF: `read_lef`

从 GDSII: `read_gds`

从 OASIS: `read_oasis`

读取所有 scenario 对应的时序库

```
read_db -mode_label m1 AMS_BLK_m1_c1.db
```

```
read_db -mode_label m1 AMS_BLK_m1_c2.db
```

```
read_db -mode_label m1 AMS_BLK_m1_c3.db
```

```
read_db -mode_label m2 AMS_BLK_m2_c1.db
```

```
read_db -mode_label m2 AMS_BLK_m2_c2.db
```

```
read_db -mode_label m2 AMS_BLK_m2_c3.db
```

#检查

```
check_workspace
```

生成完整的 reference libraries

```
commit_workspace -output AMS_BLK_ETM.ndm
```

6). ETM 流程

ETM 使用类似标准单元库的读取, 需要 link 和把 reference 添加到 reference_list 中 (`lappend nt_ref_lib "AMS_BLK_ETM.ndm"; create_lib -technology tech.tf -ref_libs $nt_ref_lib TOP.ndm`)。

对于存在多个 mode 的 ETM 叫“moded ETM”, 需要指定使用的 mode, 命令“`current_mode *; set_cell_mode * $instance`”;

可以切换顶层使用的类别: `set_reference -block AMS_BLK_ETM.ndm:AMS_BLK.timing $instance` (转成 view); `set_reference -block AMS_BLK.ndm:AMS_BLK.abstract $instance` (转成 abstract)。

9. ECO

ECO 可以对设计进行功能、时序、噪声、串扰违例进行优化。

1). 未流片 ECO

芯片还没有 tape out，可以进行添加、删除、移动、变更等操作。主要步骤如下：

1. 使用 “eco_netlist” 或网表编辑指令编辑网表，进行 ECO 修改；
2. 使用 “place_eco_cells” 更新 ECO placement；
3. 添加 Filler 。使用 “ create_stdcell_fillers -post_eco ” 配合 “remove_stdcell_fillers_with_violation” 添加 Filler（不引入 Violation）；
4. 更新绕线信息 “route_eco” 或手动绕线。

route_eco -reroute modified_nets → 不建议，会导致有 DRC；

route_eco -reroute **modified_nets_first_then_others** → 推荐，ECO 绕线后再修绕线 DRC；

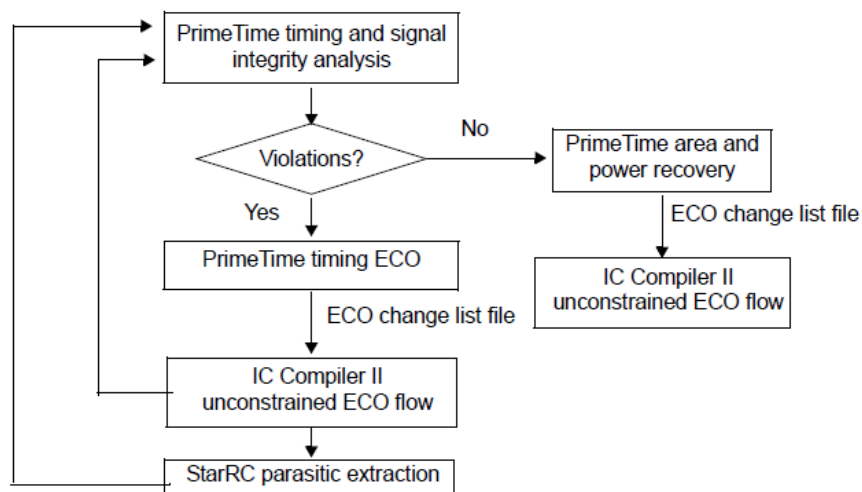
2). 已流片 ECO

已流片，不能对 placement 进行调整，只能对绕线金属和孔进行调整。前提是设计中预布了 spare cell（通常分为手动添加；placement 后使用 “add_spare_cells” 添加；chipfinish 阶段使用 “create_sdtcell_fillers” 添加）。主要步骤如下：

1. 开启流片后 ECO 选项：设 design.eco_freeze_silicon_mode 为 true；
2. 使用 “eco_netlist” 或网表编辑指令编辑网表，进行 ECO 修改；
3. 分析 ECO 后 spare cell 的 mapping 情况，使用 “check_freeze_silicon”；
4. 将 ECO 变化 mapping 到 spare cell 中。自动命令 “place_freeze_silicon”；手动指定 mapping，使用 “map_freeze_silicon”，支持根据文件 mapping，格式为 “ECO_cell_* spare_cell_*”；
5. 更新绕线信息 “route_eco” 或手动绕线。

3). Signoff ECO

这种 ECO 流程将 PT 的分析结果迭代会 ICC2，以此为依据进行 ECO。基本流程如下图：



4). Incremental Signoff ECO

鉴于 Signoff ECO 迭代花费较长时间。所以增加 Incremental Signoff ECO 模式。在该模式 ECO 中，ICC2 不会直接用 ECO netlist 进行更改，而是模拟 ECO 变化，输出一个文件以便下一次 ECO 迭代。基本流程如下：

1. 开启 design library, “open_block”;
2. 读取 PT 的 ECO tcl, 开始记录 ECO 对设计的影响 “record_signoff_eco_changes -start -input fix_eco.tcl”;
3. place_eco_cells
4. route_eco;
5. 结束记录 ECO 对设计的影响, “record_signoff_eco_changes -stop”。

5). 手动例化 Spare Cells

手动例化 spare cell 可以在网表中例化或使用 “create_cell、connect_pins”。一般为避免干扰，spare cell 的输入连接到电/地。对于非 physical-only cell，且输入除时钟外都浮空或连接到电/地，输出浮空这样的单元，工具也会识别成 spare cell 的。

如果在 placement 后插 spare cell，需要摆放和 legalize。参考流程如下：

1. 均匀分布 “spread_spare_cells -cells * -voltage_areas * -boundary *” (创建时最好命名规则区别于标准单元);
2. legalize: place_eco_cells -legalize_only -cells [get_cells -hierarchical spares*]

6). 自动例化 Spare Cells

使用命令 “add_spare_cells -cell_name * -lib_cell * -num_instances * (或 -num_cells {reference_cells 数目...})”。

控制 spare cell 分散的层级: -hier_cell; 分散的范围: -boundary; 电压域: -voltage_areas。

需要使用 “place_eco_cells -legalize_only -cells” legalize。

7). Programmable 添加 Spare Cells

需要工艺单元库支持，存在 gate array filler cells。这种单元兼具 filler 和 spare cell 的功能。基本流程如下：

1. 创建 std cell 和 gate array filler cells 的对应关系。具体为设置两种 cell 中 “psc_type_id” 属性为相同值 (正整数)。(在 ECO 过程中根据此值判断使用什么类型的 spare cell)

例: set_attribute [get_lib_cells fill_lib/fill1x] psc_type_id 1
set_attribute [get_lib_cells stdcell_lib/BUF1] psc_type_id 1
set_attribute [get_lib_cells stdcell_lib/INV1] psc_type_id 1

2. 使用 “create_stdcell_fillers -lib_cells *” 添加 spare cell。

8). 使用“eco_netlist”对网表进行修改

命令“eco_netlist”是将 PT ECO 后“模拟网表”与原设计网表比较，生成变更 tcl 脚本。一种方式是直接以当前模块进行比较(默认)，可以指定“-block、-golden_lib、-working_block、-working_lib”；另一种模式是指定对比的网表，“-by_verilog_file”。输出文件用“write_changes”指定。

默认忽略纯物理单元（-compare_physical_only_cells，比较）；对时序 ECO 必须设“-extract_timing_eco_changes”，该选项令不仅比对功能变化，还考虑时序影响因素；忽略地电网路的差异。

9). 网表修改相关指令

Create/remove_ports/nets/cells、connect_cell、disconnect_cell、size_cell、add_buffer、add_buffer_on_route、change_link。

add_buffer_on_route 是 add_buffer 的高级版，可以控制 buffer 插的位置，绕线层等信息。

对没有 unique 的 MIM（Multiple-instanced-module）修改，使用以下格式：

```
edit_module [get_module add_block] { \
    change_link [get_cells U25] AND2 \
    change_link [get_cells U61] XOR2
}
```

对 size 后不满意，可使用“revert_cell_sizing”恢复 size 之前状态。使用“-include_adjacent_sized_cells”或“-adjacent_cell_distance”控制复原后 cell 的位置摆放策略。

10). 摆放 ECO Cells

命令“place_eco_cells”。

选项“-unplaced_cells”作用于所有为 place 单元；“-cells”指定单元；“-eco_changed_cells”只对由于 ECO 需要摆放的单元有效（判断单元“eco_change_status”属性，该属性为导致 ECO 变化的对应指令）。

摆放控制选项：-channel_aware，可以使用 macro 之间区域；-fixed_connection_net_weight *，设大于 1 代表 ECO 单元靠近 fixed object 摆放；对某条线优先考虑，先设置线权重“set_eco_placement_net_weight”，再“place_eco_cells -honor_user_net_weight”；设置 fanout 阈值，“-max_fanout”，对于超出的单元不再摆放；忽略某些特定单元 pin，“-ignore_pin_connection”；

创建虚拟连接，用于 place，示例如下：

```
create_virtual_connection -name VC_0 -pins {U1/Y ECO17/A}
create_virtual_connection -name VC_1 -pins {U2/Y ECO17/B}
create_virtual_connection -name VC_2 -pins {ECO17/Y D17_OUT} -weight 3
place_eco_cells -cells ECO17 -use_virtual_connection
```

legalize 控制选项：place_eco_cells，“-legalize_only”；legalize 模式：“-legalize_mode free_site_only（默认，不移动已存在单元，可能导致 ECO 单元位移较大） |

allow_move_other_cells (ECO 移到最近合法位置，允许移动其他单元) |
minimum_physical_impact (结合上两种，先不移动已有单元，后期可以)。

控制位移：“-displacement_threshol”，“-max_displacement_threshol”，在 minimum_physical_impact 模式中当 cell 超过 max 的阈值时，将不会继续 legalize，将这些单元记录在“epl_max_displacement_cells”collection 中。

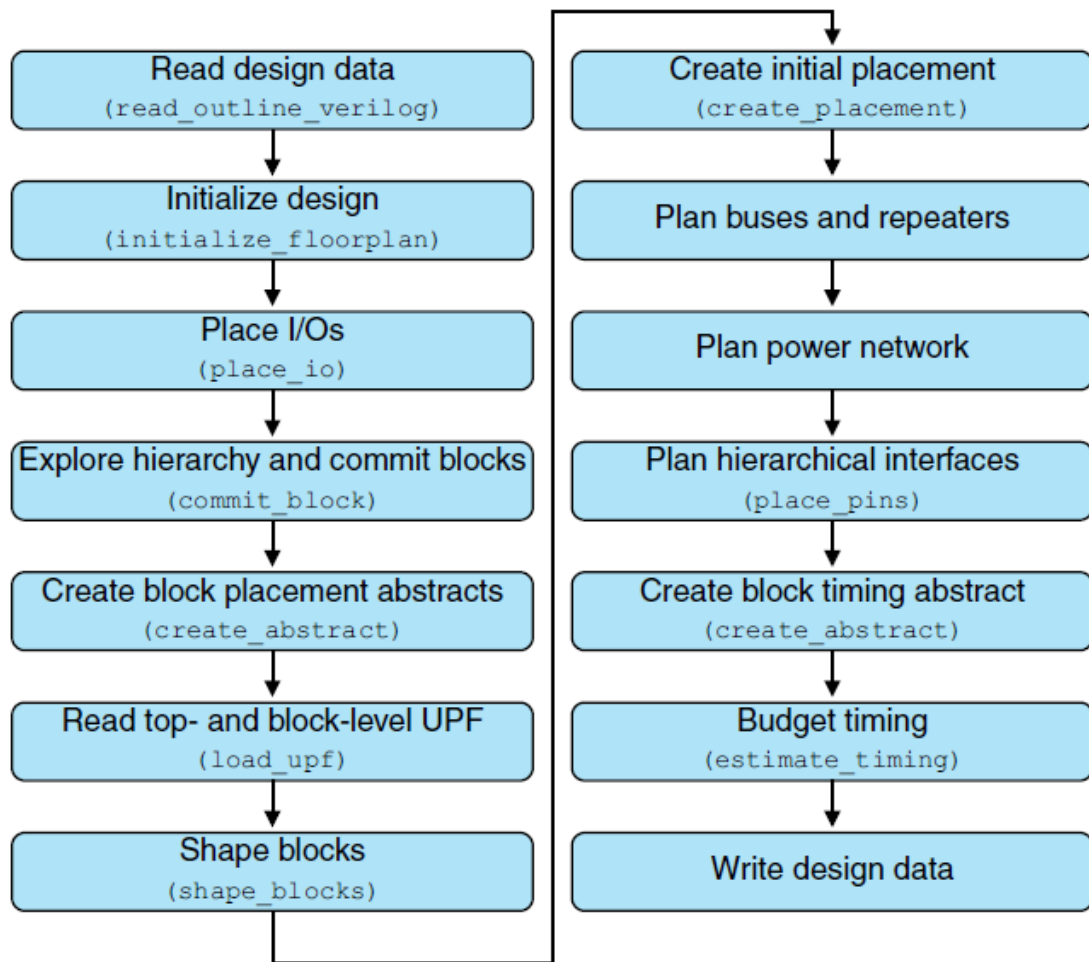
当 ECO cell 与 filler 重合时，移除 Filler，“-remove_filler_references”。

11). Adding Buffers to Drive Specific Loads of a Nets

在不同层次之间插 buffer，使用“split_fanout”。需要指定“-load、-lib_cell”。在“load-pin”之前插 buffer，位置尽量靠近 load-pin；“-drive”指定驱动的线，如果驱动跨层次，使用“-hierarchy”，位置在 drive-pin 与 load-pin 之间；“-on_route”指定按照绕线的拓扑结构插 buffer，在 load 第一个公共路径交汇点插。

五、Design Planing

层次化设计流程示意图：



针对大规模、复杂设计，可以对部分逻辑打包成小模块，打包策略按照大小等量，功能相近逻辑放在一起，还应考虑顶层是否保留部分逻辑等。（ICC 流程，在 Hierarchy View 中将某些 module 打包成 plan group，将创建的 plan group 移至 core 内，约束形状及 pin 信息，确认后提交成 soft macro。）

Hierarchical Floorplan 主要步骤：

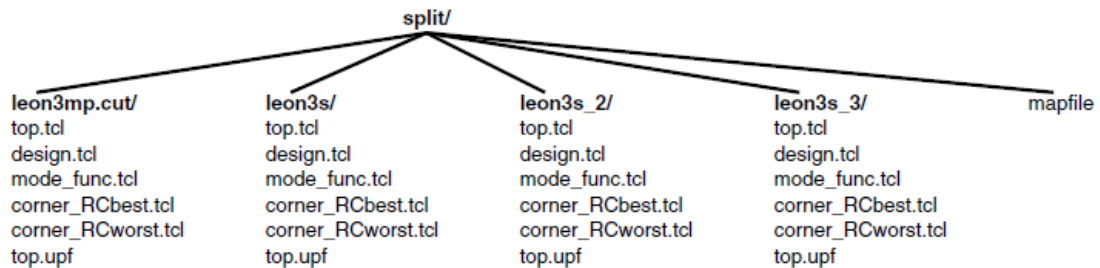
1. 创建 Placement Abstracts
2. Applying Padding to the Blocks
3. Placing and Shaping Blocks
4. Creating the Initial Macro Placement
5. Creating the Power Network
6. Pushing Down Physical Objects
7. Assigning Pins
8. Performing timing budgeting

1. 提取约束

1). 基本流程

1. 读取完整网表, `read_verilog -top * *.v`。(不支持 `read_verilog_outline`);
2. 载入全局 UPF, `load_upf *.upf → commit_upf` (有解析、修改设计与实际冲突的作用);
3. 设置时序环境:
对于 MCMC 设计, 使用 `create_corner`、`create_mode`、`read_sdc`、`set_corner_status`、`set_parasitic_parameters`、`set_process_number`、`set_temperature`、`set_voltage` (UPF 描述电地网络时, 使用该命令指定 net group)
4. 指定写出的约束, `set_budget_options -add_blocks {$instance}`;
5. 提取约束, `split_constraints`, 默认从当前设计的 SDC 和 UPF 提取顶层和各模块约束。如层次较深, 使用 “`-design_subblocks`”。

下图为 `split_constraints` 的示例输出:

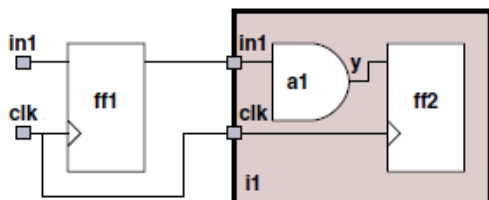


Mapping 文件:

```
leon3s    SDC  leon3s/top.tcl
leon3s    UPF  leon3s/top.upf
leon3s_2  SDC  leon3s_2/top.tcl
leon3s_2  UPF  leon3s_2/top.upf
leon3s_3  SDC  leon3s_3/top.tcl
leon3s_3  UPF  leon3s_3/top.upf
```

2). 示例

示例一电路图如下:



顶层约束:

```
create_clock -period 1 [get_ports clk]
set_input_delay 0.5 [get_ports in1]
```

```
set_multicycle_path 2 -from ff1/clk -through i1/a1/y
```

提取出的顶层约束：

```
create_clock -period 1 [get_ports clk]
```

```
set_input_delay 0.5 [get_ports in1]
```

```
set_multicycle_path 2 -from ff1/ck -through i1/a1/y
```

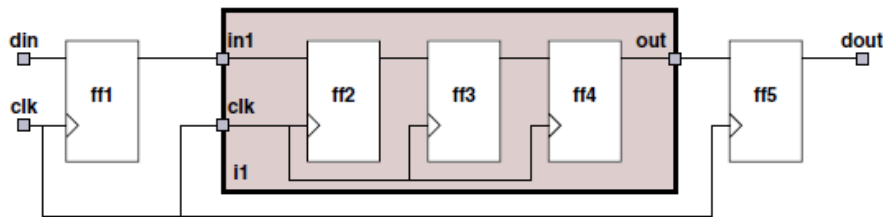
模块 i1 约束：（没有端口 delay 信息）

```
create_clock -add name "clk" -period 1 [get_ports clk]
```

```
create_clock -name virtual_clk -period 1
```

```
set_multicycle_path 2 -from virtual_clk -through [get_ports in1] -through a1/y
```

示例二电路图如下：



顶层约束：

```
create_clock -period 1 [get_ports clk]
```

```
set_input_delay -clock [get_clocks clk] 0.5 [get_ports din]
```

```
set_output_delay -clock [get_clocks clk] 0.5 [get_ports dout]
```

```
set_false_path -from [get_pins i1/ff2/CK] -to [get_pins i1/ff3/D]
```

提取出顶层约束：

```
create_clock -name clk -period 1 [get_ports {clk}]
```

```
set_input_delay -clock [get_clocks {clk}] 0.5 [get_ports {din}]
```

```
set_output_delay -clock [get_clocks {clk}] 0.5 [get_ports {dout}]
```

模块 i1 约束：

```
create_clock -name "clk" -period 1 [get_ports {clk}]
```

```
set_false_path -from [get_pins {ff2/CK}] -to [get_pins {ff3/D}]
```

3). 平行处理任务

run_block_script 支持对多个模块运行某一任务脚本，示例为同时生成各模块的 timing abstracts。

1. 写运行脚本。示例脚本如下：

```
# Perform design setup
```

```
source ../scripts/setup.tcl
```

```
# Open the block
```

```
open_block $block_libfilename:$block_refname.design
```

```
# Merge previous changes to the abstract into the top-level
```

```
merge_abstract （将模块的变更适用到顶层，这样顶层才能抽出精准的 abstract）
```

```
# Source block-level constraints
```

```
source ./split/${block_refname}/top.tcl
```

```
# Insert feedthroughs
```

add_feedthrough_buffers（默认在输出端附近插，可以用“-buffer_side both|input|output”设置）

```
# Create the block-level timing abstract
create_abstract -estimate_timing（层次复杂时，需指定层次，默认为最底层）
# Save the block
save_lib
close_lib
```

2. 使用“run_block_script”平行在各模块上应用上述脚本。例：

```
set_host_options -name lsf -submit_command "bsub -q amd64"
run_block_script -script create_block_timing_abstract.tcl -cells {*} -host_options lsf -
work_dir create_block_timing_abstract
```

4). 监控分布式提交进程

1. 开启分布式进程管理：run_monitor_gui
2. 执行进程。例：create_abstract -**placement** -host_options block_script -all_blocks（默认 create_abstract 抽取的 abstract view 有时序信息，-placement）

```
Submitting job for block leon3s ...
Submitting job for block leon3s_2 ...
Submitting job for block leon3s_3 ...
All tasks created.
...
```

3. 在 GUI 视窗中查看
以下指令支持分布式提交进程：

```
create_abstract -all_blocks | -blocks、create_placement -floorplan、estimate_timing、
merge_abstract、route_global -floorplan -virtual_flat、shape_blocks
```

2. 创建 Floorplan

Floorplan 基本分为 Channeled Floorplans（有顶层逻辑）、Abutted Floorplans（无顶层逻辑）、Narrow-Channel Floorplans（基本无顶层逻辑，顶层只预留时钟通道或为部分单元预留空间）。

读取网表文件步骤：

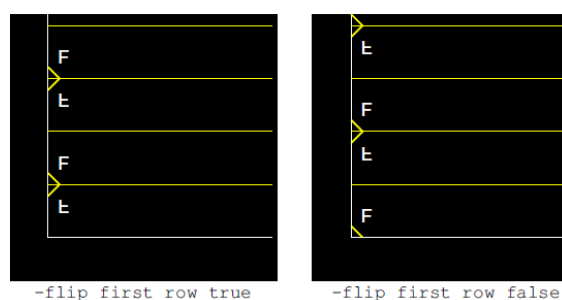
1. create_lib/open_lib；
例：set libs [glob -directory \${lib_dir} *.ndm] → 抓取指定目录下所有 ref.ndm 库
create_lib -technology technology_file.tf -ref_libs \$libs lib/design.ndm
2. read_verilog/read_verilog_outline。
例：read_verilog_outline -top * *.v

read_verilog_outline 适用于大设计，该指令对下层模块（各级层次）创建 outline view（保留层次信息，没有 cell 和 net）。该命令并不会直接割裂设计。需要指定某些模块单独综合（分析、黑盒）时，需要 commit_block；把它复原成完整网表，使用“expand_outline”。

创建 Floorplan：

命令: initialize_floorplan。选项: -core_offset → core-die 距离; -core_utilization ratio; -keep_boundary (根据 die 调整 core); -core_shape R (矩形) | L | T | U; -orientation N (默认) | W | S | E]; -core_side_length {core 对应各边长度; 主要使用在多边形 Floorplan}; -core_side_ratio {core 对应各边比值}; -coincident_boundary true | false (是否 die 跟随 core 变化, 需要设置 offset); -core_offset {distance | vertical_distance horizontal_distance | side_1 side_2 ... side_8} (仅一个值, 对应所有边; 两个值, 分别对应纵向边和横向边; 多个值则对应各边序号); -row_core_ratio 0~1 (该选项设置 cell row 之间的间距 (为绕线预留空间), 默认 1, 即 cell row 紧贴, 此值应该大于设计的利用率); -flip_first_row true (默认) | false; -keep_macro_placement (默认会边更 macro 位置); -keep_std_cell_placement; -keep_block_placement; -keep_pg_route (默认删掉所有); -keep_signal_route; -keep_io_placement; -pin_snap off (默认) | wire_track (将 terminal 移至 track 上, 不移动电地、有 IO 约束的 terminal); -honor_pad_limit (按照 IO pad 规划面积, 忽略默认 0.7 利用率); -site_def * (默认使用 tf 中最小的 site); -use_site_row (默认工具创建 siteArray, 该选项指定创建 siteRow)

对于复杂多边形 Floorplan, 使用 “initialize_floorplan -keep_boundary” 配合自定义边界创建。基本操作: set_attribute [get_designs \$design] boundary { { * * } { * * } ... } → initialize_floorplan -keep_boundary。



ICC2 支持对 FinFET 器件的摆放设置。默认设置在工艺文件中。“report_grids -type finfet” 报 FinFET 相关设置。例:

FinFET Grid Description

Top Design leon3mp.ndm:leon3mp.outline

FinFET Grid defined true

FinFET Grid X Pitch 0.152

FinFET Grid Y Pitch 1.672

FinFET Grid X Offset 0

FinFET Grid Y Offset 0

1

“check_finfet_grid” 检查 macro、pad 等是否放在 FinFET 隔点上。以下命令会考虑 FinFET 信息:

create_blackbox、reate_grid、reate_io_guide、reate_io_ring、create_macro_array、create_placement、create_power_switch_array、create_power_switch_ring、create_site_array、create_site_row、initialize_floorplan、place_io、report_grids、set_block_grid_references、set_grid、set_signal_io_constraints、hape_blocks

“report_design -floorplan 或 report_design -all” 会报 core/die 面积、ite row、Blockage、Power domain、Group bound、Move bound、Route guide、Multibit registers、Reserved placement

groups、Layers。

3. 黑盒

基本流程为：识别（get_attribute）→创建黑盒 reference（create_blackbox）→创建黑盒时序 model（create_black_*、set_blackbox_*）→确认提交黑盒（commit_blackbox_timing）。

1). 识别

可以通过属性 “ has_timing_model 、 **hierarchy_hard_macro_count** 、 hierarchy_has_shadow_netlist、hierarchy_has_shadow_netlist_only、hierarchy_pad_cell_count、hierarchy_physical_only_cell_count、**hierarchy_std_cell_count**、hierarchy_switch_cell_count、**is_unbound**、target_boundary_area” 识别黑盒。

例如： get_cells -hierarchical * -filter {hierarchy_std_cell_count==0 && hierarchy_hard_macro_count==0}

get_cells -hierarchical * -filter "is_unbound==true"

2). 创建黑盒 reference

命令 “create_blackbox” 将某些 module 变成对应物理属性的一个黑盒。例：
create_blackbox -target_boundary_area 面积 \$instance; 搜 blackbox: get_cells -hierarchical *
-filter {**design_type==black_box**}

3). 创建黑盒时序 model

默认与黑盒端口相关路径是无约束的，可以设置时序 model，以便 timing budgeting 和优化。基本流程：

1. 定义时钟、端口驱动、负载： set_blackbox_clock_port、set_blackbox_port_drive、set_blackbox_port_load;
2. 设置时钟网络延时，pin-pin 延时，setup/hold 约束：create_blackbox_clock_network_delay、create_blackbox_constraint 、 create_blackbox_delay 、 create_blackbox_drive_type 、 create_blackbox_load_type;

为允许 timing budget 调整黑盒设置： set_app_options -name **plan.budget.bbt_fixed_delay** -value false

3. 确认提交黑盒： commit_blackbox_timing。

4). 示例

约束：

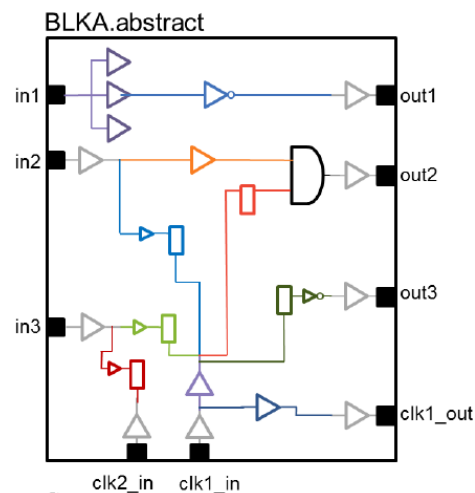
set_blackbox_clock_port [get_ports {clk1_in clk1_out clk2_in}]

```

create_blackbox_load_type -lib_cell buf1 lt1
set_blackbox_port_load -type lt1 -factor 3 [get_ports in1]
create_blackbox_clock_network_delay -value 3.5 clk_1
create_blackbox_delay -rise_from in1 -fall_to out1 -value 5.0
create_blackbox_delay -fall_from in1 -rise_to out1 -value 5.2
create_blackbox_delay -from in2 -to out2 -clock main_clock -value 0.4
create_blackbox_constraint -setup -edge rise -from clk2_in -to in3 -value 1.2
create_blackbox_constraint -hold -edge rise -from clk2_in -to in3 -value 0.8
create_blackbox_delay -from clk1_in -to clk1_out -value 1.1
create_blackbox_constraint -edge rise -from clk1_in -to in3 -clock main_clock -value 0.5
create_blackbox_constraint -edge rise -from clk1_in -to in2 -value 1.3
create_blackbox_delay -rise_from clk1_in -to out2 -value 2.2
create_blackbox_delay -fall_from clk1_in -to out3 -value 1.7
commit_blackbox_timing

```

结果：



4.规划 IO 及 Flip-Chip Bumps

基本流程：创建 IO ring/guide（create_io_ring、create_io_guide）→创建 Bump 阵列（create_bump_array）→创建电地 IO 约束（set_power_io_constraints）→创建信号 IO 约束（set_signal_io_constraints）→摆 IO（place_io）→写出 IO 设置（write_io_constraints）→绕线 RDL（route_rdl_flip_chip）。

1). 创建 IO ring/guide

创建 IO ring 可以用“create_io_ring”直接创建或使用 4 条“create_io_guide”约束 IO ring 结构。

命令：create_io_ring。-offset *指定 ring 到边界距离；-inside/outside \$ring_name 指定 ring

在边界的里/外面；-guides {\$guide_names}；-bbox；-pad_cell_list \$pad_list 指定与 ring 关联的 IO；-corner_height 指定 ring 的宽度，默认当 ring 关联 IO 时，使用其中最大的高度，没有 ring 关联 IO 时，使用 IO 库中小的高度。

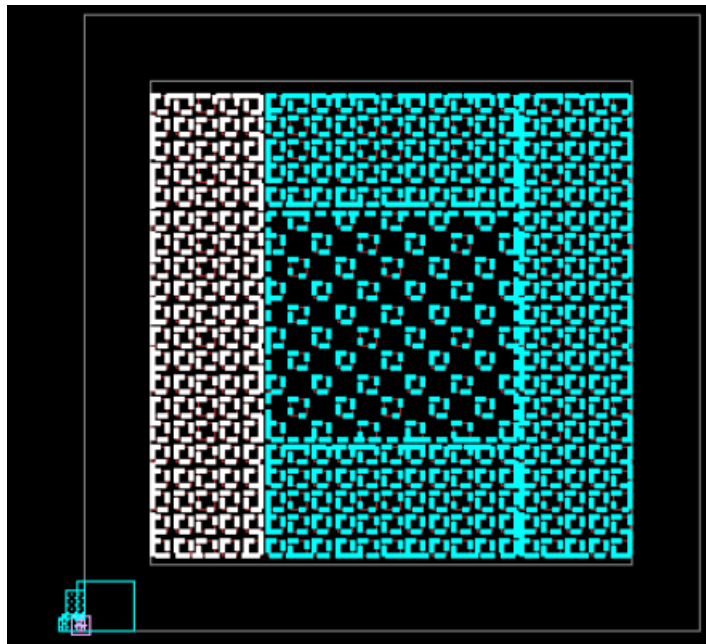
命令：create_io_guide。-name；-side left|right|bottom|top；-line { {x y} length}指定 ring 起点及延伸距离；-offset；-pad_cell_list \$pad_list。

跟 ring 相关指令如下：add_to_io_guide、get_io_guides、remove_from_io_guide、remove_from_io_ring。

2). 创建 Bump 阵列

摆放 Bump 应该在 IO driver cell 摆放之前。在 die-driven 流程，创建一个区域给 bump，让工具自己决定 bump 的位置。命令：create_bump_array。例：

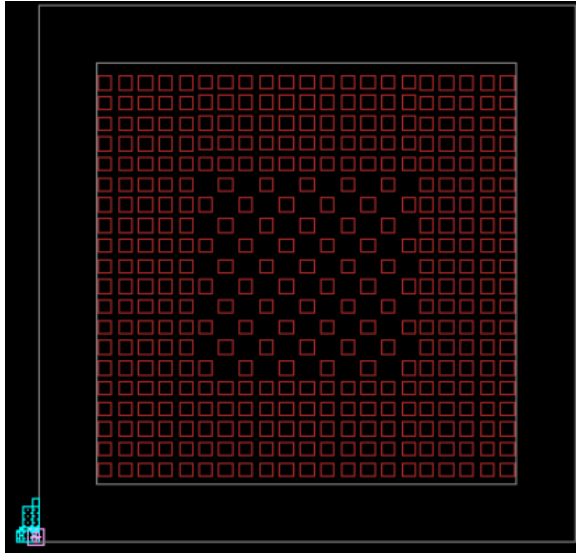
```
create_bump_array -lib_cell BUMP -delta {140 140} -bbox {{410 450} {1175 3278}} -name left
create_bump_array -lib_cell BUMP -delta {140 140} -bbox {{2620 450} {3325 3278}} -name right
create_bump_array -lib_cell BUMP -delta {140 140} -bbox {{1100 450} {2650 1125}} -name bottom
create_bump_array -lib_cell BUMP -delta {140 140} -bbox {{1100 2555} {2650 3250}} -name top
create_bump_array -bbox {{1100 1150} {2650 2550}} -pattern staggered_1 -lib_cell {BUMP} -delta {140 140} -name pg_bump (用在 core 处)
```



选项“-pattern inline | staggered_1 | staggered_2”设置 bump 交叠选项，staggered_1 代表 bump 放在行/列之和为基数的点上；staggered_2 为偶数隔点上；选项“-repeat {columns rows}”限制行/列的 bump 数。

3). 预设位置摆放 bump

在封装驱动的设计，bump 位置有封装决定，bump 位置常由 DEF/AIF 文件读取。例：
read_aif bumps.aif



4). IO 约束

电地 IO 约束： `set_power_io_constraints -io_guide_object [get_io_guides *] -ratio {{3 VDD_NS}} {7 VSS_NS} -reference_cell {VDD_NS VSS_NS}`

该指令控制在指定 IO guide 中 IO 的摆放规则，上例说明在相邻的 VDD_NS 之间信号 IO 最多可以插 3 个，相邻的 VSS_NS 之间信号 IO 最多可以插 7 个。

选项“-share {{2 VDD}} {3 VSS}}”代表 2 个 VDD 驱动一个 Bump，3 个 VSS 驱动一个 Bump；选项“-spacing {spacing libcell}”指定连个 IO 之间的最大距离；选项“-offset {spacing libcell}”指定 IO 到 IO guide 的最大距离。

信号 IO 约束： `set_signal_io_constraints -io_guide_object * -constraint { {order_only} ...} 或 {pad_1 10 pad_2 20 pad_3}`，代表 pad_1 和 pad_2 之间距离 10，pad_2 和 pad_3 之间距离 20；`-constraint {{pitch} pad_1 pad_2 ...}`则指定 lo 之间距离为恒值。

place 基本流程：

1. 摆放 IO (place_io)；对于有 RDL 绕线约束的，需要设置-rule *；为了防止对已摆好的 IO 重新操作，摆完 lo fix 住。
2. 写出当前 IO placement 约束 (write_io_constraints)；

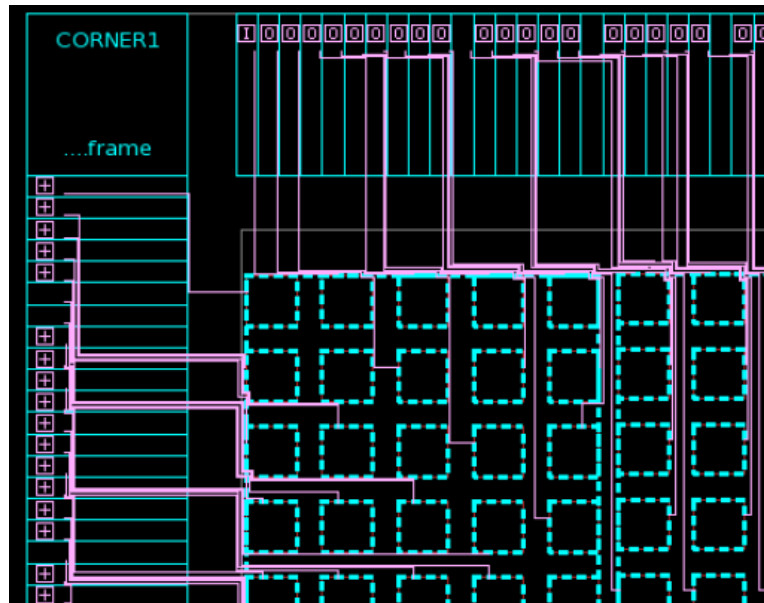
place_io 选项“-matching_types”可选 IO 和 bump 的对应列表，对于有明确的一对一连线关系的连接，这种关联将保留并考虑。

5). DRL 绕线

基本流程：

1. 定义 RDL 绕线 rule (create_routing_rule rdrrule -widths {MRDL 2} -spacings {MRDL 1})；
2. 使用 RDL rule (set_rdl_nets [get_nets -of_objects [get_pins -of_objects [get_cells -filter "design_type==flip_chip_pad"]]] → set_routing_rule [get_nets \$rdl_nets] -rule rdrrule)
3. 默认 RDL 绕线为 45° 绕线，若想设置为 90° 绕线，通过“set_app_options -name flip_chip.route.layer_routing_angles -value {{MRDL (RDL 层名) 90_degree}}”
4. 对 RDL 绕线

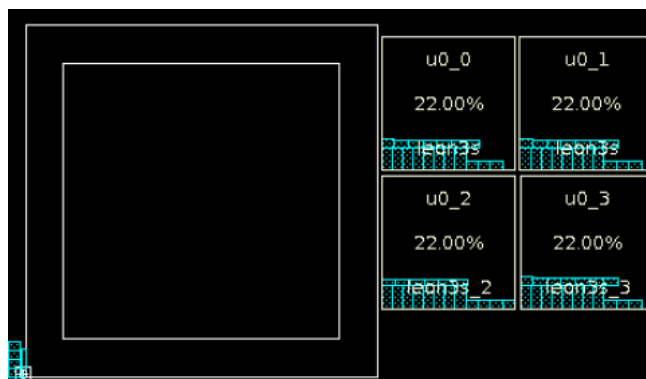
按照上流程的示例操作:



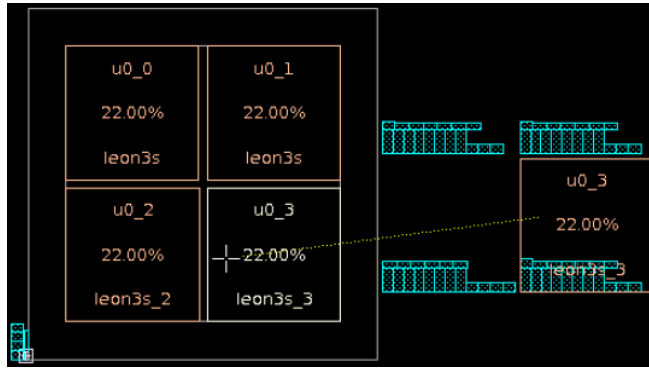
工具支持对 design 的 partition，将某逻辑层次变成独立的物理模块。基本流程为分析逻辑层次，创建 module 边界 (explore_logic_hierarchy)，提交成模块 (commit_block)，创建 placement 用 abstract (create_abstract)。

使用 View > Hierarchy Browser 查看逻辑层次，注意打包逻辑的大小。

1. 遵照设置创建 module 边界 (例: `explore_logic_hierarchy -create_module_boundary -cell {u0_0 u0_1 u0_2 u0_3}`, 默认按照顶层利用率创建) (**GUI 选择、创建更直观**)



2. (选) 将 boundary 移至 core 内，查看调整。



3. 重置或删除 (explore_logic_hierarchy -organize、explore_logic_hierarchy -remove)

3). 提交成模块

1. 为模块创建新库 (可选、推荐)
create_lib ../lib/leon3s.ndm -technology saed90nm.tf -ref_libs \$libs
save_lib
set_ref_libs -add leon3s.ndm
2. 提交成模块
commit_block -library leon3s.ndm leon3s

4). 创建 placement 用 abstract

1. 设置约束 mapping 文件
set_constraint_mapping_file split/mapfile
mapping 文件例：

leon3s	SDC	leon3s/top.tcl
leon3s	UPF	leon3s/top.upf
leon3s_2	SDC	leon3s_2/top.tcl
leon3s_2	UPF	leon3s_2/top.upf
leon3s_3	SDC	leon3s_3/top.tcl
leon3s_3	UPF	leon3s_3/top.upf
leon3mp	SDC	leon3mp/top.tcl
leon3mp	UPF	leon3mp/top.upf2
2. 检查顶层顶层 cell view 类型，应该是 design view；如果是 outline view (使用 read_verilog_outline 读取)，使用“expand_outline、save_lib -all”将 outline 转化成 design view；
3. 设置分布式进程 (set_host_options -name block_script host_settings)；
4. 使用分布式进程操作 (create_abstract -placement -host_options block_script -all_blocks)。

5). 调整模块原点

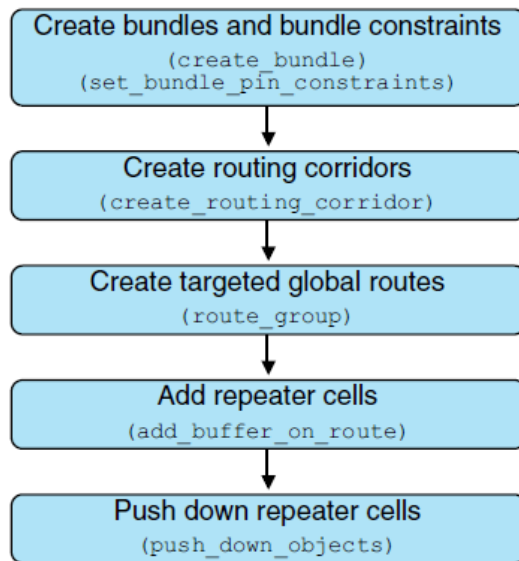
默认原点是{0 0}，move_block_origin -to {* *}可以调整原点。

6. 电压设计

略，见 user guide。

7. 全局规划

在 Floorplan 阶段，完成 block shaping 和 macro placement 后，进行全局 bus 规划。基本流程如下图：



1). 创建 Bundle（束）约束

工具支持以 Bundle 形式约束一组线，Bundle 可以嵌套。基本流程：

1. 创建 Boundle: 指定线 → `create_bundle -name $name [get_nets **]`
匹配线 → `create_bundles_from_patterns -net_name_prefix **`

控制选项：线顺序 “-net_order ascending | descending”；容量 “-min/maximum_nets min/max”；-no_braces ({}); -no_brackets ([]); -no_angle_bracket (<>); -no_underlines; -no_colons (:); -no_parentheses (()).

2. （选）检查，“report_bundles [get_bundles \$name]”
3. 增减，移除：add_to_bundle、remove_from_bundle

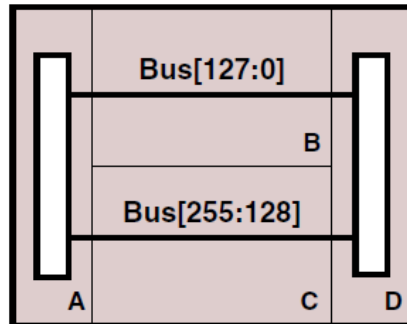
GUI 操作：View > Net Assistant

创建 Boundle 约束：

1. set_bundle_pin_constraints。选项：-bundles *；-cells *；-self（仅用在顶层）；-keep_pins_together true | false；-sides */-range {start end}控制 pin 位置；-bundle_order off | ordered | increasing | decreasing |equal-distance 控制 pin 顺序；-allow_feedthroughs true | false；-allowed_layers *；-pin_spacing pin 之间 track 数；-pin_spacing_distance pin 之间距离
2. （选）检查：report_bundle_pin_constraints

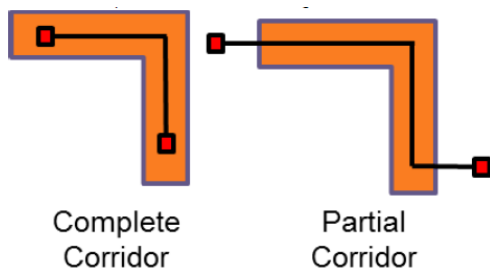
示例：


```
create_bundle -name bundle1 [get_nets {Bus_0 Bus_1 ... Bus_127}]
set_bundle_pin_constraints ... -bundle bundle1 -allowed_layers {M3 M5} -cell Block B
create_bundle -name bundle2 [get_nets {Bus_128 Bus_129 ... Bus_255}]
set_bundle_pin_constraints ... -bundle bundle2 -allowed_layers {M3 M5} -cell Block C
```



2). 使用绕线优先通道（Corridor）

Corridor 分完整型和部分型。具体示例如下：



基本流程：

1. 创建：**create_routing_corridor** -name corridor_a -boundary {{1925 2170} {1980 3260}} -min_layer_name M1 -max_layer_name M4 -object [get_nets irq0[0]]（GUI：Create > Routing Corridor）
2. 检查：report_routing_corridors
3. 拓展通道形状：**create_routing_corridor_shape**；添加新线到通道中：add_to_routing_corridor。
4. 设置 Corridor 类型：设 route.global.connect_pins_outside_routing_corridor, true 为 partial 型；默认 false，为 complete 型。
5. 对 Corridor 内线绕线：**route_group -global_planning true -nets [get_nets -of_object [get_routing_corridors corridor_a]]**

3). 添加 Repeater cells

通过“**add_buffer_on_route**”添加 Repeater cells。此操作在执行完对特定线“route_group”、创建 block pin 和 **pushing objects into blocks** 之前。

```
add_buffer_on_route -allow_insertion_over_cell（在顶层插入，后面可以应用到模块中）
[get_cells 模块] -repeater_distance * -lib_cell stdcell/buffer1
-location; -user_specified_buffers; -repeater_distance_length_ratio 0~1 约束 buffers 之间
```

的距离；

-scaled_by_layer/-scaled_by_width 需要-repeater_distance，格式各层一一对应，没有定义的层默认 scaling number 为 1；

命令默认考虑 placement blockage、macro (soft|hard)，可以在 macro 上添加；之后需 legalize_placement；route_eco。

4). 将 Repeater cells 应用到模块中 (push down)

push_down_objects 可将上步中添加的 Repeater cells 应用到下层模块中。当“push down”后，属性 shadow_status 为 pushed_down，该属性还可为“copied_down、copied_up、normal、pulled_up、pushed_down、virtual_flat”。

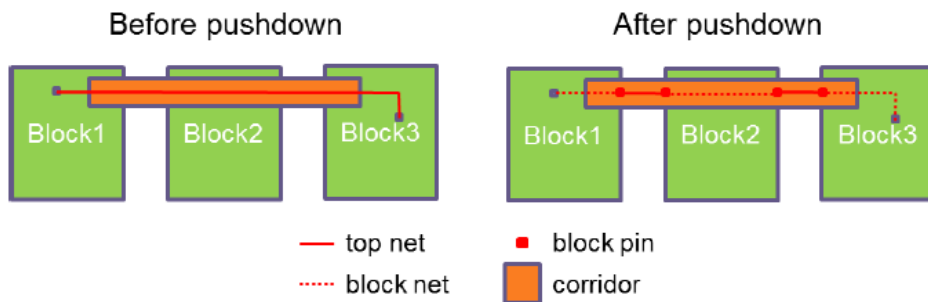
将“线”应用到模块中，并创建反馈：

1. set_individual_pin_constraints -nets [get_nets *] -allow_feedthroughs true (线对应模块而言既有输入，又有输出，需要设置允许反馈绕线)

2. push_down_objects [get_nets -of_objects [get_routing_corridors *]]

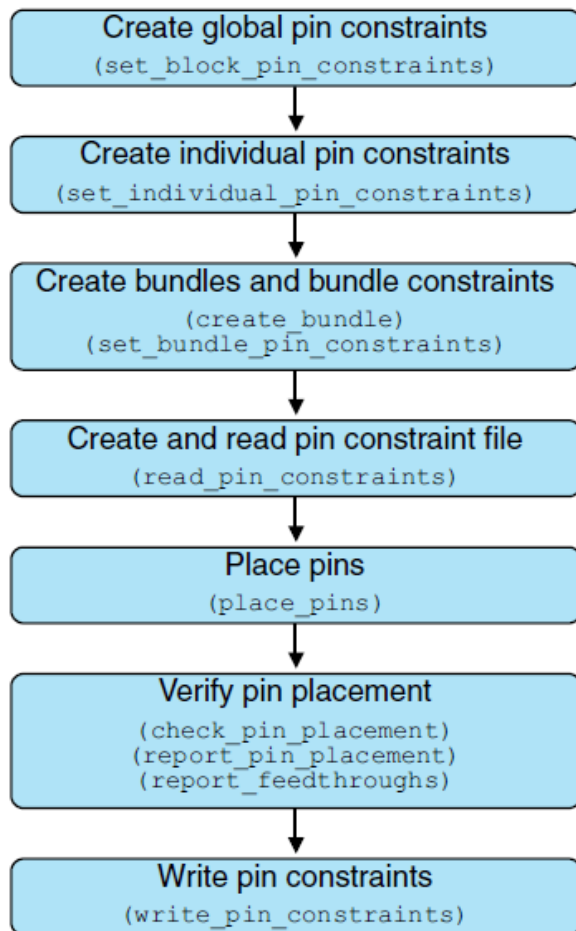
将“单元和相连线”应用到模块中：

1. push_down_objects [get_cells eco_*]



8. Pin 规划

基本流程如下图：



1). 全局约束

全局约束包括 pin 层、距离、模块（-cell）、边（-sides、-exclude_sides）、是否允许反馈（-allow_feedthroughs true | false，默认反馈通常情况和 multiply instantiated blocks（MIB）有效）。基本流程如下：（对所有 pin 有效）

1. **set_block_pin_constraints** -pin_spacing 1 -allowed_layers {M2 M3 M4 M5 M6 M7 M8 M9}
选项-corner_keepout_num_tracks tracks、-corner_keepout_distance distance 控制 pin 到 corner 的最小距离；-pin_spacing tracks、-pin_spacing_distance distance 约束 pin 之间的间距；-hard_constraints 当 pin 不在 track 上强制移动。
2. report_block_pin_constraints
（GUI-Task > Task Assistant，Pin Assignment > **Block-based** Constraints）

2). 分立约束

基本流程：

1. set_individual_pin_constraints
支持选项：-cells；-allowed_layers；-pins；-nets；-ports；-pin_spacing \$num_track；-pin_spacing_distance；-sides；-width {{层 *}}...-length {层 *}}...指定 pin 的形状（宽、长）；-offset（pin 与模块边界的最小距离）；-location；

2. report_individual_pin_constraints

(GUI-Task > Task Assistant, Pin Assignment > **Pin-based Constraints**)

3). 读取约束

读取: read_pin_constraints -file_name *;

报告读取的约束: report_individual_pin_constraints;

格式: 结尾用 “;”, 用 “#” 注释。

(1). 拓扑图

拓扑图是一种约束形式, 优先级高于全局 pin 约束。拓扑图中匹配模块 (顶层) 中线的摆放、绕线方式。

基本格式: (支持通配符 “*”)

start topological map;

```
{Nets net_names}
  {{Port port_name_1} {Cell cell_name_1}
  {Cell cell_name_1} {Cell cell_name_2}
  {Cell cell_name_2} {Pins pin_name_1}};
{Bundles bundle_names}
  {{Cell cell_name_1} {Cell cell_name_2}};
```

end topological map;

还有其他关键词 “side、offset、layer”; 边的号码按顺时针由 1 变大, 最左、最下的边为 1; 示例格式:

```
{{Cell cell_name_1} {sides side_number}
  {offset offset_1} {layers {layer_1 layer_2 ...}}}
```

示例: Net_A 经由 Cell_B 连接 Cell_A 和 Cell_C:

```
start topological map;
  {Nets Net_A}
  {{{Cell Cell_A} {Cell Cell_B}}
  {{Cell Cell_B} {Cell Cell_C}}
  {{Cell Cell_A} {Cell Cell_E}}};
end topological map;
```

示例: 两束线 Bundle1、Bundle2 连接 Cell_A、Cell_B:

```
start topological map;
  {Bundles Bundle1 Bundle2}
  {{Cell Cell_A} {Cell Cell_B}};
end topological map;
```

示例:

```
start topological map;
  {Nets Net_A}
  {{{Cell Cell_A} {sides 1}
  {offset 20.18} {layers M2}}}
```

```

    {{Cell Cell_B} {sides 3}
    {offset 40.28} {layers M4}}}
    {{{Cell Cell_B} {sides 1}
    {offset 30.28} {layers M4}}}
    {{Cell Cell_C} {sides 3}
    {offset 15.88} {layers M2}}};
end topological map;

```

(2). 控制反馈（Feedthrough）

当不允许反馈，使用“create_routing_corridor”规划走线通道，“place_pins”并不会创建反馈 pin，相关信号线在模块上层走线；拓扑图格式：

```

start feedthrough control;
{Nets net_name}
# Specify either Feedthrough or NoFeedthrough
# but not both for a given net
    {Feedthrough {cell_name_1 cell_name_2 cell_name_3}
    {NoFeedthrough {cell_name_4 cell_name_5 cell_name_6};
{Bundles bundle_name}
    {Feedthrough {cell_name_7 cell_name_8 cell_name_9}
    {NoFeedthrough {cell_name_10 cell_name_11 cell_name_12};
end topological map;
允许反馈，“place_pins”会在途径的模块上创建 pin，绕线时信号经由该模块。

```

(3). Pin 物理约束

约束位置，金属层，间距，顺序等。格式如下：

```

start physical pin constraints;
{Net net_name} {cell cell_name}
{layers {layer1 layer2}} {sides {side1 side2}}
    {offset offset1} {order {order_spec}}
    {start start_offset} {end end_offset}
    {width layer1 width1 layer2 width2}
    {length layer1 length1 layer2 length2};
{Pins pin_name} {reference reference_cell}
    {off_edge} {location {x y}};
end physical pin constraints;

```

(4). Pin 间距约束

```

start pin spacing control;
    {reference ref_cell_name} {sides {side_a side_b}}

```

```

    {layer_a spacing_a} {layer_b spacing_b};
end pin spacing control;

```

(5). 模块 Pin 约束

相关约束都可以使用 CMD “set_block_pin_constraints” 约束。

拓扑图约束支持关键词：reference、sides、exclude_sides、layers、feedthrough、spacing、spacing_distance、corner_keepout_distance、corner_keepout_num_tracks、hard_constraints；基本格式为：

```

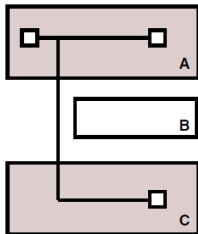
start block pin constraints;
{reference reference_name}
  {layers {layer_name_1 layer_name_2 ...}}
  {feedthrough true | false}
  {corner_keepout_num_tracks number_of_tracks}
  {spacing number_of_tracks}
  {hard_constraints {spacing location layer}}
  {exclude_sides {side_1 side_2}};
end block pin constraints;

```

4). 拓扑约束示例

当读取约束后，“place_pins” 摆 Pin。

例 1：如图连接



```
read_pin_constraints -file_name constraints.txt
```

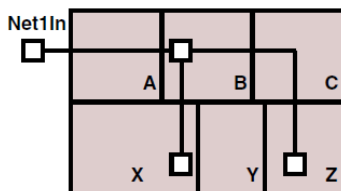
```

(start topological map;
  {Nets Net1}
  {Cell A} {Cell C};
end topological map;)

```

```
set_block_pin_constraints -allow_feedthroughs true
```

例 2：如图连接，部分约束，global router 依照设置自己判断怎么走线



```

start topological map;
  {Nets Net1}

```

```

        {Port Net1In} {Cell A}
        {Cell A} {Cell B};
    end topological map;
    start feedthrough control;
        {Nets Net1}
        {NoFeedthrough X Y};
    end feedthrough control;

```

5). 对 pin 的 Global Routing

Global Routing 是工具对 Pin 的模拟摆布，先执行此操作，在此基础上在调整 Pin 更方便，效果也往往较好。

对所有模块执行 global route:

1. `set_host_options -name distributed ...`
2. `route_global -floorplan true -host_options distributed -virtual_flat all_routing`; 如果事前已有自定义约束，使用“`route_global -use_existing_routing`”不变更原约束。

默认“`place_pins`”会按照自定义约束摆 Pin，可以遵照 Global Routing，使用“`-use_existing_routing`”；

在摆 Pin 之后，用“`check_feedthroughs -unused_feedthroughs`”检查定义但实际没有用到的反馈通道。

6). Channel Congestion Map

在 Global routing 之后，正式摆 Pin 之前，可以先评估模块/macro 之间的拥塞。例“`create_channel_congestion_map -channel_width_threshold 10`”评估间距小于 10 的通道。

GUI-View > Map > Channel Congestion。

也可以使用 **What-if Channel** 查看通道内绕线密度。

7). Placing Pins

执行 incremental pin placement，只对没有人工预摆好的 Pin 处理；`set_app_options -name plan.pins.incremental -value true`；需要设置事先“`remove_feedthroughs`”；优化过程中会在不改变边的情况下，调整位置，变更金属层以或得更好的 QoR。

摆 Pin 命令：`place_pins`

对“`place_pins`”有效的参数：

`plan.pins.exclude_clocks_from_feedthroughs`

`plan.pins.layer_range` range

`plan.pins.new_port_name_tag`

`plan.pins.new_port_name_tag_style` prefix | suffix

`plan.pins.pin_range` range:

`plan.pins.reserved_channel_threshold` width

`plan.pins.synthesize_abutted_pins`

是否允许时钟树上的反馈

指定优化时最大可以变更层数

反馈 pin 的标识符

新 Pin 标识符添加位置

指定优化时最大可以位移距离

为小于此值的通道预留绕线空间

当 true，新 Pin 创建在与原始相邻的模块

8). 检查 Pin Placement

1. 基础检查 “**check_pin_placement**”
2. 检查反馈 “**check_feedthroughs**” (常值传递、复用反馈、未用反馈等)
3. 检查 MIB “**check_mib_for_pin_placement**”; 例: `check_mib_for_pin_placement -swapped_connections -top_level_terminal_locations`
4. 将当前设置输出 “`report_pin_placement > pin_placement.rpt`”
5. 将反馈设置输出 “`report_feedthroughs -reporting_style net_based`”

9). 写出反馈型绕线指令

“`place_pins`” 和 “`push_down_objects`” 可产生反馈型绕线，为写出具体的 tcl 指令，使用 “**write_shadow_eco**”

示例: `write_shadow_eco -command_style icc2 -output feedthroughs.tcl`

10). 顶层/模块中交互

顶层 / 模块的交互分两种，一种是将顶层的东西下推到模块中 (`set_push_down_object_options`、`push_down_objects`)；一种是将模块中东西上推到顶层中 (`set_pop_up_object_options`、`pop_up_objects`)。

自顶推下:

1. 指定要推的东西。例: `set_push_down_object_options -object_type signal_routing -block_action {copy create_pin_shape}` (copy 说明上层的 objects 保留，默认移除)
2. 检查设置。例: `report_push_down_object_options -object_type signal_routing`
3. 检查下推的线是否存在潜在问题。例: `check_nets_for_push_down [get_nets -of [get_cells CLK_LV*]]`
4. 下推。例: `push_down_objects [get_nets -of [get_cells CLK_BUFFER*]]`

默认 `push_down_objects` 在存在反馈线的模块上创建 electrically equivalent (EEQ) pins，若要创建 feedthrough ports 使用 “`set_individual_pin_constraints -allow_feedthroughs true -nets [get_nets * -filter "net_type == signal"]`”

自低推上:

1. 设置上推的东西。例: `set_pop_up_object_options -object_type blockage -block_action keep`
2. 报告设置。例: `report_pop_up_object_options -object_type blockage`
3. 到模块级，收集将上推的东西。例: `current_design alu; set placement_blockages [get_placement_blockages]`
4. 到顶层。例: `current_design top`
5. 上推。例: `pop_up_objects -blocks [get_cells u1/u2] $placement_blockages`

通用说明:

可以上 / 下推的东西: `-object_type {pg_routing signal_routing routing_guide routing_corridor blockage pin_guide pin_blockage cells}`

指定顶层对推后东西处理: `-top_action keep | remove`

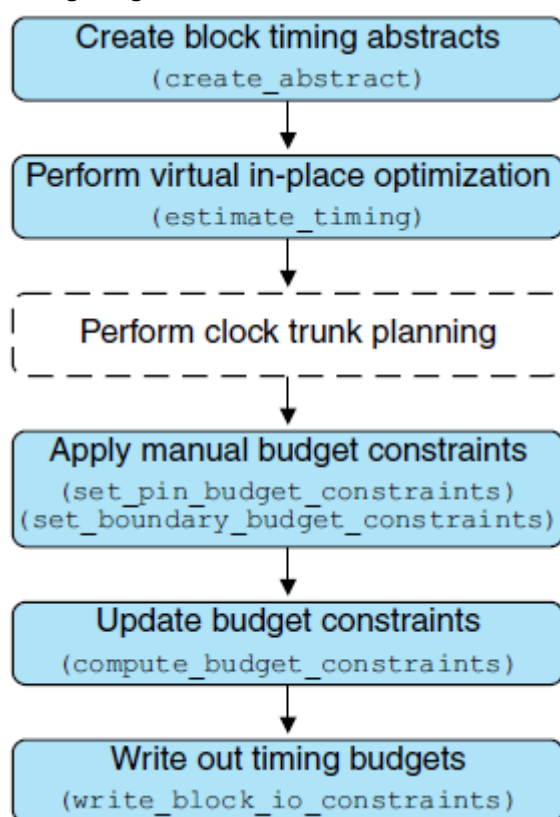
指定模块对退后东西处理: `-block_action {copy | keep | create_pin_shape}`

在推之前先检查时候是否有冲突: `-routing_overlap_check true | false`
当 MIB 情况存在不符时是否继续推: `-ignore_misalignment true|false`
时候根据位置、层命名新 Pin: `-location_based_terminal_naming true|false`

9. Timing Budget

层次化设计需要评估模块与顶层、模块与模块之间的时序。可以将芯片级约束切分成顶层和模块级约束。为加快评估速度，支持“lightweight timing abstract representations”，忽略高扇出线，精确评估端口时序。

Timing Budget 基本流程:



前提是模块的 Pin 已经摆好。使用“run_block_script”对所有模块同时操作。

1). 创建 Block Timing Abstracts

模块 Pin 需摆好。使用情况：顶层 full-design 时规划模块 Pin 及端口时序；顶层读模块 design view，评估 Pin 和端口，写出反馈数据。

1. 打开模块。例: `open_block leon3mp.ndm:leon3s.design`
2. 将 Abstract 中的修改应用到 design view 中:“merge_abstract”(例: 在 Abstract 中摆 Pin, 该指令可以将 Pin 信息传递到模块 design view 中并应用)
3. 读取模块约束。例: `source split/leon3s/top.tcl` (约束由 split_constraints 切出)

4. 插反馈型路径 buffer, add_feedthrough_buffers
5. 创建 Abstract。例: create_abstract -estimate_timing (对端口执行 virtual timing optimization, 并将数据反标到 Abstract) (默认在最底层创建, 故逻辑层次较多的设计需考虑到这点)
6. 保存。save_lib

2). 虚拟局部 (Virtual In-Place) 优化

这种优化策略对端口有效, 且不会对网表变更。

1. 打开顶层。例: open_block leon3mp.ndm:leon3mp
2. 读顶层约束。例: source ./split/leon3mp.cut/top.tcl
3. 优化。estimate_timing。默认会创建一个新 corner “estimated_corner”, 评估后的修改并不会影响其他 corner, 也就不会造成实际改变; 可以指定分析 corner, 使用 “-force_user_corner”。

可以事前设 “plan.clock_trunk.estimate_block_latency” 为 true, 这样 “estimated_corner” 就会对顶层时钟树进行大致分析, 并插 “anchor buffer0”。

4. 报端口路径时序。例: report_timing -modes [all_modes] -corners estimated_corner

Report : timing

-path_type full
-delay_type max
-nworst 1
-max_paths 1

Design : leon3mp

Startpoint: u0_1/p0/CLK (rising edge-triggered flip-flop)

Endpoint: u0_1/pd/cmemo0/CSB2 (falling edge-triggered flip-flop)

Mode: func

Corner: estimated_corner

Scenario: func

Path Group: pci_clk

Path Type: max

Launch clock: pci_clk r

Capture clock: pci_clk f

Point	Fanout	Cap	Trans	Incr	Path

clock pci_clk (rise edge)				0.00	0.00
clock network delay (ideal)				0.90	0.90
u0_1/p0/CLK (SDFFX2_RVT)			0.00	0.00	0.90 r
u0_1/p0/QN (SDFFX2_RVT)	4	30.77 A	0.15	0.17 A	1.07 f *
u0_1/U351/A2 (AND2X1_RVT)			0.06 A	0.00 A	1.08 f
...					
u0_1/U621/Y (NBUFFX8_RVT)	1	10.81 E	0.03	0.04 E	1.78 f *
...					

“A” 说明数据从 create_abstract -estimate_timing 中取得;

“E” 说明数据从 estimate_timing 取得;

“a” 说明数据从静态时序分析的 create_abstract -estimate_timing 中取得;

“S” 说明数据从 SPEF 或 SDF 中取得。

3). 自定义 Budget 约束

命令“**compute_budget_constraints**”会根据延时分布写出模块的端口约束（包括边界的 port 的约束）。也可自己写模块的约束，应用后再 **create_abstract**，在到顶层评估。

自定义设置：set_pin_budget_constraints（对模块 port）

例：set_pin_budget_constraints [get_pins u0_0/rstn] -internal_percent 40，顶层将这条路径的 40%延时作为 u0_0/rstn 内部延时。

自定义设置：set_boundary_budget_constraints（对顶层 port）

例：set_boundary_budget_constraints -name OutputLoad -default -load_capacitance 0.15
set_boundary_budget_constraints -name OutputLoad -corner RCworst -load_capacitance 0.15
set_boundary_budget_constraints -name driving_cell_buf -driving_cell NBUFFX2_RVT
set_pin_budget_constraints -late_boundary driving_cell_buf -inputs u0_0/*（-late_boundary 后加之前“set_boundary_budget_constraints”定义的约束，late 说明对 setup 有效；early 对 hold 计算有效）

自定义设置：set_latency_budget_constraints。自动计算“compute_budget_constraints -latency_targets actual”。Latency（延时）绝对值并不是十分重要，但是信号之间的相对时间可能会影响 Data path。这里 late 和 early 也是对应 setup（max path）和 hold（min path），当设不同值时，相当于 OCV 加紧。

例：set_latency_budget_constraints -corner RCworst -early_latency 5 -late_latency 5 -clock clk，计算 budgeting 时，clk 的 latency 是 5

4). 刷新 Budget 信息

当顶层执行完 virtual in-place optimization 和对模块的 abstracts 后，可以更新 Block timing budgets 信息。

一般模块的约束包括：

set_input_delay、set_output_delay

set_driving_cell、set_load

在 data path I/O 定义 Virtual Clock

Virtual clock 的 latency（source、network）

实际时钟的 source latency

生成模块约束：

1. 指定评估的模块。例：set_budget_options -add_blocks {u0_0 u0_1 u0_2 u0_3}
2. 产生约束并更新。例：compute_budget_constraints -estimate_timing；如果之前已优化，只需计算更新，执行 compute_budget_constraints。

命令 compute_budget_constraints 支持筛选（slack，feedthrough 等）

3. 报告。例：report_budget -html budget_report.html

例：report_budget -through u0_0/n114__FEEDTHRU_0

Report : report_budget -through u0_0/n114__FEEDTHRU_0

Module : leon3mp

Mode : func

Path: (S= 1.031 / D= 2.303(0.70) / B= 3.333) pci_clk.top_r->clk_r

From top level startpoint

Segment (S= 0.182 / D= 0.248(0.00) / B= 0.430) 13%

Through input pin u0_0/n114__FEEDTHRU_0

Segment (S= 0.226 / D= 0.385(0.00) / B= 0.612) 18%

Through output pin u0_0/n114__FEEDTHRU_2

Segment (S= 0.045 / D= 0.015(0.00) / B= 0.060) 2%

...

例: icc2_shell> report_budget -warning_pins > warning_pins.txt

icc2_shell> shell head warning_pins.txt

Report : Budgeted pins with no timing paths

Module : leon3mp

Mode : func

u0_0/ahbi[31]

u0_0/ahbi[30]

u0_0/ahbi[29]

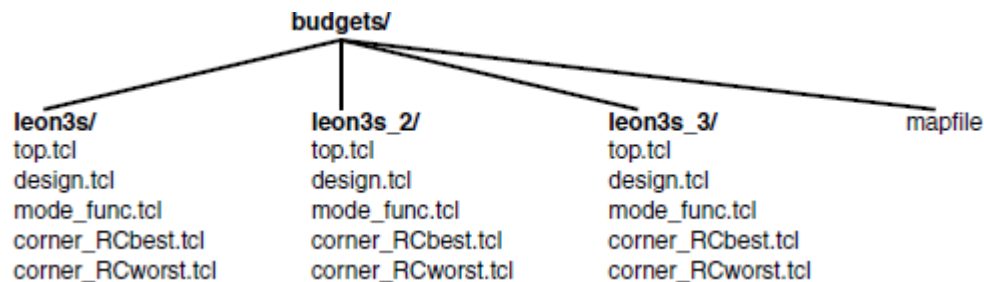
...

5). 写出 Budget 设置

例: write_budgets -blocks {u0_0 u0_1 u0_2 u0_3} -force (覆盖) (-all_blocks)

默认 write_budgets 写出 set_budget_options -add_blocks { * }中定义的模块。

生成的结构目录为:



模块的约束应先读 “split_constraints” 创建的，再读 “write_budgets”。

1. 读 constraint mapping。例: set_constraint_mapping_file split/mapfile

icc2_shell> shell cat split/mapfile

leon3s SDC leon3s/top.tcl

leon3s UPF leon3s/top.upf

...

2. 读 “split_constraints” 约束。例: source split/leon3s/top.tcl

3. 读 budgets mapping。例: set_constraint_mapping_file budgets/mapfile

icc2_shell> shell cat budgets/mapfile

leon3s BUDGET leon3s/top.tcl

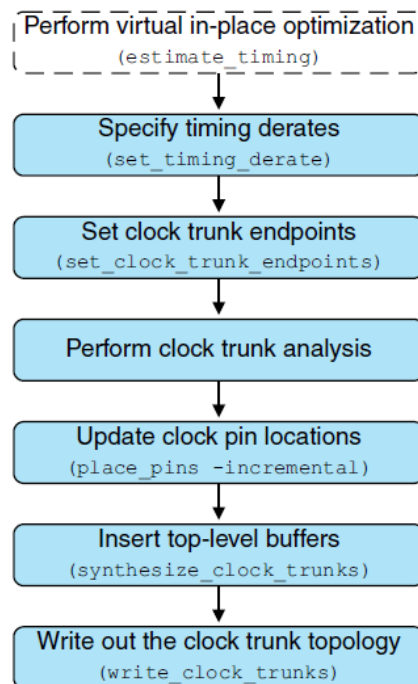
leon3s_2 BUDGET leon3s_2/top.tcl

leon3s_3 BUDGET leon3s_3/top.tcl

4. 读 I/O budget 约束。例：source budgets/leon3s/top.tcl

10. 时钟主干（clock trunk）规划

为精准控制时钟网络，建议执行时钟主干规划。该规划分析 global clock network，说明 worst-case clock paths，并提供对其的修复。基本规划流程：



1). 为时钟数规划的虚拟局部优化

为进行时钟树规划，需要先执行“timing budgeting”。

1. set plan.clock_trunk.estimate_block_latency true; 开启“estimate_timing”对时钟树的评估
2. estimate_timing

2). 指定 Derates 和时钟 Endpoints

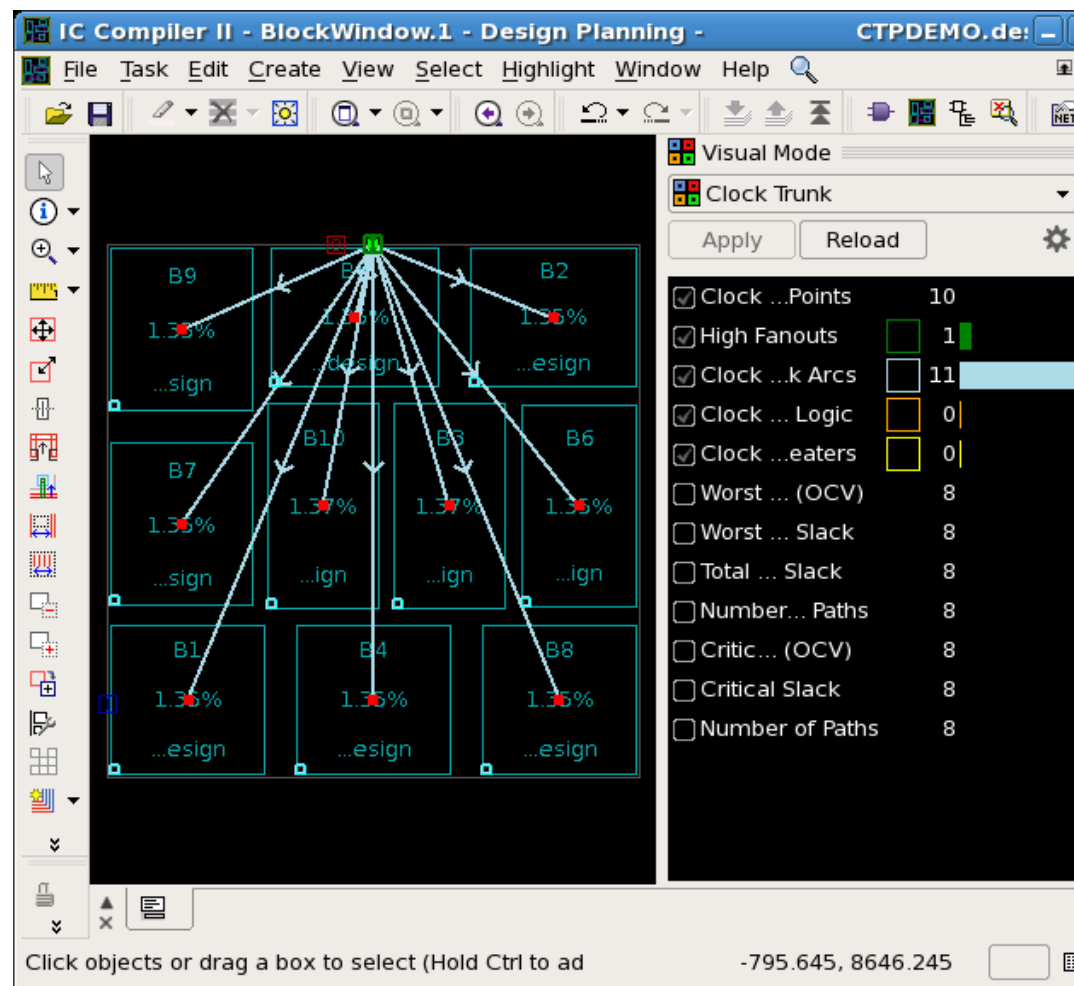
在预插 clock anchor cells 后设置。

1. set_timing_derate -early 0.95 -net_delay
set_timing_derate -late 1.05 -net_delay
2. set_clock_trunk_endpoints {u1/clk1}。设置时钟树到各模块的时钟 Port

3). 执行时钟树分析

在 GUI 中插看时钟分线，插 buffer，查看时序路径。GUI: Design Planning > Clock Trunk

Planning > Clock Trunk Planning



可以查看 Clock trunk end points、High-fanout clock segments、Clock trunk arcs、Clock trunk logic、Clock trunk repeaters、Worst negative slack (OCV)、Total negative slack、Number of violating paths、Critical slack (OCV)、Critical slack、Number of paths

主要分析在 GUI 上完成。

4). 更新时钟 Pin 位置

调整某些 Pin 位置，之后在新移动的基础上重新摆 Pin。

1. set_app_options -name plan.pins.incremental -value true
2. place_pins

5). 在顶层插时钟树 Buffer

在模块与模块之间的时钟路径可能比较差，可以在其间插 buffer 以优化。

1. Design Planning > Clock Trunk Planning > Clock Trunk Planning, 点击Manual Planning
2. GUI选择模块，buffer将插在其中
3. 点击Add buffer

“`synthesize_clock_trunks`”会自动分析并插buffer。

6). 写出时钟树位置

```
write_clock_trunks -clock clk -file clocktrunk.tcl
```